



Universidad Abierta Interamericana

Facultad de Tecnología Informática

Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Gestión del proyecto

Plan de sprints, hoja de ruta, lo que falta y cambios sobre el documento original

Alumno: Pettini, Patricio Ezequiel

Legajo: B00072710-T1

Sede: Sede Norte — Boulogne

Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 17/09/2026

Índice

Plan de sprints.....	9
Cómo se gestiona.....	9
Definición de Terminado.....	10
Definición de Preparada	10
Sobre los sprints cerrados.....	10
Épicas	11
S1 · Relevamiento, arquitectura y entorno [x]	11
S2 · Modelo de datos, DAO y patrones [x].....	12
S3 · Seguridad [x].....	12
S4 · Núcleo funcional [x]	13
S5 · Predictivo, auditoría de arquitectura y CI [~] en curso	14
Terminadas	14
Pendientes del sprint	15
S6 · Multidioma en la interfaz y sistema visual.....	15
H-38 · Toda la interfaz sale del diccionario.....	16
H-39 · Sistema visual y componentes	16
S7 · Pantallas, calidad y deuda.....	17
H-40 · Las pantallas rediseñadas.....	17
H-41 · Los CP-01..CP-14 son una suite identificable	17
H-42 · El RNF de carga está medido.....	17
H-43 · Partir IAdministracionBusiness (E9)	17
H-44 · Las pruebas no dejan residuo.....	18
S8 · Endurecimiento, producto y documentación	18
H-45 · Revisión contra OWASP Top 10.....	18
H-46 · ADR sobre gestión de identidad.....	18
H-47 · Estados vacíos y esqueletos de carga	18
H-48 · Panel de calibración de reglas.....	19
H-49 · Los manuales	19

H-50 · El documento del TFI al día	19
Cierre · Entrega	19
H-51 · El horizonte financiero, resuelto	19
H-52 · Ensayo de la defensa	19
H-53 · Congelamiento	19
S10 · Correcciones finales	21
Métricas del proceso.....	21
Riesgos	22
Tablero	22
Hoja de ruta	23
Calendario	23
Avance por épica.....	24
Reparto del backlog	24
Sprints	25
[x] S1 · Relevamiento, arquitectura y entorno	25
[x] S2 · Modelo de datos, DAO y patrones.....	25
[x] S3 · Seguridad.....	26
[x] S4 · Nucleo funcional	27
[~] S5 · Predictivo, auditoria de arquitectura y CI.....	27
[] S6 · Multiidioma en la interfaz y sistema visual	28
[] S7 · Pantallas, calidad y deuda.....	29
[] S8 · Endurecimiento, producto y documentacion	29
[] S9 · Cierre y entrega.....	30
[] S10 · Correcciones finales.....	30
Cómo se lee esto	31
Lo que falta	33
Fechas	33
1 · La Entrega 1 de negocio, del viernes 18/09	33
2 · Lo que se puede hacer hoy	34

3 · Lo que depende de una fecha.....	34
4 · Lo que los dos módulos nuevos dejaron anotado	35
5 · Lo que está declarado y no es deuda escondida	35
Retomar el trabajo en otra máquina	36
Cómo se mantiene este documento.....	36
Cambios sobre el documento original	38
D-01 · Entorno del sistema: Java a C#.....	38
D-02 · Multi-organización en el DER	39
D-03 · Alcance de la IA: sumar clasificación / triage	39
D-04 · Requerimiento funcional nuevo para el triage	40
D-05 · Pantallas del sistema	41
D-06 · Diagramas de arquitectura.....	41
D-07 · Anexo de transparencia sobre el uso de IA.....	41
D-08 · Requerimientos no funcionales de carga y seguridad	42
D-09 · Patrones de resiliencia en la arquitectura base	42
D-10 · El plan comercial pasa a ser parte del sistema	43
D-11 · La asignación distingue IA de heurística	44
D-12 · Tres atributos que el motor predictivo necesita.....	45
D-13 · El estado de la incidencia avanza al asignarse	45
D-14 · Precios de los planes y sueldos del equipo	46
D-15 · Dos inconsistencias menores detectadas al construir	48
D-16 · Cifrado reversible y clave de API por organización	49
D-17 · Cuándo corre el análisis predictivo	49
D-18 · La patente lleva nivel de acceso.....	50
D-19 · La bitácora se filtra por usuario y los errores tienen su vista	50
D-20 · El ciclo de vida completo de la incidencia	51
D-21 · Los reportes son PDF generados en el servidor	51
D-22 · Cuatro casos de uso que el modelo de permisos prometía.....	52
D-23 · Valor terminal en la evaluación financiera	53

D-24 · Las pantallas del capítulo 10.5 son capturas del sistema	53
D-25 · Los diagramas de paquetes, componentes y despliegue salen del código.....	54
D-26 · Numeración de figuras y tablas, e índices propios	54
D-27 · Los documentos satélites toman el formato del documento principal	55
D-28 · Rediseño de la interfaz y los temas.....	56
D-29 · Las pantallas del 10.5 se describen, no sólo se muestran	58
D-30 · Verificación: OWASP, cobertura y carga	58
D-31 · ADR 0014: gestión de identidad propia	59
D-32 · Los diagramas salen de Enterprise Architect	59
D-33 · El modelo se recalibra: macro del REM y estructura financiable	60
D-34 · El modelo dejaba dos supuestos afuera	62
D-35 · El presupuesto se alinea con el material de la cátedra.....	64
D-36 · Alcance de la IA: la guía de reparación paso a paso	66
RF-15 · Sugerencia de procedimiento de reparación	66
CU-018 · Consultar la guía de reparación sugerida	66
D-37 · Las dos metas que no coincidían con el modelo, resueltas	68
La meta de equipos: el Excel estaba mal, no el documento	68
La participación de mercado: punto intermedio	69
D-38 · El embudo se vuelve autogestionado y el universo se deriva	69
El embudo: sale la demostración en vivo	69
El universo de PyMEs: se deriva de datos públicos	70
D-39 · D-32 decía «aplicado» y solo cuatro diagramas lo estaban.....	71
D-40 · Fuera los emojis y las flechas tipográficas	72
D-41 · La sección financiera se contradecía a sí misma	73
D-42 · Lo que encontró la revisión de coherencia	73
D-43 · El multiidioma no cumplía lo que el requisito promete	75
D-44 · Lo que quedaba abierto de OWASP, y la capa de negocio	76
D-45 · El mantenimiento deja de ser sólo un registro: entra el plan y la agenda	77
RF-16 · Planificación del mantenimiento preventivo	77

CU-019 · Definir un plan de mantenimiento.....	77
CU-020 · Gestionar la agenda de mantenimiento	78
D-46 · La facturación del servicio: el modelo de ingresos entra al sistema	79
RF-17 · Facturación del servicio	79
CU-021 · Facturar el período.....	80
D-47 · La fecha de la Entrega 2 de negocio, corregida en el sentido contrario.....	81
D-48 · D-47 estaba mal: la lámina se leyó cruzada, no la fecha	81
D-49 · Distrito Tecnológico y derechos de exportación, declarados como no aplicables	82
D-50 · Los tres diagramas de secuencia que faltaban, en 10.4.4	83
D-51 · El 18/09 era la entrega, no la devolución	83
D-52 · La reasignación manual no reasignaba, y el CP-08 no lo veía.....	84
D-53 · El RNF-05 pasa a ser verificable	85
D-54 · El frontend se alinea al prototipo, y el prototipo se corrige	86
D-55 · Las figuras pierden el epígrafe y ganan texto alternativo.....	90
D-56 · Los vínculos de los DER dejan de cruzar por encima de las tablas	91
D-57 · El documento del TFI también pierde los epígrafes de figura	91
D-58 · Los diagramas del documento estaban cuatro versiones atrás.....	92
D-59 · La barra de color al costado de la tarjeta se va.....	93
D-60 · Dos defectos del tablero, encontrados revisando lo recién escrito	94
D-61 · Todo el paquete toma el formato del trabajo de 3.º año.....	94
D-62 · Fuera también los epígrafes de tabla.....	95
D-63 · Las referencias pasan de APA 6 a APA 7	96
D-64 · La línea del encabezado se cortaba justo arriba del escudo.....	96
D-65 · Las 24 capturas del 10.5 eran anteriores al rediseño del frontend	97
D-66 · El diagrama general de casos de uso, rehecho	97
D-67 · El paquete deja de tener figuras chicas y tablas mal cortadas	98
D-68 · Ningún vínculo pasa ya por encima de una entidad	99
D-69 · Qué archivo es el último, después de un git clone	100
D-70 · El sistema pasa a estar desplegado y accesible	100

D-71 · El entorno de demostración no es la infraestructura prevista	101
D-72 · El entorno de demostración pasa a Microsoft Azure.....	102
D-73 · El plan de negocio vendía cuatro capacidades y el sistema tiene seis	103
D-74 · El documento saltaba del capítulo 7 al 10: se escriben el 8 y el 9	104
D-75 · La estrategia de producto se contradecía consigo misma	105
D-76 · La meta de equipos quedó en dos números distintos dentro del documento.....	106
D-77 · El capítulo 9 prometía algo que la pantalla declara fuera de alcance	107
D-78 · El capítulo 9 tenía seis apartados y el índice de la cátedra pide cuatro	108
Fuentes.....	109

Plan de sprints

Autor: Patricio Pettini · **Legajo:** B00072710-T1

Período: 06/07/2026 – 28/11/2026 · **Ocho sprints de dos semanas, la semana de cierre y el tramo de correcciones finales**

Congelamiento propio: viernes 30/10/2026 · **Entrega final a la cátedra:** miércoles 11/11/2026

Sprints cerrados	S1 a S4
Sprint en curso	S5 (31/08 – 11/09)
Sprints planificados	S6 a S8, cierre (S9) y correcciones finales (S10)

El trabajo se adelantó al calendario. Las fechas de cada sprint son las que se planificaron en julio y no se retocan; los estados de las historias son lo que efectivamente está hecho. Al 09/09/2026 están terminadas todas las de S6, S7, S8 y S10, y las dos que quedan —el ensayo de la defensa y el congelamiento— dependen de fechas que todavía no llegaron. Se deja así, con la diferencia a la vista, en lugar de correr las fechas hacia atrás para que coincidan: un cronograma que se reescribe cada vez que se cumple no sirve para medir nada.

Cómo se gestiona

Scrum adaptado a un equipo de una persona. La adaptación es explícita, no un recorte por comodidad:

Ceremonia	Cómo se aplica	Por qué así
Planificación	Una hora al inicio del sprint	Comprometer alcance, no llenar horas
Daily	Nota diaria de una línea en el tablero	Sin equipo no hay sincronización que hacer; lo que sí hace falta es el registro de en qué se fue el día

Ceremonia	Cómo se aplica	Por qué así
Revisión	Demostración sobre el sistema corriendo	Es la única evidencia que vale: código que compila no es software que anda
Retrospectiva	Media hora, con una acción concreta	Sin acción, la retro es una queja

Tres niveles: épica a historia a tarea. La historia describe un cambio observable para alguien. La tarea es trabajo técnico que por sí solo no se le puede mostrar a nadie.

Definición de Terminado

Una historia está terminada cuando cumple **todas**:

1. Compila sin advertencias — el CI usa `-warnaserror`.
2. Tiene pruebas que fallarían si el cambio se revirtiera.
3. `dotnet test` y `npm test` pasan **enteros**, no sólo lo nuevo.
4. Se verificó **sobre el sistema corriendo**, no leyendo el código.
5. Si contradice o precisa el documento del TFI, quedó en `docs/CAMBIOS-DOCUMENTO.md`.
6. Si fue una decisión de arquitectura, tiene su ADR.
7. Commiteada con un mensaje que explica **por qué**, no qué.

El punto 4 no es decorativo, y es la lección más caras del proyecto. Los dos defectos más serios encontrados hasta hoy —la bitácora que no registraba ningún evento de negocio, y el motor predictivo que no se disparaba solo— eran **ausencias**, no errores. Nada fallaba, ninguna prueba se ponía en rojo, y el código leído se veía correcto. Los dos aparecieron usando el sistema.

Definición de Preparada

Una historia entra a un sprint cuando tiene criterios de aceptación verificables —«pasa X, entonces Y», no «que ande bien»— y cuando se sabe **cómo se va a demostrar**.

Sobre los sprints cerrados

S1 a S4 se documentan de forma retrospectiva: la planificación formal se incorporó durante S5. Lo que sí es verificable de esos sprints es el resultado —los ADR firmados, el esquema, las pruebas que corren y el código— y es lo que se lista en cada uno. El proceso formal, con tablero y ceremonias, corre desde S5.

Épicas

#	Épica	Por qué existe	Sprints
E1	Arquitectura y entorno	Es lo que evalúa la cátedra y condiciona todo lo demás	S1–S2
E2	Seguridad	Req. Arq. 002 y 004, y el modelo Familia/Patente	S3, S5, S8
E3	Núcleo funcional	Los CU-001 a CU-013 del documento	S4
E4	Análisis predictivo e IA	Es lo que distingue al sistema de un ticketing	S5
E5	Integración continua y entorno reproducible	Que la verificación no dependa de que yo me acuerde	S5
E6	Multiidioma e interfaz	Req. Arq. 001, y la defensa visual	S6–S7
E7	Calidad demostrable	Que un tercero pueda ver que las pruebas cubren lo que el documento promete	S7
E8	Producto y documentación	Es la mitad de la nota	S8, cierre
E9	Deuda técnica	Lo que sé que está mal y todavía no arreglé	S5, S7

S1 · Relevamiento, arquitectura y entorno [x]

06/07 – 17/07 · Épica E1

Objetivo. Decidir la arquitectura y dejarla firmada antes de escribir la primera línea de negocio.

Historia	Resultado
H-01 · Migrar el stack de Java/Spring a C#/.NET	ADR 0002. Delta D-01 sobre la sección 10.2.5
H-02 · Separar negocio de seguridad en dos bases	ADR 0003. Dos cadenas de conexión; un DAO de negocio no puede leer la de seguridad
H-03 · Acceso a datos con DAO explícito	ADR 0004. Se descartó EF Core: vacía de contenido la capa DAO y el Unit of Work, que son objeto de evaluación
H-04 · Multi-organización desde el modelo	ADR 0005. Delta D-02 sobre el DER
H-05 · Esqueleto de las cinco capas y la base en Docker	Domain / DAO / BLL / Service / Api, dependencias sólo hacia abajo. SQL Server 2022 con volumen y healthcheck

Retrospectiva. La decisión que más rindió fue escribir los ADR antes del código: cuando después apareció la tentación de usar EF Core «para ir más rápido», el motivo por el cual no ya estaba escrito.

S2 · Modelo de datos, DAO y patrones [x]

20/07 – 31/07 · Épica E1

Objetivo. Las dos bases aplicadas y el acceso a datos completo, con los patrones que la cátedra evalúa en uso real.

Historia	Resultado
H-06 · Esquema de negocio	16 tablas. Toda tabla lleva id_organizacion
H-07 · Esquema de servicio	Usuario, Rol, Familia, Patente, Bitácora, Idioma, Traducción, Backup
H-08 · DAO genérico y mappers	IGenericDao<T> heredado por tres interfaces reales; IMapper<T> con 12 implementaciones
H-09 · Fábricas y unidad de trabajo	Abstract Factory (FactoryDao, FactoryBusiness) y SqlTransactRepository: el número correlativo de incidencia y la incidencia van juntos o no van
H-10 · Aislamiento por organización en el DAO	El identificador de organización no viaja en la petición : se resuelve del contexto de sesión, así ningún controlador puede pedir datos de otra
H-11 · Pruebas de integración del DAO	Contra el SQL Server del contenedor, no contra un doble: el sistema usa disparadores, MERGE y BACKUP/RESTORE

Defectos del sprint. Tres invisibles al leer, que sólo aparecieron ejecutando: PLAN y BACKUP son palabras reservadas de T-SQL; un VARCHAR(10) truncaba 'ADVERTENCIA'; y los índices filtrados fallan si sqlcmd no recibe -I, porque deja QUOTED_IDENTIFIER en OFF.

Retrospectiva. Acción tomada: los scripts de base se **ejecutan**, nunca se revisan. Se sostiene desde entonces.

S3 · Seguridad [x]

03/08 – 14/08 · Épica E2

Objetivo. Req. Arq. 002 y 004, y el modelo de permisos de la cátedra.

Historia	Resultado
H-12 · Contraseñas	PBKDF2-HMAC-SHA256, 120 000 iteraciones, comparación en tiempo constante
H-13 · Familias y patentes con Composite	ComponenteSeguridad con nodo y hoja, familias anidadas y guarda de ciclos
H-14 · Autenticación con JWT	El token lleva usuario y organización y no las patentes: si las llevara, revocar un permiso no tendría efecto hasta que expirara
H-15 · Autorización por patente	[RequierePatente], con las patentes leídas de la base en cada petición
H-16 · Bitácora inmutable	Disparador INSTEAD OF UPDATE, DELETE que rechaza con error 50001. No alcanza con no exponer la operación en la aplicación
H-17 · Respaldos	BACKUP con CHECKSUM y verificación con RESTORE VERIFYONLY

Defectos del sprint. Una dependencia circular IContextoSesion a ISeguridadService a IFactoryDao a IContextoSesion: el login funcionaba y la primera petición autenticada colgaba. Se resolvió partiendo IFactoryDaoSeguridad, que no depende del contexto.

Retrospectiva. El [RequierePatente] original aceptaba una sola patente y daba 403 al administrador en INCIDENCIA_VER_PROPIAS. Acción: los permisos se verifican **con un usuario de cada perfil**, no sólo con el administrador.

S4 · Núcleo funcional [x]

17/08 – 28/08 · Épica E3

Objetivo. Los casos de uso del documento, de punta a punta y con interfaz.

Historia	Resultado
H-18 · Inventario de activos	ABM, filtros, detalle con historial
H-19 · Incidencias de punta a punta	Alta, clasificación del texto libre, asignación, reasignación manual con la regla de permisos del documento
H-20 · Mantenimientos	Preventivos, correctivos y predictivos, vinculados al equipo
H-21 · Las 19 pantallas	Panel de administración y módulo del cliente
H-22 · Seis temas y tres densidades	Sustitución de tokens: ningún componente conoce el tema activo
H-23 · Contraste AA medido	12 fallas por tema claro, incluido el original. Cero fallas al cerrar, peor par en 4.9

Defectos del sprint. Dos del mismo tipo, y peligrosos porque el SQL se leía correcto: el *join* de técnicos pasaba por `Usuario_Organizacion` y devolvía cero técnicos, y `UsuarioDao.PorOrganizacion` tenía el mismo sesgo, con lo cual los nombres de los Partner salían vacíos.

Retrospectiva. Acción: toda consulta con JOIN sobre tablas de relación se prueba con **datos de los dos lados**, incluido el caso de la fila que no existe.

S5 · Predictivo, auditoría de arquitectura y CI [~] en curso

31/08 – 11/09 · Épicas E4, E5, E2, E9

Objetivo. *Que el análisis predictivo funcione solo, que los requisitos de arquitectura de la cátedra estén todos cubiertos, y que la verificación deje de depender de que yo me acuerde de correrla.*

Terminadas

Historia	Resultado
H-24 · Motor predictivo con estrategias	Cinco <code>IEstrategiaRegla</code> , score ponderado por criticidad. ADR 0007
H-25 · Proveedor de IA conmutable	<code>IProveedorIA</code> con implementación real y simulada. ADR 0006. Deltas D-03, D-04
H-26 · Disyuntor sobre el servicio externo	Decorator + Circuit Breaker. ADR 0009. Es la demostración del CP-14
H-27 · Multiidioma en el backend	179 claves × 2 idiomas. El diccionario se sirve sin autenticar , porque la pantalla de login también se traduce
H-28 · Cifrado reversible	AES-256-GCM, separado del hash. ADR 0010. La clave de API pasa a guardarse cifrada por organización
H-29 · Patente.TipoAcceso	11 patentes de lectura, 8 de escritura, 7 de administración. Delta D-18
H-30 · La bitácora registra los eventos de negocio	Ocho de nueve códigos no existían en el catálogo y se descartaban en silencio. Corregido, con dos pruebas que lo sostienen
H-31 · El motor se dispara solo	Por hecho y por barrido programado. ADR 0011. Delta D-17
H-32 · Bitácora por usuario y pantalla de errores	CU.Arq.002 y CU.Arq.007. Delta D-19
H-33 · Bulkhead (E9)	Estaba comprometido en el ADR 0009 y no estaba en el código

Historia	Resultado
H-34 · El sistema entero en Docker	docker compose up --build: base, esquema, API y frontend detrás de nginx, sin .NET ni Node en la máquina
H-35 · Integración continua	GitHub Actions, tres trabajos: backend contra SQL Server real, frontend, y las dos imágenes

Pendientes del sprint

Historia	Criterio de aceptación
H-36 · La bitácora de demostración en un solo huso	Las filas previas al arreglo del reloj están en UTC y ordenan por encima de las nuevas. El disparador impide reescribirlas —correctamente—, así que el entregable es el procedimiento de reinicio de la base de demo, documentado y probado
H-37 · Publicar la cobertura como línea de base	El CI la publica y el porcentaje queda en el README. Sin umbral que bloquee todavía: primero se mide. Un umbral elegido sin datos se cumple escribiendo pruebas que no prueban nada

Defectos del sprint. Los dos peores de todo el proyecto, y los dos fueron **ausencias**: la bitácora sin eventos de negocio (apareció contando filas por tipo de evento, no leyendo código) y el motor que no se disparaba solo (lo reportó el usuario). Ninguna prueba podía fallar: no había código roto, faltaba código.

Y tres del entorno, invisibles hasta ejecutar: los patrones de `.dockerignore` **no son recursivos**, así que el `obj/` de los seis proyectos entraba a la imagen y el publish fallaba con un error que apuntaba a NuGet; la imagen `mssql-tools18` no existe; y la imagen de runtime de `.NET` no trae `curl` ni `wget`, con lo cual un healthcheck que los use deja el contenedor en *unhealthy* para siempre sin que nada esté roto.

Más una que importa al negocio: el contenedor corre en UTC, y el sistema mide un SLA de cuatro horas. Tres horas de corrimiento cambian si una incidencia está en término o vencida.

Retrospectiva. Acción tomada: **auditar contra datos, no contra el código**. Los dos hallazgos serios salieron de contar filas y de usar el sistema. Se incorpora al Definición de Terminado como el punto 4.

S6 · Multiidioma en la interfaz y sistema visual

14/09 – 25/09 · Épica E6

Objetivo. Cerrar Req. Arq. 001 de punta a punta y aplicar la primera mitad

del rediseño.

H-38 · Toda la interfaz sale del diccionario

Como usuario, *quiero* cambiar el idioma y que cambie **toda** la pantalla, *para* usar el sistema en el idioma que elijo.

Criterios de aceptación

- Ningún texto visible está escrito en un componente: todos salen de t(clave).
- El selector cambia la interfaz sin recargar.
- La preferencia sobrevive a cerrar y volver a entrar.
- La pantalla de inicio de sesión se traduce **antes** de que haya sesión.
- Una clave faltante cae al idioma por defecto y aparece en el panel de cobertura.
- **Una prueba automatizada falla si un componente trae texto literal.**

Tareas

- T-38.1 Proveedor de i18n que consuma el endpoint anónimo del diccionario.
- T-38.2 Reemplazar los textos de las 19 pantallas.
- T-38.3 Selector de idioma en Apariencia y en el encabezado.
- T-38.4 Panel de cobertura de traducción, que el backend ya expone.
- T-38.5 La prueba de literales. Es la misma técnica que ya evitó que volvieran los códigos de bitácora sueltos: lo que el compilador no puede agarrar, lo agarra una prueba.

H-39 · Sistema visual y componentes

Criterios

- Escala de espaciado, tipografía, radios y elevación, como tokens.
- Los seis temas siguen funcionando sin que ningún componente conozca el activo.
- Cero fallas de contraste AA, **medidas** y no estimadas.
- Las tablas de registro conservan su densidad: espejan el formulario en papel que el técnico tiene al lado mientras carga.

Riesgo declarado: depende de una entrada externa (el resultado de la herramienta de diseño). Si no llega, se adelanta H-43 y H-39 pasa a S7.

Compromiso: H-38, H-39.

S7 · Pantallas, calidad y deuda

28/09 – 09/10 · Épicas E6, E7, E9

Objetivo. Terminar el rediseño y que la calidad se pueda auditar.

H-40 · Las pantallas rediseñadas

Dashboard, análisis predictivo, detalle de incidencia y mis equipos, en claro y oscuro, con los estados vacío, cargando y error.

H-41 · Los CP-01..CP-14 son una suite identificable

Como evaluador, *quiero* correr los casos de prueba del documento por su número, *para* verificar que el sistema hace lo que el capítulo 10 promete.

Criterios

- `dotnet test --filter "CP=CP-07"` corre exactamente ese caso.
- `dotnet test --filter "Categoria=CasosDePrueba"` los corre a todos.
- Cada prueba nombra el caso del documento que cubre.
- Los no cubiertos se identifican y se escriben.
- La tabla CP a prueba a archivo se **genera**, no se escribe a mano.

H-42 · El RNF de carga está medido

El ADR 0008 compromete 500 activos y 5 000 incidencias con respuesta bajo dos segundos. **Está afirmado y no medido.**

Criterios

- Un generador con semilla fija carga ese volumen.
- Se miden el listado, el panel predictivo y el dashboard: tres corridas, mediana.
- El resultado se publica **sea cual sea**. Si no cumple, el hallazgo es el entregable y se abre la historia de optimización. Un RNF incumplido y declarado es defendible; uno afirmado sin medir, no.

H-43 · Partir IAdministracionBusiness (E9)

Hoy tiene 18 métodos de cinco temas —mantenimientos, usuarios, bitácora, IA, respaldos—. Todo consumidor depende de las cinco cosas: es una violación de segregación de interfaces, y es propia.

Criterio: cuatro interfaces por tema, cada consumidor dependiendo sólo de la

suya, sin cambiar comportamiento y con las pruebas pasando igual.

H-44 · Las pruebas no dejan residuo

Correr la suite dos veces seguidas da el mismo resultado y no agrega filas a la base de demostración.

Compromiso: H-40, H-41, H-42, H-43, H-44.

S8 · Endurecimiento, producto y documentación

12/10 – 23/10 · Épicas E2, E8

H-45 · Revisión contra OWASP Top 10

Criterios

- Cada punto tiene una fila: cómo está mitigado o por qué no aplica.
- Los hallazgos se corrigen o se documentan como riesgo aceptado, con el motivo.
- Queda como anexo del documento.

Tareas

- T-45.1 Límite de intentos **por IP** en el login. Hoy hay bloqueo por usuario, que no frena a quien prueba una contraseña contra cien usuarios distintos —que es como se hace el ataque de verdad.
- T-45.2 Cabeceras: HSTS, CSP, X-Content-Type-Options, Referrer-Policy.
- T-45.3 Verificar que ningún mensaje revele si un usuario existe.
- T-45.4 Probar el aislamiento entre organizaciones **endpoint por endpoint**, con un usuario de una intentando leer datos de otra.

H-46 · ADR sobre gestión de identidad

No se implementa un IAM externo: se **decide y se documenta**, con el análisis y la ruta de migración para un despliegue real. El modelo Familia/Patente con Composite es objeto de evaluación, y delegarlo en un producto externo convertiría lo evaluado en configuración de un contenedor ajeno.

H-47 · Estados vacíos y esqueletos de carga

Hoy una tabla vacía se ve igual que una que todavía no cargó, y eso es un error de producto. Cada listado distingue «no hay nada», «todavía no cargó» y «falló», y cada vacío dice qué hacer.

H-48 · Panel de calibración de reglas

Como administrador, *quiero* ver cuántas alertas de cada regla se atendieron y cuántas se descartaron, *para* ajustar los umbrales con datos y no a ojo.

Criterio: por regla, generadas, atendidas, descartadas y la tasa de descarte.

Una regla con 90 % de descarte está mal calibrada, y el panel tiene que hacerlo evidente.

H-49 · Los manuales

Instalación, Usuario Gerente, Usuario Empleado y Programador, con los nombres del modelo de referencia, generados desde Markdown como el anexo de trazabilidad.

H-50 · El documento del TFI al día

Aplicar los deltas D-01 a D-19 al .docx, con las capturas del sistema rediseñado.

Compromiso: H-45 a H-50.

Cierre · Entrega

26/10 – 30/10 · Épica E8

H-51 · El horizonte financiero, resuelto

El VAN a cuatro años queda en –193,9 M porque la ventana termina justo cuando el proyecto da vuelta. **Es una decisión del autor**, no un error del modelo: extender a 5–6 años o incorporar valor terminal. Se aplica la elegida y se justifica en el plan de negocio.

H-52 · Ensayo de la defensa

Criterios

- Recorrido completo de punta a punta, cronometrado.
- Las tres demostraciones que dependen de que el sistema ande —CP-14 con la IA caída, el barrido predictivo generando alertas, el 403 por patente— probadas **en la máquina donde se va a defender**.
- Las diez preguntas más probables, con su respuesta y dónde está la evidencia.

H-53 · Congelamiento

Criterios

- Etiqueta **anotada** v1 sobre el commit que ese día pase la verificación completa. Anotada y no liviana porque deja escritos autor, fecha y motivo, que es lo que se pregunta de un congelamiento.
- La base de demostración **regenerada desde la semilla** y recién ahí respaldada, con el respaldo en Drive **y** en un pendrive. La de desarrollo arrastra restos —una incidencia llamada «Prueba de reevaluacion automatica» y saltos en la numeración— y eso se ve en pantalla. Regenerarla no cuesta trabajo manual: aplicando db/ y esperando dos minutos, el barrido predictivo deja el parque evaluado solo.
- Un clon limpio levanta con `docker compose up --build` y pasa todo **en la notebook donde se va a defender**, con las imágenes ya bajadas. Son 2,3 GB de SQL Server más lo que se construye: bajarlas el mismo día no es una opción. La corrida de prueba sobre la máquina de desarrollo está en `docs/tecnico/CLON-LIMPIO.md`.

La regla de alcance, escrita antes y no después. A partir del 30/10 entra lo que corrige algo que ya existe y lo que pida la devolución de la Entrega 2. No entra ninguna pantalla, endpoint ni tabla nueva. Se fija ahora porque después del congelamiento cualquier cosa se justifica como corrección.

La demostración va con el servicio de IA real. Se carga crédito antes de la defensa; hoy el sistema corre con el proveedor simulado, que es el valor por defecto. Conmutar es cargar la clave en Configuración › Integración de IA —por organización, y se guarda cifrada— sin tocar código. Queda una cosa por resolver en el ensayo: con el proveedor real activo, `PREDICTIT_IA_FALLAR_CADA` ya no sirve para demostrar el CP-14 porque sólo afecta al simulado, así que la caída se muestra volviendo esa organización a Simulado con la variable en 1.

Por qué el 30/10 y no la segunda entrega. Se evaluó moverlo. La Entrega 2 de negocio es el 16/10 y congelaría el sistema antes de la entrega que lo evalúa; la de tecnología todavía no tiene fecha confirmada con la cátedra. El 30/10 cae después de las dos en cualquier escenario razonable.

Congelamiento: viernes 30/10/2026. No es la fecha de la cátedra —la entrega final es el **miércoles 11/11/2026**, y la Entrega 2 de negocio el **16/10**—: es la fecha en que este proyecto deja de sumar alcance.

Los doce días que quedan entre una y otra son el colchón. Se reservaron sin alcance asignado —un plan sin colchón es un plan que ya sabe que va a incumplirse— y se usan para las correcciones que traiga la Entrega 2, para el ensayo de la defensa y para el video.

S10 · Correcciones finales

31/10 – 28/11 · Épicas E6, E8, E9

El tramo que absorbe lo que sale de revisar lo hecho. Se planificó vacío y hoy tiene diecisiete historias terminadas, todas del mismo tipo: cosas que aparecieron al mirar el sistema y los documentos con el criterio de alguien de afuera, y no al construir una funcionalidad nueva.

- **H-78 a H-83** · Las pantallas que la documentación no describía, la guía de reparación y el módulo del solicitante con el rediseño puesto.
- **H-84 a H-86** · El CP-15 automatizado sobre el recorrido completo, los veintidós diagramas saliendo del modelo y las capturas rehechas.
- **H-87** · El multiidioma cumpliendo lo que el requisito no funcional promete: se veía bien en castellano y en inglés mostraba castellano.
- **H-88** · La revisión de coherencia de punta a punta entre los documentos y el sistema, con los dos controles que deja.

Que este tramo tenga contenido y los anteriores estén cerrados es el efecto buscado del colchón: el alcance nuevo se congela el 30/10 y lo que entra después son correcciones, no funcionalidad.

Métricas del proceso

Tres, y ninguna de vanidad:

Métrica	Qué mide	Cómo se registra
Compromiso vs. entregado	Si estoy planificando de forma realista	Por sprint, en la retrospectiva
Defectos encontrados usando el sistema	Cuántos se me escapan al código y a las pruebas	Etiqueta defecto-de-uso en el tablero
Cobertura de pruebas	Tendencia, no valor absoluto	Publicada por el CI

La segunda es la que más importa acá, y el proyecto ya tiene la evidencia: los dos defectos más graves no los encontré ninguna prueba. Si ese número no baja, el problema es cómo pruebo, no cuánto pruebo.

Riesgos

Riesgo	Prob.	Impacto	Mitigación
El rediseño llega tarde o no sirve	Media	Alto	S6 tiene H-38 como alcance independiente; H-39 se corre a S7 sin romper el plan
La medición de carga incumple el RNF	Media	Medio	El resultado se publica igual y se abre la historia de optimización
Sobreestimar el sprint	Alta	Medio	Es lo que mide la primera métrica. Se ajusta con datos, no con voluntad
El CI con SQL Server es inestable	Media	Bajo	Las pruebas de integración pasan a un trabajo aparte durante un sprint
Cambio de alcance del tutor	Baja	Alto	El mes de noviembre

Tablero

Siete columnas: **Backlog** · **Preparada** · **Sprint actual** · **En curso** · **En revisión** · **Bloqueada** · **Terminada**.

Dos reglas que hacen que el tablero sirva:

1. **Un solo ítem en «En curso».** Con una persona, tener tres cosas empezadas y ninguna terminada es el estado más fácil de alcanzar y el menos útil.
2. **«Bloqueada» exige decir por qué y desde cuándo.** Un bloqueo sin fecha no se resuelve: se olvida.

Cada tarjeta lleva épica, criterios de aceptación, sprint, y al cerrarse **el enlace al commit**. Esa última parte es la que convierte el tablero en evidencia: cada historia terminada se puede seguir hasta el código que la implementa.

Hoja de ruta

Trabajo Final de Ingenieria - UAI - Pettini, Patricio Ezequiel (B00072710-T1)

![CI](https://github.com/PatricioPettini/trabajo-final-ingenier-a/actions/workflows/ci.yml)

Nota. El trabajo se adelanto al calendario. Las fechas de cada sprint son las planificadas en julio y no se retocan; los estados de las historias son lo que efectivamente esta hecho. Al 09/09/2026 estan terminadas todas las de S6, S7, S8 y S10, y las dos que quedan -el ensayo de la defensa y el congelamiento- dependen de fechas que todavia no llegaron. Se deja asi, con la diferencia a la vista, en lugar de correr las fechas hacia atras para que coincidan: un cronograma que se reescribe cada vez que se cumple no sirve para medir nada.

Sprint en curso: S5 Predictivo, auditoria de arquitectura y CI — cierra el 2026-09-11

Que el analisis predictivo funcione solo, que los requisitos de arquitectura de la cathedra esten todos cubiertos, y que la verificacion deje de depender de que yo me acuerde de correrla.

Calendario

```
gantt
    title PredictIT – del relevamiento a la entrega
    dateFormat YYYY-MM-DD
    axisFormat %d/%m
    todayMarker stroke-width:3px,stroke:#A03030,opacity:0.6

    section Construido
    S1 · Relevamiento, arquitectura y entorno :done, 2026-07-06, 2026-07-17
    S2 · Modelo de datos, DAO y patrones :done, 2026-07-20, 2026-07-31
    S3 · Seguridad :done, 2026-08-03, 2026-08-14
    S4 · Nucleo funcional :done, 2026-08-17, 2026-08-28

    section En curso
    S5 · Predictivo, auditoria de arquitectura y CI :active, 2026-08-31, 2026-09-11

    section Planificado
    S6 · Multiidioma en la interfaz y sistema visual :2026-09-14, 2026-09-25
    S7 · Pantallas, calidad y deuda :2026-09-28, 2026-10-09
    S8 · Endurecimiento, producto y documentacion :2026-10-12, 2026-10-23
```

S9 · Cierre y entrega :crit, 2026-10-26, 2026-10-30

S10 · Correcciones finales :2026-10-31, 2026-11-28

section Después de la entrega

Correcciones y ajustes finales :2026-11-02, 2026-11-27

Avance por épica

	Épica	Avance		Por qué existe
E1	Arquitectura y entorno		10/10	Es lo que evalua la catedra y condiciona todo lo demas
E2	Seguridad		14/14	Req. Arq. 002 y 004, y el modelo Familia/Patente
E3	Nucleo funcional		10/10	Los CU-001 a CU-013 del documento
E4	Analisis predictivo e IA		4/4	Es lo que distingue al sistema de un ticketing
E5	Integracion continua		3/3	Que la verificacion no dependa de que yo me acuerde
E6	Multiidioma e interfaz		9/9	Req. Arq. 001, y la defensa visual
E7	Calidad demostrable		8/8	Que un tercero pueda ver que las pruebas cubren lo que el documento promete
E8	Producto y documentacion		29/31	Es la mitad de la nota
E9	Deuda tecnica		3/3	Lo que se que esta mal y todavia no arregle

Reparto del backlog

```
pie showData
  title Historias por estado
  "Terminada" : 90
  "Pendiente" : 2
pie showData
  title Historias por épica
  "Arquitectura y entorno" : 10
```

```

"Seguridad" : 14
"Nucleo funcional" : 10
"Análisis predictivo e IA" : 4
"Integración continua" : 3
"Multiidioma e interfaz" : 9
"Calidad demostrable" : 8
"Producto y documentación" : 31
"Deuda técnica" : 3

```

Sprints

[x] s1 · Relevamiento, arquitectura y entorno

██████████ 5/5 · 2026-07-06 a 2026-07-17

Objetivo. Decidir la arquitectura y dejarla firmada antes de escribir la primera línea de negocio.

	Historia	Épica
[x]	H-01 Migrar el stack de Java/Spring a C#/.NET	E1
[x]	H-02 Separar negocio de seguridad en dos bases	E1
[x]	H-03 Acceso a datos con DAO explícito	E1
[x]	H-04 Multi-organización desde el modelo de datos	E1
[x]	H-05 Esqueleto de las cinco capas y la base en Docker	E1

Retrospectiva. La decisión que más rindió fue escribir los ADR antes del código: cuando después apareció la tentación de usar EF Core para ir más rápido, el motivo por el cual no ya estaba escrito.

[x] s2 · Modelo de datos, DAO y patrones

██████████ 6/6 · 2026-07-20 a 2026-07-31

Objetivo. Las dos bases aplicadas y el acceso a datos completo, con los patrones que la cátedra evalúa en uso real.

	Historia	Épica
[x]	H-06 Esquema de la base de negocio	E1
[x]	H-07 Esquema de la base de servicio	E1

	Historia	Épica
[x]	H-08 DAO generico y mappers	E1
[x]	H-09 Fabricas y unidad de trabajo	E1
[x]	H-10 Aislamiento por organizacion aplicado en el DAO	E1
[x]	H-11 Pruebas de integracion del DAO	E7

<details><summary>Defectos del sprint</summary>

Tres invisibles al leer, que solo aparecieron ejecutando: PLAN y BACKUP son palabras reservadas de T-SQL; un VARCHAR(10) truncaba 'ADVERTENCIA'; y los indices filtrados fallan si sqlcmd no recibe -I, porque deja QUOTED_IDENTIFIER en OFF.

</details>

Retrospectiva. Accion tomada: los scripts de base se ejecutan, nunca se revisan.

[x] s3 · Seguridad

6/6 · 2026-08-03 a 2026-08-14

Objetivo. Req. Arq. 002 y 004, y el modelo de permisos de la catedra.

	Historia	Épica
[x]	H-12 Hash de contraseñas	E2
[x]	H-13 Familias y patentes con Composite	E2
[x]	H-14 Autenticacion con JWT	E2
[x]	H-15 Autorizacion por patente	E2
[x]	H-16 Bitacora inmutable	E2
[x]	H-17 Respaldo y restauracion	E2

<details><summary>Defectos del sprint</summary>

Una dependencia circular contexto -> seguridad -> DAO -> contexto: el login funcionaba y la primera peticion autenticada colgaba. Se resolvio partiendo IFactoryDaoSeguridad, que no depende del contexto.

</details>

Retrospectiva. El [RequierePatente] original aceptaba una sola patente y daba 403 al administrador. Accion: los permisos se verifican con un usuario de cada perfil, no solo con el administrador.

[x] s4 · Nucleo funcional**6/6** · 2026-08-17 a 2026-08-28**Objetivo.** Los casos de uso del documento, de punta a punta y con interfaz.

	Historia	Épica
[x]	H-18 Inventario de activos	E3
[x]	H-19 Incidencias de punta a punta	E3
[x]	H-20 Mantenimientos	E3
[x]	H-21 Las 19 pantallas	E3
[x]	H-22 Seis temas y tres densidades	E6
[x]	H-23 Contraste AA medido	E6

<details><summary>Defectos del sprint</summary>

Dos del mismo tipo, peligrosos porque el SQL se leía correcto: el join de tecnicos pasaba por Usuario_Organizacion y devolvía cero tecnicos, y UsuarioDao.PorOrganizacion tenia el mismo sesgo, con lo cual los nombres de los Partner salían vacíos.

</details>

Retrospectiva. Acción: toda consulta con JOIN sobre tablas de relación se prueba con datos de los dos lados, incluido el caso de la fila que no existe.

[~] ss · Predictivo, auditoria de arquitectura y CI**19/19** · 2026-08-31 a 2026-09-11

Objetivo. Que el análisis predictivo funcione solo, que los requisitos de arquitectura de la cátedra estén todos cubiertos, y que la verificación deje de depender de que yo me acuerde de correrla.

	Historia	Épica
[x]	H-24 Motor predictivo con estrategias	E4
[x]	H-25 Proveedor de IA conmutable	E4
[x]	H-26 Disyuntor sobre el servicio externo	E4
[x]	H-27 Multiidioma en el backend	E6
[x]	H-28 Cifrado reversible	E2
[x]	H-29 Patente.TipoAcceso	E2
[x]	H-30 La bitácora registra los eventos de negocio	E2

	Historia	Épica
[x]	H-31 El motor predictivo se dispara solo	E4
[x]	H-32 Bitacora por usuario y pantalla de errores	E2
[x]	H-33 Bulkhead para los procesos en segundo plano	E9
[x]	H-34 El sistema entero en Docker	E5
[x]	H-35 Integración continua	E5
[x]	H-36 La bitacora de demostración en un solo huso	E9
[x]	H-37 Publicar la cobertura como línea de base	E7
[x]	H-54 El ciclo de vida completo de la incidencia	E3
[x]	H-55 Cambio de estado operativo del equipo	E3
[x]	H-56 Reportes exportables a PDF	E3
[x]	H-57 Historial técnico de la organización	E3
[x]	H-58 Cierre de sesión con rastro, y restauración expuesta	E2

<details><summary>Defectos del sprint</summary>

Los dos peores del proyecto, y los dos fueron ausencias: la bitacora sin eventos de negocio (apareció contando filas por tipo de evento) y el motor que no se disparaba solo (lo reportó el usuario). Ninguna prueba podía fallar: no había código roto, faltaba código. Mas tres del entorno: los patrones de .dockerignore no son recursivos, la imagen mssql-tools18 no existe, y el runtime de .NET no trae curl ni wget. Y una que importa al negocio: el contenedor corre en UTC y el sistema mide un SLA de 4 horas.

</details>

Retrospectiva. Acción tomada: auditar contra datos, no contra el código. Se incorpora a la Definición de Terminado como el punto 4.

[] s6 · Multiidioma en la interfaz y sistema visual

3/3 · 2026-09-14 a 2026-09-25

Objetivo. Cerrar Req. Arq. 001 de punta a punta y aplicar la primera mitad del rediseño.

	Historia	Épica
[x]	H-38 Toda la interfaz sale del diccionario	E6
[x]	H-39 Sistema visual y componentes del rediseño	E6
[x]	H-59 El front cablea el ciclo de vida y el ABM de equipos	E6

[] s7 · Pantallas, calidad y deuda

5/5 · 2026-09-28 a 2026-10-09

Objetivo. Terminar el rediseño y que la calidad se pueda auditar.

	Historia	Épica
[x]	H-40 Las pantallas rediseñadas	E6
[x]	H-41 Los CP-01 a CP-14 son una suite identificable	E7
[x]	H-42 El RNF de carga está medido	E7
[x]	H-43 Partir IAdministracionBusiness	E9
[x]	H-44 Las pruebas no dejan residuo	E7

[] s8 · Endurecimiento, producto y documentacion

9/9 · 2026-10-12 a 2026-10-23

Objetivo. Cerrar la seguridad con una revision sistematica y dejar el producto y los documentos presentables.

	Historia	Épica
[x]	H-45 Revisión contra OWASP Top 10	E2
[x]	H-46 ADR sobre gestión de identidad	E2
[x]	H-47 Estados vacíos y esqueletos de carga	E8
[x]	H-48 Panel de calibración de reglas	E8
[x]	H-49 Los manuales	E8
[x]	H-50 El documento del TFI al día	E8
[x]	H-60 Diccionario de datos	E8
[x]	H-61 Diagramas del sistema	E8
[x]	H-62 La carpeta de entregables	E8

[] s9 · Cierre y entrega

 10/12 · 2026-10-26 a 2026-10-30

Objetivo. Entregar el 30/10. Noviembre queda entero para correcciones.

	Historia	Épica
[x]	H-51 El horizonte financiero, resuelto	E8
[]	H-52 Ensayo de la defensa	E8
[]	H-53 Congelamiento y entrega	E8
[x]	H-63 Las capturas del documento	E8
[x]	H-64 Los diagramas del documento salen del código	E8
[x]	H-65 Numeración de figuras y tablas en el documento	E8
[x]	H-66 Los entregables toman el formato del documento principal	E8
[x]	H-67 La ficha de equipo muestra el historial y el riesgo	E8
[x]	H-68 Presentación de la defensa	E8
[x]	H-69 Revisión de estilo de la prosa de los documentos	E8
[x]	H-70 Rediseño 02: capa de tokens, tres temas y riesgo sin color	E8
[x]	H-76 Los diagramas salen de Enterprise Architect	E8

[] s10 · Correcciones finales

 21/21 · 2026-10-31 a 2026-11-28

Objetivo. Lo que quedó abierto después de congelar la entrega del sistema. No entra en la versión que se entrega el 30/10: entra en el mes de ajustes previo a la defensa.

	Historia	Épica
[x]	H-71 Pruebas de la capa de negocio	E5
[x]	H-72 El cuerpo de las pantallas también sale del diccionario	E7
[x]	H-73 Cerrar A05 del OWASP Top 10	E6
[x]	H-74 Licencia del repositorio	E8
[x]	H-75 La descripción de las 17 pantallas del capítulo 10.5	E8
[x]	H-77 Los textos que viven dentro del código	E7

	Historia	Épica
[x]	H-78 Tres pantallas que ni el capítulo 10.5 ni los manuales describen	E8
[x]	H-79 Los cuatro casos de uso que el modelo de permisos promete (D-22)	E8
[x]	H-80 Confirmar los siete cambios marcados «por verificar»	E8
[x]	H-81 Revisar de punta a punta la primera entrega de negocio	E8
[x]	H-82 Armar la entrega parcial de sistema, recortando lo que sobra	E8
[x]	H-83 El panel de la guía de reparación en el detalle de la incidencia	E8
[x]	H-84 CP-15: la guía de reparación con el proveedor caído	E8
[x]	H-85 Rediseño del frontend: aplicar la dirección financiera	E8
[x]	H-86 Las catorce imágenes de diagramas que faltan reemplazar a mano	E8
[x]	H-87 El multiidioma cumpla a medias lo que el RNF promete	E6
[x]	H-88 Revisión de coherencia de punta a punta, como auditor externo	E8
[x]	H-89 Lo que quedaba abierto de la revisión OWASP: A02 y A08	E2
[x]	H-90 La capa de negocio, con pruebas propias	E7
[x]	H-91 El mantenimiento deja de ser solo un registro: el plan y su agenda	E3
[x]	H-92 La facturación del servicio: el modelo de ingresos entra al sistema	E3

Cómo se lee esto

[x]	Terminada: cumple las siete condiciones de la Definición de Terminado
[~]	En curso
[]	Pendiente

El detalle de cada historia —criterios de aceptación, tareas, riesgos— está en PLAN-DE-SPRINTS.md. La fuente única de la que sale **esta página, el tablero de Trello y los issues de GitHub** es backlog.json: tres vistas, un solo backlog. Una vista que se mantiene aparte se desincroniza en la primera semana.

```
python _a-roadmap.py      # regenera esta página
python _a-trello.py       # crea o actualiza el tablero de Trello
powershell -File _a-github.ps1 # hitos, etiquetas, issues y tablero de GitHub
```

_{Generado desde backlog.json el 2026-09-02. No editar a mano.}

Lo que falta

PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1 ·

Grupo 2 · Universidad Abierta Interamericana

*Estado al **11 de septiembre de 2026**, después de los dos módulos nuevos*

—plan y agenda de mantenimiento, y facturación del servicio—.

Este documento existe porque la respuesta a «¿qué falta?» estaba repartida entre el backlog, el plan de la Entrega 1, la revisión OWASP y los LEEME de cada carpeta, y juntarla a mano cada vez es la forma más rápida de olvidarse de algo. Aquí está entero, separado por lo único que importa para saber si se puede hacer hoy: **de qué depende**.

Lo que **no** está acá es lo que ya se hizo. Para eso están la hoja de ruta y el registro de cambios.

Fechas

Entrega 1 de negocio · Grupo 2	viernes 18 de septiembre de 2026
Entrega 2 de negocio	16 de octubre de 2026
Entrega 2 de tecnología	fecha por confirmar con la cátedra
Congelamiento propio	30 de octubre de 2026
Entrega final	11 de noviembre de 2026

Son las del **Grupo 2**. La cátedra alterna el orden de los grupos entre entregas, así que la fecha del Grupo 1 no sirve de referencia.

1 · La Entrega 1 de negocio, del viernes 18/09

Está armada y verificada. La carpeta es ENTREGABLES/ENTREGA NEGOCIO/PRIMER ENTREGA/ y lleva tres archivos: el plan de negocio en Word y PDF, el presupuesto financiero recortado a diez hojas, y el log de prompts. El modelo recalcula sin errores y la celda de control —el escenario base contra el flujo de fondos— da cero.

La fecha es la del **Grupo 2**. La cátedra alterna el orden entre entregas: en la primera el Grupo 1 entregó el 11/09 y el Grupo 2 entrega el 18; en la segunda se invierte y el Grupo 2 abre el 16/10.

Después viene la Entrega 2, del 16/10, y es lo único con trabajo de redacción por delante: su contenido depende de qué traiga la corrección de ésta, así que no se puede adelantar.

Lo que **ya está calculado** en el libro y sólo hay que recortar: modelo de inversión, amortizaciones, impuestos, métricas SaaS, flujo de fondos, matriz de riesgos y los tres escenarios. Son las once hojas que la primera entrega deja afuera.

Lo que **falta escribir**: el documento de la entrega, y el script docs/negocio/_entrega-2.py que arma la carpeta —hoy existe el de la primera y el recorte al revés es de dos líneas—.

2 · Lo que se puede hacer hoy

Rehacer a mano en Enterprise Architect los tres diagramas nuevos. Los CU-019, CU-020 y CU-021 ya tienen su diagrama de secuencia en 10.4.4 (D-50), pero son los que genera _diagramas-ea.py, no versiones rehechas a mano: la tabla EQUIVALENCIA de docs/tecnico/_diagramas-del-modelo.py cubre CU-001 a CU-013 y Modelo Enterprise Architect - PredictIT.eap no tiene los tres nuevos. Son correctos —salen de leer los controladores y los business—, pero se ven distintos de los otros trece. Dibujarlos en el modelo y sumarlos a EQUIVALENCIA cierra la diferencia.

La corrida de clon limpio en la notebook donde se va a defender. Es el criterio 3 de H-53 y conviene no dejarlo para el 30/10: si en esa máquina falta Docker, falta memoria o el build se cae, mejor enterarse con semanas. Son 2,3 GB de imágenes que no se bajan en el pasillo. El procedimiento y lo que salió al correrlo acá están en tecnico/CLON-LIMPIO.md.

Confirmar con la cátedra la fecha de la Entrega 2 de tecnología. Es el único dato del cronograma que el proyecto no tiene, y aparece como «por confirmar» en la hoja de ruta y en los LEEME.

Cargar el crédito de la API de Anthropic. La demostración va con el servicio real (decisión 6 de H-53). Que la clave esté cargada no alcanza: sin saldo la llamada falla y el sistema asigna por respaldo, que es lo correcto pero no es lo que se quiere mostrar. Verificarlo **el día anterior**.

3 · Lo que depende de una fecha

H-52 · Ensayo de la defensa — semana del 26/10. El guion está escrito (defensa/GUION-DE-ENSAYO.md): el minutado, el recorrido paso a paso y las preguntas más probables con su respuesta. Lo que falta es ensayarlo **con el reloj**, tres veces, y probar en la máquina de la defensa las tres demostraciones que dependen de que el sistema ande: el CP-14 con la IA caída, el barrido predictivo generando alertas y el 403 por falta de patente.

Queda una cosa por resolver ahí: con el proveedor real activo, PREDICTIT_IA_FALLAR_CADA ya no sirve para mostrar el CP-14 porque sólo afecta al simulado. La caída se demuestra volviendo esa organización a **Simulado** con la variable en 1.

H-53 · Congelamiento — 30/10. Etiqueta anotada v1 sobre el commit que ese día pase la verificación completa; la base de demostración regenerada desde la semilla y respaldada, con el respaldo en Drive y en un pendrive; y el clon limpio andando en la notebook de la defensa. Las seis decisiones que la historia pedía están tomadas y escritas en el plan de sprints.

El video. Lo graba el autor.

4 · Lo que los dos módulos nuevos dejaron anotado

Ninguna de las tres es deuda: son decisiones de alcance, escritas acá para que no parezcan olvidos en la defensa.

El comprobante no es fiscal. No hay CAE, ni punto de venta, ni integración con AFIP, y por eso la entidad se llama Comprobante y no Factura. Es el registro interno del servicio. Integrarlo con facturación electrónica es trabajo posterior y no cambia el modelo: agrega campos y una llamada externa.

La agenda no nivela la carga. Si diez equipos del mismo tipo tienen su preventivo el mismo día, el sistema agenda los diez para ese día. Repartir el trabajo entre los técnicos disponibles es una función distinta —y una que exige saber la capacidad de cada uno—, y no está.

No hay aviso automático de lo que vence. La agenda hay que mirarla. Las notificaciones son la sección declarada fuera de alcance del capítulo 10.5, y esto entra ahí: el día que haya notificaciones, «tenés tres preventivos vencidos» es la primera que corresponde.

5 · Lo que está declarado y no es deuda escondida

Nada de esto es un olvido: está escrito, con su motivo, y se dice así en la defensa en lugar de esperar que no lo pregunten.

	Dónde está declarado
A07 · No hay segundo factor de autenticación. Es consecuencia de decidir gestión de identidad propia sin IAM externo, con su ruta de migración escrita	ADR 0014 · Revisión OWASP
A08 · Los artefactos no se firman. Las dependencias sí están congeladas con su hash y el CI restaura en modo bloqueado; lo que falta es la attestation de procedencia de las imágenes	Revisión OWASP

	Dónde está declarado
TLS no está activo. Está configurado y probado; no se activa porque en localhost el único certificado posible es autofirmado	frontend/nginx-tls.conf
La capa de negocio es la de menor cobertura, con 29,1 % contra 51,4 % del proyecto	Cobertura
La medición de carga es local. No mide latencia de red, ni carga sostenida de horas, ni escrituras concurrentes sobre la misma incidencia	Carga
Dos copias del proyecto no conviven en la misma máquina, por el nombre fijo de los contenedores	Clon limpio

Retomar el trabajo en otra máquina

Todo lo que hace falta está en el repositorio; nada del estado vive sólo en la máquina donde se trabajó. El procedimiento completo —con lo que salió al correrlo— está en `tecnico/CLON-LIMPIO.md`. En corto:

```
git clone <repo> ; cd TFI
copy .env.ejemplo .env      # y completar las tres claves
docker compose up -d        # SQL Server, API y frontend
.\db\aplicar.ps1            # esquemas y datos de demostración
cd backend ; dotnet test     # tienen que dar 340 en verde
cd ..\frontend ; npm ci ; npm test
```

Dos cosas que **no** están en el repositorio y hay que tener a mano:

- Las tres claves del `.env` —contraseña de SQL Server, clave de firma del JWT y clave de cifrado—, que no se versionan a propósito.
- **Enterprise Architect**, si se van a tocar los diagramas. Sin EA, `_diagramas-ea.py` avisa y no hace nada: las imágenes que ya están en `docs/diagramas-ea/` siguen sirviendo.

Y una que conviene saber antes de empezar: **dos copias del proyecto no conviven en la misma máquina**, porque los contenedores tienen nombre fijo.

Para verificar que todo quedó consistente después de cualquier cambio, el único comando que hay que recordar es `python docs/_verificar-alineacion.py`: son dieciocho controles que comparan lo que el proyecto declara contra lo que efectivamente mide.

Cómo se mantiene este documento

A mano, y con dos anclas que no dependen de que alguien se acuerde: las historias pendientes salen del `backlog.json` —que es la fuente única del avance— y las fechas son las mismas que

verifican los controles de alineación. El control 18 compara las dos cosas contra este archivo y falla si dejan de coincidir.

Cambios sobre el documento original

Registro vivo. Cada vez que una decisión de desarrollo contradice o precisa lo que dice el documento (Trabajo Final de Ingeniería - Pettini Patricio Ezequiel, v2.0, 10/07/2026), se anota acá **en el momento**. En la fase de cierre se aplica todo de una sola vez sobre el .docx.

Estado del documento de trabajo (docs/tfi/TFI-PredictIT.docx): completo respecto de la versión definitiva de septiembre. Se armó uniendo los dos Word parciales —uno traía el capítulo 10 con sus diagramas, el otro los capítulos 6 y 7— y portando desde el PDF las dos secciones que no existían en ninguno: **6.5.3** (justificación económica de los precios) y **7.7** (presupuesto y punto de equilibrio), que son la respuesta del autor a la devolución del 04/07/2026.

Estados: pendiente (identificado, todavía no escrito) · redactado (texto nuevo listo en docs/) · pendiente de volcar al .docx (el trabajo está hecho, falta pegarlo) · aplicado (ya volcado al .docx) · aplicado fuera del .docx (vive en un anexo o en el código, y ahí corresponde) · descartado (se decidió no hacerlo, con el motivo).

Del D-11 en adelante el registro había pasado a prosa libre y el estado se perdió: 22 de los 32 cambios no decían si estaban aplicados. Se les devolvió el campo, y los que no se pudieron confirmar quedaron marcados como tales en lugar de darse por hechos.

El 09/09 se cerró esa deuda. Los siete que quedaban sin confirmar se verificaron leyendo el documento, uno por uno: tres estaban aplicados y cuatro no —y decir «por verificar» los hacía parecer casi hechos, que es peor que decir que faltan—. Ya no queda ningún cambio con el estado sin determinar. Además, tres que decían depender de regenerar el DER, las pantallas o los diagramas quedaron marcados como lo que son: la dependencia se resolvió hace días y lo único que falta es pegarlos en el .docx.

D-01 · Entorno del sistema: Java a C#

Sección	10.2.5 Descripción del entorno del sistema (tabla)
Estado	aplicado

Dice hoy: Framework Spring Boot · Lenguaje Java · Base de datos MySQL · Servidor Web IIS 10 · Infraestructura Servidor - Azure.

Tiene que decir: Framework ASP.NET Core 9 (Web API) + React 19 · Lenguaje C# y TypeScript ·

Base de datos Microsoft SQL Server 2022 (dos bases: negocio y servicio) · Servidor Web Kestrel detrás de reverse proxy · Infraestructura contenedores Docker, con despliegue previsto en Azure.

Por qué: C# es el lenguaje de la carrera y el del proyecto de referencia de 3.º año, lo que permite reutilizar el modelo de capas (DOMAIN / DAO / BLL / SERVICE) que evalúa la cátedra. SQL Server acompaña esa decisión y además habilita BACKUP/RESTORE DATABASE nativo, que es lo que pide el Req. Arq. 003.

D-02 · Multi-organización en el DER

Sección	10.4.7 Diagrama Entidad Relación (DER Negocio) · 10.4.6 Diagrama de clases
Estado	aplicado — los tres diagramas entidad-relacion del .docx se reemplazaron el 09/09 por los que exporta Enterprise Architect, que se leen de las bases y por lo tanto traen Organizacion. El mapeo se resolvió por proporcion para no poner un diagrama en la sección de otro

Dice hoy: El DER de negocio no tiene entidad Organizacion; las entidades cuelgan directamente unas de otras.

Tiene que decir: Se incorpora Organizacion como raíz del modelo de negocio. Ubicacion, Equipo, Incidencia, Mantenimiento, AlertaPredictiva, ReglaAlerta y ConfiguracionAsignacion llevan id_organizacion. El aislamiento entre organizaciones se resuelve en la capa DAO a partir del contexto de sesión.

Por qué: El propio plan de negocio define el Segmento C (proveedores tercerizados de soporte) como canal estratégico y vende un **Plan Partner multi-cliente** (sección 5.3.6). Sin entidad organización ese plan no es implementable. Además, es el requisito que sostiene la sección 6.4.1 cuando habla del "portal multi-cliente".

D-03 · Alcance de la IA: sumar clasificación / triage

Sección	1.7 Capacidades centrales · 2.2.1.1 Cuadro comparativo · 6.4.1 Qué es y qué no es · 10.2.3 Alcance del desarrollo
Estado	aplicado en 1.7, 6.4.1 y 10.2.3 — falta el cuadro comparativo 2.2.1.1

Dice hoy: La IA interviene únicamente en la asignación automática de incidencias a técnicos.

Tiene que decir: La IA interviene en dos puntos del ciclo de la incidencia:

(a) **triage / clasificación** del texto libre del solicitante: categoría del problema, severidad sugerida y detección de duplicados sobre el mismo equipo; y (b) **asignación** del técnico responsable. El triage corre primero y alimenta el contexto de la asignación. En ambos casos rige el mismo principio: heurística por reglas primero, IA como refuerzo, y resultado siempre editable por el técnico.

Por qué: El usuario del Segmento A es no técnico y describe los problemas en lenguaje

coloquial ("no anda", "hace un ruido raro"). Sin una capa de interpretación, la categoría y la severidad que llegan al motor de asignación son ruido, y la recomendación de técnico se degrada. El triage es el paso que convierte ese texto en datos utilizables.

D-04 · Requerimiento funcional nuevo para el triage

Sección	10.2.2 Especificaciones de requerimientos (tabla de RF)
Estado	aplicado el RF-07b y el índice de CU — falta la especificación completa de CU-006b y su caso de prueba

Dice hoy: RF-08 cubre la asignación mediante IA; no hay ningún RF de clasificación.

Tiene que decir: Se agrega un RF previo a RF-08 (numeración a definir al aplicar, para no romper las referencias existentes de RF-08 y RF-08b):

- **Entrada:** texto libre de la incidencia, equipo afectado e incidencias abiertas del mismo equipo.
- **Proceso:** aplicar heurísticas de clasificación por palabras clave y, como refuerzo, consultar al servicio de IA externo para obtener categoría, severidad sugerida y posibles duplicados. Si el servicio no responde, conservar el resultado de las heurísticas.
- **Salida:** incidencia clasificada con categoría y severidad sugeridas, y aviso de duplicado cuando corresponda, todo editable por el técnico.

Por qué: Derivado de D-03. Sin RF asociado, la funcionalidad no tendría caso de uso ni caso de prueba que la respalden.

D-05 · Pantallas del sistema

Sección	10.5 Pantallas del sistema (10.5.1.1 a 10.5.2.4)
Estado	pendiente de volcar al .docx — la dependencia ya no existe: las capturas reales estan hechas (D-24) y las pantallas descritas (D-29), los dos aplicados. Lo unico que falta es reemplazar los mockups del 10.5

Dice hoy: Mockups de referencia de las 16 pantallas.

Tiene que decir: Capturas de las pantallas reales del sistema implementado.

Por qué: Los mockups actuales son bocetos previos al desarrollo. Al cierre se reemplazan por capturas de la aplicación funcionando, que son las que se muestran en la defensa.

D-06 · Diagramas de arquitectura

Sección	10.4.4 Diagrama de paquetes · 10.4.5 Diagrama de componentes · 10.4.8 Diagrama de despliegue
Estado	aplicado — paquetes, componentes y despliegue son los tres de Enterprise Architect. Componentes y despliegue ya lo estaban; el de paquetes se reemplazo el 09/09

Dice hoy: Diagramas genéricos, previos a la definición del stack.

Tiene que decir: Diagramas que reflejen la solución real: los cinco proyectos del backend con sus dependencias, el frontend React como componente separado, las dos bases de datos y el servicio externo de IA como sistema externo.

Por qué: Con el cambio de stack (D-01) y la incorporación del proveedor de IA conmutable, los diagramas actuales dejan de representar el sistema construido.

D-07 · Anexo de transparencia sobre el uso de IA

Sección	Nueva. Anexo al final del documento, después de Fuentes
Estado	redactado en docs/ — falta el anexo en el .docx

Dice hoy: El documento no menciona el uso de herramientas de IA en el desarrollo.

Tiene que decir: Se incorpora un anexo de transparencia con dos partes:

1. **Registro de uso de IA** (docs/trazabilidad/AI-USAGE-LOG.md) — qué herramientas se usaron, para qué tareas, qué se aceptó, qué se modificó y cómo se validó, por entrega.
2. **Prompts críticos y validación técnica** (docs/trazabilidad/PROMPTS-CRITICOS.md) — las consultas que afectaron decisiones de arquitectura, en formato C4, con el proceso por el cual cada propuesta se aceptó o se descartó.

Se suma además la carpeta docs/adr/ como anexo: las decisiones de arquitectura en formato Nygard, referenciadas desde el capítulo 10.3.

Por qué: Es un entregable exigido. Y tiene valor propio para la defensa: separa

lo que aportó la herramienta de lo que decidió el autor, que es la pregunta que va a aparecer.

D-08 · Requerimientos no funcionales de carga y seguridad

Sección	10.2.2 Especificaciones de requerimientos (tabla de RNF)
Estado	aplicado

Dice hoy: RNF-04 fija menos de 2 segundos con 500 activos y 5000 incidencias, pero no dice cómo se verifica. RNF-02 habla de seguridad sólo como control de acceso.

Tiene que decir: RNF-04 se amplía con el perfil de carga concreto y el percentil 95 como métrica, y se agregan requerimientos de análisis estático de código (SAST), verificación de dependencias con vulnerabilidades conocidas y perfilado de memoria bajo carga sostenida. El detalle está en el ADR 0008.

Por qué: Dado el incremento en la velocidad de codificación que aporta la asistencia por IA, el nivel de exigencia en pruebas de estrés, perfilado de memoria y análisis estático debe ser superior al estándar tradicional. Los RNF actuales no alcanzan para sostener esa afirmación con evidencia.

D-09 · Patrones de resiliencia en la arquitectura base

Sección	10.3.2 Componentes de la arquitectura base · 10.4.2.7 (CU-007) · 10.4.9 (casos de prueba)
Estado	aplicado en el glosario — falta 10.3.2, el flujo alternativo de CU-007 y su caso de prueba

Dice hoy: El comportamiento ante falla del servicio de IA se resuelve únicamente con la asignación de respaldo (CU-007, flujo alternativo; CP-14).

Tiene que decir: Se incorpora un componente de resiliencia a la arquitectura

base, con **circuit breaker** sobre el servicio de IA y **bulkhead** entre las operaciones interactivas y los procesos en segundo plano. El flujo alternativo de CU-007 contempla el circuito abierto —no se intenta la llamada— además del fallo individual, y se agrega un caso de prueba para la transición de estados del disyuntor.

Por qué: Surge de la auditoría de arquitectura previa a la implementación

(docs/trazabilidad/PROMPTS-CRITICOS.md · PC-05). El timeout solo protege al sistema de colgarse, pero no de un servicio consistentemente caído: sin disyuntor, cada incidencia registrada durante una caída paga los 15 segundos completos. Ver ADR 0009.

D-10 · El plan comercial pasa a ser parte del sistema

Sección	10.1.7 Información gestionada · 10.4.7 DER · 10.5.1.7.1 pantalla de Organización
Estado	aplicado en el modelo de datos — falta el DER y la pantalla

Dice hoy: El documento define los planes, sus precios, los equipos que incluye

cada uno y el precio del equipo adicional (5.3.6), y apoya en esos valores la justificación económica de los precios (6.5.3). Pero el sistema no los conoce: nada del modelo de datos representa un plan ni un límite de equipos.

Tiene que decir: Se incorpora PlanComercial al modelo de negocio, con abono

mensual, equipos incluidos, precio del equipo adicional y fecha de vigencia. Organizacion lo referencia. La pantalla de Organización muestra el plan, los equipos administrados contra los incluidos, los adicionales y el total mensual resultante, y avisa cuando conviene otro plan.

Se suma además, en esa misma pantalla, el consumo del período del servicio de IA —consultas de clasificación, de asignación, y cuántas terminaron en respaldo—.

Por qué: Es la respuesta a la devolución del 04/07/2026, que pide reforzar la

justificación económica de los precios y vincularla con la estructura operativa. La sección 6.5.3 afirma que el consumo de IA cuesta del orden de ARS 1.500 por cliente por mes; el sistema ya

registra cada consulta en `RecomendacionAsignacion` y `ClasificacionIncidencia`, así que exponer ese contador convierte una estimación en una medición. Y con el plan modelado, la afirmación de que la organización de demostración está por encima de lo que incluye su plan deja de ser una nota al pie y pasa a ser algo que el sistema calcula: 52 equipos, 37 adicionales, ARS 244.400 mensuales contra los ARS 89.000 del abono base. Los importes siguen a D-14: con los precios anteriores la cuenta daba ARS 101.500 contra ARS 34.900.

Nota: `PLAN` es palabra reservada en T-SQL, por eso la tabla es `PlanComercial`.

D-11 · La asignación distingue IA de heurística

Estado: aplicado — el documento distingue asignacion por IA, por heuristica y de respaldo

Sección del documento: 10.2.2 (RF-08), 10.4.7 (DER de negocio), 10.5.1.4

(pantalla de incidencias)

Dice hoy: RF-08 describe la asignación automática de incidencias mediante

inteligencia artificial, y el modelo registra el origen de la recomendación como IA o respaldo.

Tiene que decir: El origen de una asignación admite **tres** valores: IA

cuando la decidió el proveedor externo, `HEURISTICA` cuando la decidió el proveedor determinístico del sistema, y `RESPALDO` cuando ninguno respondió y se aplicó la regla de menor carga. Lo mismo en `Incidencia.tipo_asignacion`, que además conserva `PENDIENTE` y `MANUAL`.

Por qué: El proveedor determinístico no es un modelo de lenguaje: resuelve por

coincidencia de términos y por un puntaje que pondera especialidad, carga e historial. La primera implementación registraba esas decisiones como «IA», porque era el único valor que el modelo admitía, y eso falsea exactamente lo que el trabajo tiene que poder demostrar: qué decidió una IA y qué decidió una regla escrita a mano. Con la distinción, el anexo de trazabilidad puede afirmar cuántas asignaciones fueron de cada tipo, y la pantalla de detalle lo dice con todas las letras.

Es además lo que hace verificable la comparación: el proveedor determinístico es la línea base contra la cual se mide si la IA aporta algo. Sin distinguir los orígenes, «la IA asigna mejor» no sería una afirmación comprobable.

D-12 · Tres atributos que el motor predictivo necesita

Estado: aplicado — verificado el 09/09 leyendo la sección: el documento enumera «criticidad del equipo y antigüedad» entre las entradas del scoring del motor

Sección del documento: 10.4.7 (DER de negocio)

Dice hoy: El DER define TipoMantenimiento con nombre,

CategoriaIncidencia con nombre y descripción, y PrioridadIncidencia con nombre y nivel.

Tiene que decir:

- TipoMantenimiento.es_preventivo (booleano).
- CategoriaIncidencia.id_especialidad (referencia a Especialidad, opcional).
- PrioridadIncidencia.horas_objetivo (entero, opcional).

Por qué: Cada uno resuelve algo que sin el atributo no se puede calcular.

La regla MANTENIMIENTO_VENCIDO cuenta los días desde el último mantenimiento **preventivo**. Un correctivo es la reparación posterior a la falla, no el mantenimiento que se dejó de hacer: si contara, el equipo que más se rompe sería el que nunca dispara la alerta, que es lo contrario de lo que la regla busca. Sin el atributo habría que deducirlo del nombre del tipo, y el nombre lo puede cambiar la organización.

La asignación necesita saber qué especialidad resuelve la categoría de la incidencia: es el dato central del contexto que se le manda al proveedor. Sin él, la asignación sólo podría mirar la carga de trabajo.

horas_objetivo es el objetivo de resolución pactado por prioridad, y es lo que permite marcar una incidencia como fuera de objetivo en el listado y en los indicadores de RF-13. Es opcional a propósito: la prioridad más baja no tiene objetivo pactado, y una prioridad sin objetivo no puede incumplirlo. Guardarlo como cero significaría lo contrario.

D-13 · El estado de la incidencia avanza al asignarse

Estado: aplicado — verificado el 09/09: el documento describe los siete estados y las ocho transiciones, y dice que la incidencia «pasa a Asignada cuando la asignación automática encuentra técnico»

Sección del documento: 10.2.2 (RF-08), 10.3 (CU-007), 10.5.1.4

Dice hoy: El caso de uso de asignación automática registra el técnico

responsable de la incidencia.

Tiene que decir: Al quedar con técnico, la incidencia pasa del estado inicial

—«Pendiente de asignación»— al siguiente estado no terminal, «Asignada». La reasignación manual sólo produce ese cambio si la incidencia seguía en el estado inicial: si el técnico ya la había puesto «En curso», reasignarla no la hace retroceder. Si la regla de respaldo no asignó a nadie, la incidencia permanece pendiente de asignación.

Por qué: Apareció al recorrer la pantalla: todas las incidencias mostraban

«Pendiente de asignación» aunque tuvieran responsable asignado, y el estado dejaba de significar algo. El estado se resuelve por posición en el catálogo y no por nombre, porque la organización puede renombrar sus estados y el alta de incidencias no se puede romper por eso.

D-14 · Precios de los planes y sueldos del equipo

Estado: aplicado — el documento ya trae 89.000 / 239.000 / 69.000 y no menciona los precios anteriores

Sección del documento: 5.3.6 (precios), 6.5.3 (márgenes), 7.6 (plantilla),

7.7 (punto de equilibrio)

Dice hoy: Plan Inicial ARS 34.900 con 15 equipos incluidos y ARS 1.800 por

equipo adicional; Plan Estándar ARS 99.900; Plan Partner ARS 27.900. Punto de equilibrio en 220 clientes. La plantilla de 7.6 no declara los sueldos.

Tiene que decir: Plan Inicial ARS 89.000 con 15 equipos incluidos y ARS 4.200

por adicional; Plan Estándar ARS 239.000 con 50 incluidos y ARS 3.500 por adicional; Plan Partner ARS 69.000 con 30 incluidos y ARS 2.800 por adicional —precios de 2027—. El punto de equilibrio pasa a estar en el año 4 del modelo.

Por qué: Salió de construir el presupuesto financiero con sueldos de mercado

en lugar de los que tenía la primera versión del modelo.

La primera versión asignaba entre ARS 700.000 y 900.000 de sueldo bruto a cada rol. A cualquier tipo de cambio plausible eso son unos USD 500 a 600 mensuales: por debajo de un desarrollador junior. No era una cifra de mercado desactualizada. Sin estar dicho en ninguna parte, esa cifra era el **retiro de subsistencia de los socios fundadores**. El modelo ahora separa las dos cosas: el salario de mercado de cada rol, el que cobra un empleado desde que entra, y el porcentaje de ese salario que los socios efectivamente retiran, que va del 30 % en el año 1 al

100 % en el año 4. La brecha entre ambos es el aporte de los socios al proyecto, y es lo que la tasa de corte del VAN tiene que remunerar.

Con los sueldos corregidos, los precios del documento no sostienen el negocio en ningún año. Al cierre del año 4, con los 352 clientes que proyecta el propio embudo, los números daban:

Concepto	Año 4
Ingresos	ARS 256,4 M
Costos fijos	ARS 17,9 M
Costos variables	ARS 45,8 M
Costos de RRHH	ARS 511,7 M
Resultado operativo	ARS -319,1 M

El punto de equilibrio con esa estructura de costos está en **886 clientes**, y el embudo del documento llega a 352. El problema no estaba en la escala del embudo sino en que el ingreso por cliente resultaba de ARS 60.700 mensuales, unos **USD 19** al tipo de cambio proyectado, para un servicio que administra alrededor de dos docenas de equipos. Las herramientas de RMM del mercado cobran entre USD 2 y USD 4 por equipo por mes; el precio del documento equivale a **USD 1,2 por equipo**, alrededor de un tercio.

Los precios revisados llevan el abono a entre USD 2,5 y USD 3,1 por equipo, dentro de ese rango. Con ellos el año 4 cierra con un resultado operativo de **ARS +60,2 M** y es el primer año positivo del modelo.

Lo que queda por decidir y no se resolvió acá: el VAN a cuatro años sigue siendo negativo (ARS -194 M en el escenario base, con tasa de corte del 60 % nominal), porque la ventana del modelo termina justo en el año en que el proyecto se da vuelta. Es lo habitual en un negocio por suscripción, donde el repago llega después. Se resuelve extendiendo el horizonte a cinco o seis años o incorporando un valor terminal, que es la práctica estándar para valuar SaaS. Las dos opciones cambian la sección 7.7 y conviene elegir una sola.

Alcance del cambio: los precios están aplicados en el presupuesto financiero.

Falta bajarlos a la sección 5.3.6 del .docx, recalcular los márgenes de 6.5.3 y actualizar los datos de demostración del sistema (db/04-seed-negocio.sql, tabla PlanComercial), que hoy tienen los precios viejos. La pantalla de Organización los toma de la base, así que se corrige sola una vez cambiado el seed.

D-15 · Dos inconsistencias menores detectadas al construir

Estado: aplicado en el sistema — las dos correcciones son de código; el documento no las describe ni tiene por qué

a) La columna «Responsable» del Dashboard muestra técnicos.

Sección: 10.5.1.1 (pantalla de Dashboard)

En la tabla «Equipos con mayor riesgo», la columna rotulada **Responsable** trae nombres de técnicos —Martín Acosta, Julieta Pérez, Lucas Romano—. Pero en esa tabla cada fila es un **equipo**, y un equipo no tiene técnico asignado: lo tienen sus incidencias. El atributo del modelo es `Equipo.id_responsable`, que apunta a la persona que usa el equipo y responde por él.

Se mantiene el rótulo «Responsable» y se corrige el dato: la columna muestra el responsable del equipo. Poner ahí al técnico obligaría a elegir uno entre los de todas sus incidencias, que es una pregunta sin respuesta única.

b) La meta de 1.000 equipos al mes 12 no cierra con el segmento.

Sección: 5.3.3 (metas cuantitativas), 4.2 (segmentación)

El documento fija 25 clientes y 1.000 equipos administrados al cierre del primer año. Son 40 equipos por cliente en promedio. El Segmento A se define como PyMEs de 5 a 50 empleados: una empresa de ese tamaño tiene del orden de 20 a 25 equipos —una computadora por persona más impresoras, servidor, UPS y red—, y 40 sólo se alcanza en el extremo superior del rango.

El presupuesto financiero, con parques de 18 a 24 equipos para el Plan Inicial y la mezcla de planes que proyecta el embudo, llega a **386 equipos** al mes 12, no a 1.000. La diferencia no viene del embudo. Son los 40 equipos por cliente promedio, que contradicen la propia definición del segmento.

Se recomienda **bajar la meta del primer año al orden de 450 equipos** y declarar los 1.000 como objetivo del segundo año. La alternativa —sostener los 1.000— obliga a que la mezcla se corra hacia el Plan Estándar, que apunta a empresas de 50 a 150 equipos y por lo tanto a otro segmento.

c) El margen de 6.5.3 se calcula sobre el abono solo.

Sección: 6.5.3 (justificación económica de los precios)

El margen del 74 % al 76 % está calculado sobre el abono base, sin contar los equipos adicionales. En el Plan Inicial con los precios revisados, un cliente de 24 equipos paga 89.000 de

abono más $9 \times 4.200 = 37.800$ de adicionales: el 30 % de su factura queda afuera del cálculo. Con parques más grandes la proporción crece. Corresponde recalcular el margen sobre el ingreso total por cliente, que es además el número que sostiene la afirmación.

D-16 · Cifrado reversible y clave de API por organización

Estado: aplicado — verificado el 09/09: «su clave de API. La clave entra y no vuelve a salir: ningún endpoint la devuelve, ni truncada»

Secciones: 10.2.2 (RNF-04, seguridad) · 10.4.7 (DER de negocio, tabla ConfiguraciónAsignación) · 10.5.1.7.7 (pantalla de integración de IA)

El documento describe la seguridad del sistema en términos de autenticación y permisos, y menciona el hash de contraseñas. No dice nada sobre credenciales de terceros, que son un dato distinto: una contraseña no hace falta recuperarla —el sistema sólo la verifica—, pero la clave de API del proveedor hay que enviarla en cada llamada, así que hay que poder recuperarla. Un hash no sirve para eso.

Se incorpora un servicio de cifrado reversible con AES-256-GCM, separado del hash y sin reemplazarlo, y la clave de API pasa a guardarse cifrada **por organización** en vez de leerse de una variable de entorno del servidor. El cambio no es sólo técnico: con la clave en el entorno, todas las organizaciones del plan Partner comparten la credencial de quien opera el servidor y el consumo de una se factura contra la cuenta de otra.

La pantalla 10.5.1.7.7 suma la carga de la clave. No suma su lectura: no hay forma de recuperarla una vez guardada, y lo que se muestra es de dónde sale la que está en uso.

El fundamento completo está en el ADR 0010.

D-17 · Cuándo corre el análisis predictivo

Estado: aplicado — escrito el 09/09 en 10.5.1.5: los tres momentos en que corre -al registrar incidencia, al registrar mantenimiento y por tarea diaria-, mas la ejecución a pedido, y por que la fecha entra por parametro

Sección: 10.2.2 (RF-11) · 10.3 (diagrama de componentes)

RF-11 dice que el sistema genera alertas cuando se cumplen las condiciones de una regla, pero no dice **cuándo** las evalúa. La omisión no es menor: construido al pie de la letra, el motor terminó evaluando sólo cuando alguien abría la pantalla y pedía el análisis, con lo cual la alerta llegaba después de que la persona hubiera ido a buscarla.

Se precisa el requisito: la evaluación se dispara al registrar una incidencia o un mantenimiento —los dos hechos que cambian los datos que las reglas miran— y además por un barrido programado diario, porque tres de las cinco reglas se cumplen por el paso del tiempo sin que ocurra ningún hecho.

El diagrama de componentes suma el proceso en segundo plano, que hoy no figura.

Ver ADR 0011.

D-18 · La patente lleva nivel de acceso

Estado: aplicado — escrito el 09/09 en 10.3.3: que la patente lleva nivel de acceso, que distingue lectura de escritura y de operacion critica, y para que sirve

Sección: 10.4.7 (DER de servicio, tabla Patente)

El DER de servicio del documento define Patente con nombre, clave y módulo. El modelo de referencia de 3.º año agrega un atributo TipoAcceso, que en ese proyecto está declarado y vale 1 para las nueve patentes, con lo cual no distingue nada.

Se incorpora el atributo y **se lo puebla**: 11 patentes de lectura, 8 de escritura y 7 de administración. Con veintiséis patentes, la lista de permisos de un usuario deja de responder «qué puede llegar a hacer esta persona» —hay que leer los veintiséis nombres—, y el nivel sí lo responde de un vistazo.

D-19 · La bitácora se filtra por usuario y los errores tienen su vista

Estado: aplicado — el filtro de bitácora por usuario y la separación de los errores del sistema están escritos en 10.3.2.3, y la pantalla de Errores ya es el apartado 10.5.1.9.6, con su captura y su sección en el manual del administrador

Secciones: 10.2.2 (RF nuevo) · 10.5.1.7.5 (pantalla de bitácora) · 10.5 (pantalla nueva)

Dos capacidades que el modelo de referencia pide como casos de uso de arquitectura — CU.Arq.002 y CU.Arq.007— y que el documento no describe.

Filtro por usuario en la bitácora. La pantalla 10.5.1.7.5 filtra por fecha y por tipo de evento. Falta la consulta que se hace de verdad cuando algo salió mal, que no es «qué pasó el martes» sino «qué hizo esta persona».

Pantalla de errores del sistema. Las excepciones no controladas ya se asientan en la bitácora con su traza completa, y no había forma de verla: la consulta general no la devuelve, y con

razón —una lista de auditoría con volcados de pila es ilegible—. Se agrega una pantalla que sale de la misma bitácora inmutable, filtrada por criticidad y mostrando la traza, con **patente propia**: una traza nombra tablas, rutas y versiones de bibliotecas, que sirven tanto para diagnosticar como para atacar.

No se agrega una tabla de errores. Un segundo registro de lo mismo se desincroniza del primero, y la bitácora ya tiene la propiedad que importa acá: nadie puede alterarla ni borrarla, ni conectándose directo a la base.

D-20 · El ciclo de vida completo de la incidencia

Estado: aplicado — el documento describe los estados Resuelta, Cerrada y Anulada

Secciones: 10.2.3 (CU-008, atención de incidencias) · 10.2.2 (RF-07) · 10.4.7 (DER, tabla Incidencia) · 10.5.1.3.2 (pantalla de detalle)

El documento describe el alta y la asignación de la incidencia con detalle, y da por supuesto lo que pasa después. Construido al pie de la letra, el resultado fue un sistema donde la incidencia se registraba y se reasignaba, y nada más: de los siete estados del catálogo sólo dos eran alcanzables, y `diagnostico`, `solucion` y `fecha_resolucion` existían en la base sin que nada las escribiera.

No es un detalle de implementación. Sin fecha de resolución no hay tiempo de reparación, y sin tiempo de reparación el SLA de cuatro horas que promete el propio documento no se puede medir.

Se precisa el caso de uso con la máquina de estados completa —ocho transiciones sobre siete estados—, las reglas de quién puede atender, y la exigencia de diagnóstico y solución para resolver. La pantalla 10.5.1.3.2 suma el panel de atención.

Ver ADR 0012.

D-21 · Los reportes son PDF generados en el servidor

Estado: aplicado — el documento tiene la seccion 10.2.2 «Reportes exportables»

Secciones: 10.2.2 (RF-14) · 10.2.5 (entorno técnico) · 10.5 (pantalla nueva)

RF-14 pide reportes «filtrados y exportables» y no dice en qué formato. Se define: **PDF, generado en el servidor**, con tres reportes —parque, incidencias y mantenimientos—.

El entorno técnico suma la dependencia y su licencia: QuestPDF Community, elegida porque no necesita binarios nativos ni un navegador headless y por lo tanto funciona dentro del contenedor de la API sin engordarlo.

El motivo de generarlo en el servidor no es técnico sino de trazabilidad: el reporte lleva impresos la fecha, el autor y los filtros aplicados, y cada emisión queda en la bitácora. Un PDF armado en el navegador lo firmaría el navegador.

Ver ADR 0013.

D-22 · Cuatro casos de uso que el modelo de permisos prometía

Estado: aplicado — CU-014 a CU-017 escritos el 09/09 con el mismo formato que los trece anteriores: tabla descriptiva, flujo normal y flujo alternativo, y sus cuatro filas en el índice de 10.4.2

Secciones: 10.2.3 (CU nuevos) · 10.4.7 (DER de servicio, Patente) · 10.5 (pantallas nuevas)

Una auditoría del 03/09 buscó en el código las **patentes declaradas que ningún endpoint exige**. Aparecieron cuatro, y cada una era un caso de uso que el modelo de permisos prometía y el sistema no tenía:

Patente	Qué faltaba
INCIDENCIA_ATENDER	El ciclo de vida (D-20)
EQUIPO_CAMBIAR_ESTADO	Pasar un equipo a «en reparación». Sólo existía darlo de baja, y el motor predictivo no evalúa los equipos no operativos: el estado cambia qué se evalúa
REPORTE_GENERAR	Los reportes (D-21)
HISTORIAL_VER_ORGANIZACION	El historial técnico del parque, con incidencias y mantenimientos en una sola línea de tiempo

Y dos códigos de evento del catálogo que nunca se escribían: LOGOUT —la bitácora tenía todas las entradas y ninguna salida— y RESTORE, con la restauración de un respaldo implementada en el servicio y sin endpoint que la llamara. Un respaldo que no se probó restaurando es una copia, no un respaldo.

El documento suma los cuatro casos de uso y las dos pantallas nuevas —Historial del parque y Reportes—.

Lo que esta auditoría dice del proceso, y conviene decirlo en la defensa: es la tercera vez que la misma técnica encuentra algo, y las tres veces el hallazgo fue una **ausencia**. Ninguna prueba podía fallar, porque no había código roto.

D-23 · Valor terminal en la evaluación financiera

Estado: aplicado — el documento tiene la sección 7.7.1 «Valor terminal e interpretación de los indicadores»

Secciones: 6.5 (evaluación económica) · 6.5.4 (indicadores)

El VAN a cuatro años quedaba negativo porque la ventana termina justo cuando el proyecto da vuelta: el año 4 cierra positivo y después no se cuenta nada. Se incorpora el **valor terminal** al cierre del horizonte, como múltiplo de los ingresos anuales.

Múltiplo y no perpetuidad de Gordon por un motivo aritmético: con una tasa de corte nominal del 60 %, el denominador ($r - g$) queda dominado por la inflación y la perpetuidad devuelve un número sin significado económico. El múltiplo es además cómo el mercado fija precio a un prestador de servicios administrados.

El resultado, con los números recalculados y no estimados:

Indicador	Valor
VAN con valor terminal (1,5x)	-48 millones
VAN sin valor terminal	-193 millones
Valor terminal descontado	+146 millones
TIR	41 % nominal · 5 % real
El VAN se vuelve positivo a	2,0x

Se toma **1,5x**, el **piso** del rango de mercado (1x a 3x), y el resultado sigue siendo negativo. La sección tiene que decir eso y no otra cosa: el proyecto crea valor en términos reales —la TIR gana cinco puntos sobre la inflación— pero no alcanza el retorno que la prima de riesgo de la tasa de corte le exige. Es la diferencia entre un negocio que no sirve y uno que sirve menos de lo que se le pidió.

La tabla de sensibilidad queda en el libro para que la conclusión no dependa de haber elegido bien un número que nadie puede conocer.

D-24 · Las pantallas del capítulo 10.5 son capturas del sistema

Estado: aplicado — el capítulo 10.5 lleva capturas del sistema

Secciones: 10.5 completa (10.5.1.1 a 10.5.2.4)

El capítulo mostraba los mockups de la etapa de diseño. Servían para acordar la interfaz antes de escribirla, pero ya no describen el sistema: los datos son inventados, los iconos aparecen

como cuadrados vacíos porque falta la tipografía de iconos, y la cifra de equipos no coincide con la de la base.

Las diecisiete pantallas del capítulo pasan a ser capturas del sistema funcionando, con los datos de la organización de demostración. Se sacan con `frontend/scripts/capturar-pantallas.mjs`, que entra con cada perfil, recorre las rutas y guarda un PNG por pantalla al mismo tamaño y con el mismo recorte. Se automatiza para que, cuando una pantalla cambie, se vuelvan a sacar todas con un comando. Una captura vieja en un documento queda ahí sin que nadie la revise, y describe un sistema que ya no existe.

Como efecto colateral, sacar las capturas encontró un defecto real. La ficha de equipo seguía mostrando los textos de una fase anterior —«todavía no está implementado el módulo de incidencias» y «requiere el motor predictivo»— cuando los tres módulos ya estaban. La ficha ahora muestra el historial de incidencias y mantenimientos en una sola línea de tiempo y el puntaje de riesgo con la participación de cada regla, calculada como peso por intensidad sobre el total.

D-25 · Los diagramas de paquetes, componentes y despliegue salen del código

Estado: aplicado — el documento tiene los diagramas de paquetes, componentes y despliegue

Secciones: 10.4.4 · 10.4.5 · 10.4.8

Los tres diagramas eran de la etapa de diseño y contradecían la solución construida. El de paquetes mostraba **MySQL** como base y una estructura de capas que no es la que tiene el código, que se organiza en Domain, DAO, BLL, Service y Api.

Se reemplazan por los diagramas de `docs/tecnico/DIAGRAMAS.md`, escritos en Mermaid y convertidos a imagen con `docs/tecnico/_diagramas-a-png.py`. El diagrama del documento y el del repositorio salen del mismo texto, así que no pueden decir cosas distintas.

D-26 · Numeración de figuras y tablas, e índices propios

Estado: descartado por decision del autor — los epígrafes numerados se quitaron a proposito de la version que se entrega

Secciones: documento completo · índice general

El documento tenía 54 figuras y 70 tablas sin numerar, y no había forma de citar ninguna desde el cuerpo del texto. Tampoco había números de página en el pie, con lo cual el índice general no servía para encontrar nada en la versión impresa.

Se agrega:

- Epígrafe numerado debajo de cada figura y arriba de cada tabla, con el número como campo SEQ de Word y no como texto. Con el campo, la numeración se corrige sola si se inserta una figura en el medio, y Word puede armar los índices.
- **Índice de figuras** e **índice de tablas** después del índice general.
- Número de página en el pie, salvo en la portada.

También se retira del documento el estilo font-claude-response-body, que había quedado adherido a cuatro párrafos del capítulo 1 al copiarlos desde una ventana del navegador, junto con su definición en la hoja de estilos.

D-27 · Los documentos satélites toman el formato del documento principal

Estado: aplicado fuera del .docx — es el formato de los documentos satélite, no del documento principal

Secciones: entregables (no altera el cuerpo del TFI)

Los documentos que acompañan al trabajo —manuales, documentación técnica, gestión— existían sólo como archivos de texto en formato Markdown. Markdown es una buena fuente, porque se versiona y se compara entre revisiones, pero no es un entregable: quien lo abre encuentra los asteriscos y los guiones a la vista.

Todos se generan ahora como documentos de Word con el mismo formato que TFI-PredictIT.docx: Calibri 12, justificado, interlineado 1,15 y márgenes de 3 cm a los lados. Cada uno lleva portada con los datos de la cátedra, índice, encabezado, numeración de páginas, epígrafes numerados y un apartado de fuentes en normas APA con las obras que efectivamente cita.

Se agrega además la **presentación de la defensa** (docs/defensa/), de veinte diapositivas, con el guion de cada una en las notas del orador y las cifras leídas del backlog, de la corrida de pruebas y de este mismo registro.

D-28 · Rediseño de la interfaz y los temas

Estado: aplicado — el 09/09 se eligió la **dirección financiera** y quedó aplicada en todas las pantallas, con **dos temas en lugar de tres**: Oscuro y Claro, más la opción de seguir al sistema operativo. Las tres densidades se quedan. Las veintiuna capturas del capítulo 10.5 se rehicieron todas juntas sobre el sistema con el rediseño puesto, y la pantalla de Apariencia es ahora el apartado 10.5.1.9.9

Secciones: 10.5 (pantallas) · 10.3.2 (componentes de la arquitectura base)

Segunda iteración del sistema visual sobre el canvas de diseño. Baja como una capa de tokens que reemplaza entera la lista de variables: escala de espaciado de nueve pasos, ocho tamaños tipográficos, radios que codifican qué clase de cosa es cada elemento, y tres temas en lugar de seis.

Tercera iteración, del 09/09 —la que está aplicada hoy—. De las dos direcciones que devolvió el canvas se eligió la financiera. Cambian la tipografía (Manrope, con IBM Plex Mono para todo lo que se compara en columna), la paleta de los dos temas, la escala de espaciado —siete pasos donde el número es el valor dividido cuatro, en vez de nueve ordinales— y los radios. La tipografía pasa de ocho tamaños numéricos a **seis roles con nombre**: el nombre dice dónde va cada uno, y así el que escribe una pantalla nueva no elige entre 13 y 14 px por gusto. Había once tamaños distintos entre 11 y 14 px escritos a mano en el CSS; la separación entre cuerpo y metadato pasa a ser el peso y el color.

Y tres reglas que no son tokens pero se aplican igual, porque cada una reserva un canal de significado: **el azul es acción y marca y nunca estado**, lo que deja libres el rojo, el ámbar y el verde para el riesgo; **el violeta es sólo IA**; y **el glow vive en el fondo de página y nunca en un dato**, para que ninguna tarjeta le compita en brillo a la alarma.

De tres temas a dos. Clásico, Grafito y Carbón se retiran; quedan Oscuro y Claro más «seguir el sistema». El argumento es medible: con seis temas hay treinta y seis pares de color que verificar cada vez que se toca un token, y con dos hay doce. Lo que la gente personaliza de verdad es «me molesta el brillo» o «no veo con poca luz», y eso lo resuelve claro contra oscuro; lo que sí varía por persona es la densidad, y por eso las tres se quedan. Quien tuviera uno de los tres guardado en el navegador queda migrado —Clásico a Claro, Grafito y Carbón a Oscuro— y ve un aviso que se cierra y no vuelve. El seed de traducciones pasó a borrar las claves que se retiran: antes sólo insertaba y actualizaba, así que un texto dado de baja quedaba cargado para siempre.

Dos contrastes de la propuesta no daban, medidos. El archivo del canvas anotaba 5,4:1 para el blanco sobre el azul del acento; medido da 4,42:1, o sea que el rótulo del botón primario no

llegaba a AA. Un punto más oscuro lo lleva a 4,88:1 sin perder el anillo de foco. Y el borde de campo daba 1,60:1 en oscuro y 1,47:1 en claro, contra los 3:1 que pide la WCAG 1.4.11 para el límite de un control: el relleno del campo no ayuda, porque sobre la tarjeta da 1,08:1, así que el borde es lo único que dice dónde empieza. Los dos se corrigieron.

Para que esto no vuelva a depender de una tabla escrita a mano, `docs/design/_contraste.py` recorre la capa de tokens y falla si algún par declarado baja del piso. Son treinta y cuatro pares por tema.

El sistema pasa a adaptarse a la pantalla. No tenía una sola media query: en un teléfono la barra lateral se comía 232 de los 375 px y la tabla salía de la pantalla sin forma de alcanzarla. Ahora sigue los cuatro cortes de la dirección —1366 de referencia, 1180 las pantallas de detalle a una columna, 1024 la barra reducida a un riel de íconos, 768 el riel como barra horizontal arriba, 480 el piso táctil de 48 px—. Las tablas **no** se colapsan a tarjetas: una lista de incidencias se compara de arriba hacia abajo y en tarjetas se pierde justamente la comparación, así que scrollean en horizontal dentro de su tarjeta. Todo eso vive en `frontend/src/estilos/responsive.css`, que va último y sólo sobrescribe: comentar ese `import` devuelve el layout fijo sin tocar un solo componente.

Para que se pudiera adaptar hubo que sacar las grillas de los estilos en línea, porque un `style={{...}}` le gana a cualquier media query. Eran siete pantallas con el mismo patrón —contenido a la izquierda, panel de contexto a la derecha— y siete anchos distintos elegidos a ojo: 340, 360, 380, 392, 400 y 420 px, y ninguno significaba algo distinto de los otros. Quedó uno.

Una dirección que no existe deja de redirigir en silencio. `/` y `/login` siguen llevando adentro, pero `/pantalla-inventada` muestra un 404 que conserva la barra lateral —la sesión sigue abierta— y dice qué dirección falló. Y un registro que no está deja de verse como una falla del sistema: abrir una incidencia anulada mostraba un cartel rojo, cuando no se rompió nada. Un rojo que aparece sin problema entrena a la gente a ignorar los rojos.

Se dan de baja tres temas claros —índigo, bosque y pizarra—. Con el acento restringido a lo accionable dejaron de distinguirse del clásico, y en un selector de seis opciones que se ven casi iguales elegir no significa nada. A quien tuviera uno guardado en el navegador se lo lleva al que lo reemplaza; sin eso, la pantalla de Apariencia le mostraría una selección que ya no está en la lista.

Corrección de contraste. El token `--tx3`, que llevan las etiquetas de columna de 11 px y los metadatos de 12 px, daba 3,15:1 sobre la tarjeta blanca. Es texto chico sin excepción de umbral, así que el piso de 4,5:1 que pide WCAG AA no se cumplía. El valor nuevo da 5,09:1.

El nivel de riesgo deja de depender del color. Se codifica cuatro veces: un glifo de tres escalones con uno, dos o tres llenos; el puntaje en monoespaciada; el rótulo ALTO, MEDIO o BAJO; y recién ahí el color. Antes la tabla mostraba un punto de color y un número, con lo cual para alguien que no distingue el rojo del ámbar un 68 y un 92 se veían igual de graves. El componente vive en un solo archivo: estaba duplicado en el tablero y en el análisis predictivo, con su propia función de color en cada uno.

Al aplicar la capa nueva apareció una dependencia que no estaba dicha en ninguna parte. El archivo de tokens anterior no tenía sólo tokens: llevaba también el reset del documento, la tipografía del cuerpo y el estilo de los enlaces. Reemplazarlo por el del canvas dejó la aplicación entera en serif y con todos los enlaces subrayados. Esa capa base pasa a vivir en su propio archivo, así la próxima bajada del canvas no se la lleva puesta.

D-29 · Las pantallas del 10.5 se describen, no sólo se muestran

Estado: aplicado — las pantallas del 10.5 llevan su descripción, no solo la captura

Secciones: 10.5.1.1 a 10.5.2.4

Cada apartado del capítulo tenía el título, la imagen y el epígrafe. Nada más. Diecisiete pantallas sin una línea que diga qué se hace en cada una, con lo cual quien lee el documento sin haber visto el sistema tiene que deducirlo de la captura.

Se agrega un párrafo por pantalla. Dice qué se hace ahí y, cuando la hay, la decisión de diseño que explica por qué se ve así: por qué el formulario de reporte no pide categoría, por qué la vista del solicitante es deliberadamente pobre en información, por qué la bitácora no ofrece una acción de edición. Eso es lo que no se puede deducir mirando la imagen.

D-30 · Verificación: OWASP, cobertura y carga

Estado: aplicado fuera del .docx — vive en el documento «Verificación del sistema», que se entrega aparte

Secciones: 10.2.2 (RNF) · anexo nuevo

Tres cosas que el documento afirmaba y que ahora están medidas, en un anexo propio (Verificación del sistema).

Revisión contra OWASP Top 10. Existe porque el ADR 0014 decide no usar un gestor de identidad externo: sin un producto de terceros que se ocupe de la seguridad, la revisión no la

hace nadie más. Recorre las diez categorías con el archivo donde está cada control y con lo que falta en cada una. Quedan abiertas dos: la configuración (A05) y el segundo factor (A07).

Cobertura de pruebas: 33,8 %, medida con coverlet y con línea de base. El número trajo un hallazgo que el promedio tapaba: la capa de negocio está al **2,3 %**, porque las pruebas de integración construyen los DAO directamente contra la base y no pasan por ella.

El requisito de carga del ADR 0008, medido. Estaba afirmado —500 activos y 5.000 incidencias con respuesta bajo dos segundos— y nunca se había verificado. Con el volumen completo, la peor medición es de **60 ms**. El informe dice también qué no mide: un solo cliente, secuencial, base local, nada sobre concurrencia.

D-31 · ADR 0014: gestión de identidad propia

Estado: aplicado fuera del .docx — es el ADR 0014, que se entrega en «Decisiones de arquitectura»

Secciones: 10.3.2.4 (seguridad)

El documento describe el modelo de familias y patentes sin decir por qué no se usa un gestor de identidad externo, que es la primera pregunta razonable en 2026 y que los requisitos de la cátedra mencionan como opción.

Queda registrado. El motivo principal no es técnico: el modelo de permisos es contenido de la materia, y delegarlo en Keycloak lo convertiría en un archivo de configuración. El ADR dice además qué se pierde, con la autenticación en dos pasos a la cabeza.

D-32 · Los diagramas salen de Enterprise Architect

Estado: aplicado — los 22 diagramas del documento salen del modelo de Enterprise Architect

Secciones: 10.4.4 · 10.4.5 · 10.4.8 · anexo de diagramas

La cátedra recomienda **Enterprise Architect**, así que los diagramas pasan a salir de ahí. Reemplazan a los que se generaban de Mermaid, que a su vez habían reemplazado a los originales de la etapa de diseño —el de paquetes decía MySQL y la solución usa SQL Server, delta D-25—.

No están dibujados a mano. docs/tecnico/_diagramas-ea.py construye el modelo por la API de automatización de EA —paquetes, elementos, conectores y la posición de cada cosa en cada diagrama— y le pide a EA que exporte la imagen. El motivo es el mismo por el que las capturas se automatizan: cuando el código cambia, los diagramas se rehacen con un comando.

Son **veinticinco**, y cubren los **veintidós apartados** del documento que llevan diagrama:

Apartado	Diagrama
10.3.2.3.1	Clases de Bitácora
10.3.2.5	Usuario, Patente y Familia: el composite
10.4.2	Casos de uso por perfil
10.4.3.1 a 10.4.3.13	Una secuencia por caso de uso
10.4.4 · 10.4.5 · 10.4.8	Paquetes, componentes y despliegue
10.4.6	Clases de la solución, por capa
10.4.7	Entidad-Relación, una por base

Los tres restantes —modelo de dominio, ciclo de vida de la incidencia y la secuencia general— van al anexo técnico de diagramas.

Dos cosas que valen la pena señalar. **Los dos DER no se dibujan**: se leen de las bases con `sys.tables` y `sys.foreign_key_columns`, así que muestran las tablas, las columnas, los tipos y las foráneas que existen de verdad —22 tablas y 34 foráneas en negocio, 15 y 16 en servicio— y no las que el diseño se propuso. Y **las trece secuencias por caso de uso** salen de leer el controlador y el business de cada uno: a un diagrama de secuencia que no coincide con el código se le cree igual, y ahí está el daño.

Los de estados y secuencia son UML de verdad —máquina de estados, y líneas de vida con barras de activación— dibujados por la herramienta y no por un script imitando su estilo.

El Mermaid se retira. Convivían dos representaciones del mismo modelo, y mantenerlas sincronizadas a mano es exactamente lo que este proyecto viene evitando en todo lo demás. Gana EA porque es la herramienta que pide la cátedra.

El modelo se entrega junto con las imágenes, en `docs/diagramas-ea/PredictIT.eapx`. Se abre en EA, se puede seguir editando a mano, y de ahí salen exactamente los PNG que están en el documento.

D-33 · El modelo se recalibra: macro del REM y estructura financiable

Estado: aplicado — el margen de 6.5.3, el punto de equilibrio y los indicadores de 7.7, y la plantilla, las cargas y el encuadre laboral de 7.6 están en el .docx (09/09)

Sección del documento: 6.5.3 (márgenes) · 7.6 (plantilla y calendario de incorporaciones) · 7.7 (presupuesto y punto de equilibrio)

Dice hoy: La plantilla de 7.6 incorpora seis personas con altas en los meses

1, 6, 9, 9 y 18. Los supuestos macroeconómicos del presupuesto estaban marcados como provisorios (inflación 35 % en 2027, tipo de cambio 1.900). Los tres conceptos del costo variable se ajustaban por inflación. El margen de contribución de 6.5.3 dice entre 74 % y 76 %. Los socios retiran del 30 % al 100 % de su salario de mercado.

Tiene que decir:

Los supuestos macro salen de una fuente citada. El **Relevamiento de Expectativas de Mercado del BCRA**, informe de agosto de 2026 —publicado el 4 de septiembre sobre 47 participantes—: inflación 20,5 % en 2027 y 14,1 % en 2028, tipo de cambio 1.800 y 2.050, TAMAR de referencia 21 % y 16 %. El REM releva tres períodos anuales para precios y dos para el tipo de cambio, así que **2029 y 2030 son extrapolación propia de la misma senda** y cada fila lo declara. Los aumentos de precios de lista y de salarios pasan a ser política propia definida *sobre* la inflación del REM —tres y cinco puntos— en lugar de valores sueltos.

El calendario de incorporaciones se ata a la facturación. Soporte en el mes 12, colaborador de desarrollo en el 20, marketing a medio tiempo —\$ 1.100.000— en el 30, segundo soporte en el 42. Tres personas al cierre del primer año, cuatro al segundo, cinco al tercero y seis al cuarto.

El retiro de los socios arranca más abajo: 15 %, 35 %, 60 % y 100 % del salario de mercado, en lugar de 30 / 55 / 80 / 100.

El costo variable se ajusta por el índice que le corresponde: infraestructura y servicio de IA **por tipo de cambio**, porque se pagan en dólares; soporte y onboarding por el índice de salarios, porque es trabajo local.

El margen de contribución de 6.5.3 pasa a ser del orden del 90 % en las tres líneas, que es lo que da el cálculo con los precios revisados en D-14. El 74–76 % que dice hoy corresponde a los precios anteriores.

Por qué: con la estructura anterior la pérdida acumulada de los tres

primeros años pasaba de 400 millones de pesos. Eso no es un proyecto que dos socios financien: es una ronda de inversión, y el trabajo se defiende como algo que se puede arrancar.

Con la **misma** proyección de ingresos —no se tocó una sola hipótesis de ventas— la recalibración deja:

	2027	2028	2029	2030
Ingresos (M ARS)	10,8	89,7	296,5	677,8
Egresos (M ARS)	30,6	137,8	273,3	457,1
Resultado operativo	-19,8	-48,1	+23,2	+220,7

El resultado operativo se vuelve positivo en **abril de 2029** con 131 clientes activos, y el capital operativo que hay que poner hasta ese punto es de **77,7 millones** —unos USD 40.100 convirtiendo el déficit de cada año al tipo de cambio de ese año—, o sea USD 20.000 por socio a lo largo de dos años y medio. Con la inversión inicial y los impuestos adentro sube a 91,3 millones, unos USD 23.600 por socio. Eso sí es financiable sin capital externo.

(Estas cifras incorporan la corrección de D-34: la versión anterior de este delta traía las del modelo con la línea del Segmento B en cero.)

La contracara hay que poder sostenerla en la defensa: **seis personas atienden 370 clientes y casi 9.900 equipos en 2030**, y dos de ellas son todo el soporte. Es coherente con un producto de mantenimiento predictivo, cuya promesa es que el trabajo por cliente no crece igual que la cartera; si no se cumpliera, la primera consecuencia sería adelantar el segundo colaborador de soporte.

Lo que esto deja abierto: las cifras de D-14 que hablan del año 4 y del VAN

—ARS +60,2 M de resultado y ARS -194 M de valor actual neto con tasa de corte del 60 % nominal— quedan **superadas por este delta**. La tasa de corte se rehízo en D-34: pasa a calcularse como la inflación del año 1 más una prima de riesgo de 20 puntos, o sea 40,5 %. El VAN, la TIR y el repago van en la Entrega 2 y se calculan con estos supuestos.

D-34 · El modelo dejaba dos supuestos afuera

Estado: aplicado — corregido en el presupuesto financiero, en el plan de la Entrega 1 y en el .docx. El punto de equilibrio de 7.7 ya dice las cifras del modelo recalibrado: 135 clientes pagos en mayo de 2029, ARS 156,2 millones de capital —USD 77.000, o USD 38.500 por socio— medidos como el peor punto del flujo acumulado, que es la magnitud relevante porque el proyecto sigue consumiendo caja después de arrancar

Sección del documento: 5.3.5 (ciclo de venta del Segmento B) · 7.7

(presupuesto y punto de equilibrio)

Dice hoy: La sección 5.3.5 describe el Segmento B con un ciclo de venta de

60 a 90 días y el primer cliente en la Etapa E3. El plan de la Entrega 1 declara que las altas del Plan Estándar son 1,2 / 4,8 / 9,6 / 14,4 por año.

Tiene que decir: lo mismo. Lo que estaba mal no era el documento: **era el modelo, que no lo aplicaba**.

Qué pasó. La fórmula de altas del Plan Estándar armaba la referencia con el ayudante que apunta a la hoja *Hipótesis*, pero la fila de entrada estaba en *Proyección de ventas*. El número de fila coincidía con «Crecimiento mensual de visitas» de Hipótesis, así que el modelo daba de alta **0,12 clientes del Segmento B en el año 1 en lugar de 1,2**, y 0,08 en el año 2 en lugar de 4,8. La línea entera valía cero y **ninguna celda daba error**: el libro calculaba perfecto y el Plan Estándar —el de mejor precio por equipo, con parques de 90 a 120 equipos— no existía.

Con la referencia arreglada, el Segmento B llega a 18 clientes activos al año 4 y el modelo entero cambia de escala:

	Antes	Corregido
Ingresos año 4 (M ARS)	548,0	677,8
Resultado operativo año 4 (M ARS)	+97,5	+220,7
Primer mes positivo	agosto de 2029	abril de 2029
Capital operativo necesario (M ARS)	116,8	77,7
Clientes al año 4	353	370
Equipos al año 4	7.722	9.866

Y una segunda cosa que la misma revisión encontró. La inflación no movía nada. Los aumentos de precios de lista y de salarios estaban tipeados con la cuenta ya hecha —23,5 % y 25,5 %, que son la inflación más 3 y más 5 puntos—, y las filas de inflación de 2028 a 2030 no las leía ninguna fórmula. La tasa de corte estaba en 60 %, que era una inflación del 35 % —la provisoria, ya reemplazada— más 25 puntos. Ahora:

- la **prima** es la entrada y el aumento se calcula: cambiar la inflación mueve precios y salarios, que es lo que el libro declara en su portada;
- la **tasa de corte** es la inflación del año 1 más una prima de riesgo de 20 puntos: **40,5 %**, con un control al lado de cuánto le gana a la tasa de referencia del REM;
- la **participación de mercado** se contrasta en la hoja *Proyección de ventas* y no a mano en el plan: 1,61 % proyectado contra 1,1 % objetivo.

Cómo se evita que vuelva a pasar. docs/negocio/_verificar.py tiene un control nuevo: **toda celda de entrada tiene que ser leída por alguna fórmula**. Una entrada huérfana —cargada, pintada de amarillo, con su nota al lado y que el modelo ignora— es exactamente el error que estuvo acá, y no lo detecta ninguna verificación de fórmulas porque no hay fórmula rota. Quedan declaradas las excepciones informativas, con el motivo escrito.

D-35 · El presupuesto se alinea con el material de la cátedra

Estado: aplicado — el presupuesto y la Entrega 1 lo traen desde el 09/09, y el .docx desde el 11/09: la carga tributaria completa con el régimen de promoción y el encuadre CAPEX/OPEX están en 7.7, la tasa de corte se nombra como costo de oportunidad en 7.7.1, y las métricas por suscripción —margen bruto, rotación, vida media, LTV, CAC y su relación— cierran el apartado 6.5.3

Sección del documento: 7.6 (plantilla y cargas) · 7.7 (presupuesto, punto de equilibrio e indicadores) · capítulo 6 (métricas del negocio)

De dónde sale: las tres clases de negocio de 2026 —Presupuesto, Costos y precios, e Impuestos e indicadores de inversión—. El modelo cubría la estructura que pide la Clase 1, pero le faltaba buena parte de lo que piden las otras dos.

Qué se agregó:

Impuestos, que eran dos y ahora son todos. El modelo tenía ingresos brutos al 3 % y ganancias al 35 %, tipeados dentro del flujo de fondos. La hoja **Impuestos** liquida ahora IVA —débito, crédito e importación de servicios digitales por RG 4356/2018—, ingresos brutos repartidos por **Convenio Multilateral** entre CABA y Provincia, impuesto de sellos, impuesto al cheque con su 33 % a cuenta, retención a beneficiarios del exterior, y ganancias con el régimen de la **Ley de Economía del Conocimiento**: reducción de hasta 60 % y bono de crédito fiscal de hasta 70 % de las contribuciones patronales. Los tributos que **no** aplican —aduaneros, patentes, inmobiliario, bienes personales— están nombrados con el motivo.

Dos decisiones de modelado que hay que poder defender: el **IVA no entra al flujo como costo**, porque es neutro para un responsable inscripto, y el **bono de crédito fiscal se parte** entre la porción que cancela Ganancias —que sí baja el costo— y la que cancela IVA —que mejora la caja—. Sumarlas infla un beneficio que no existe.

Cargas sociales completas. Faltaba la ART, que es obligatoria. Con ella y el seguro de vida el total pasa de 23,00 % a **28,30 %**, dentro del 25 % a 30 % que da la clase. Se declara además el encuadre laboral: fuera de convenio, bajo la LCT 20.744, porque no hay convenio colectivo propio de la actividad del software. Eso explica por qué los aumentos salariales del modelo son política propia y no una paritaria.

Gastos que faltaban, y CAPEX contra OPEX. Entran las **herramientas de IA del equipo** y la **conectividad del equipo remoto**. Cada concepto de costo fijo lleva ahora su clasificación CAPEX/OPEX y si se paga en el país o al exterior; lo segundo es la base del cálculo de importación de servicios digitales.

El flujo de fondos con la fórmula de la clase. Flujo de caja libre = EBIT $\times$ (1 – Ganancias) + Amortizaciones – Variación de capital de trabajo – CAPEX. El **capital de trabajo faltaba entero**, y no es menor: el cliente paga a 30 días, así que cada peso de crecimiento de la facturación inmoviliza un mes de ventas. Se separan además **EBITDA** y **EBIT**, que antes estaban fundidos en una sola fila llamada UAll.

Indicadores completos. A VAN, TIR y período de repago se suman **ROI** e **índice de rentabilidad**, que son los que la clase calcula en su ejemplo. Los tres se miden contra el **capital que hay que poner** —el peor punto del flujo acumulado, 139,6 millones— y no contra el CAPEX inicial: medido contra los 20,4 millones del período cero, el ROI daba 5.399 %, que es un número sin sentido. La tasa de corte se nombra por lo que es: **costo de oportunidad**.

Métricas del negocio por suscripción. Hoja nueva con MRR, ARR, ARPU, churn, vida media del cliente, margen bruto, **LTV**, **CAC**, la relación **LTV/CAC** y los meses de recuperación del CAC. Son las que la Clase 2 enumera para un SaaS y las que explican por qué el proyecto necesita capital antes de dar ganancia.

Gráficos. Seis, con las series tomadas por fórmula de sus hojas de origen.

Qué cambió en los números:

	Antes	Con impuestos, cargas y gastos completos
Egresos año 1 (M ARS)	30,6	32,5
Resultado operativo año 4 (M ARS)	+220,7	+205,0
Primer mes positivo	abril de 2029	mayo de 2029 , con 142 clientes
Capital total a poner (M ARS)	—	139,6 , unos USD 69.500
Presión tributaria sobre ventas	—	del 9,0 % al 5,7 %
LTV / CAC	—	de 3,5 a 8,2

Esos números son los de esta iteración y no los de hoy. La recalibración de D-33 y el arreglo de las tablas de 7.7 en D-41 los movieron después: el capital a poner quedó en **156,2 millones**, el primer mes positivo en mayo de 2029 con **135 clientes**, los egresos del año 1 en **32,4 millones**, el resultado del año 4 en **+169,6** y la relación LTV/CAC entre **3,3 y 7,7**. Se deja la tabla como estaba porque este registro es cronológico y muestra el efecto de este cambio; lo que vale para el documento es lo que dice 7.7, y eso lo verifica el control de alineación.

Y una corrección de fecha que no es menor. La Clase 1 alterna el orden de los grupos entre entregas: en la Entrega 1 va primero el Grupo 1 y en la **Entrega 2 va primero el Grupo 2, el 16 de octubre**. El proyecto tenía anotado el 23, que es la del Grupo 1.

D-36 · Alcance de la IA: la guía de reparación paso a paso

Estado: aplicado — el alcance de la IA dice tres funciones, el panel está en la pantalla de detalle con sus cuatro estados, y el documento ya tiene el RF-15 en 10.2.2, el CU-018 con su diagrama y sus dos flujos en 10.4.3.18, y el CP-15 con su resultado leído de la corrida

Secciones: 1.7 · 2.2.1.1 · 6.4.1 (alcance de la IA) · 10.2.2 (RF-15 nuevo) · 10.2.3 (CU-018 nuevo) · 10.4.3 (secuencia) · 10.5 (pantalla de detalle de incidencia) · 10.6 (casos de prueba)

Dice hoy: con el delta D-03 el alcance de la inteligencia artificial quedó

en dos funciones: **clasificación** del texto libre del solicitante (triage, categoría, prioridad y detección de duplicados) y **asignación** del técnico más adecuado.

Tiene que decir: tres funciones. Se suma la **guía de reparación paso a

paso**: dada una incidencia, el sistema sugiere al técnico un procedimiento de diagnóstico y reparación ordenado, con el riesgo declarado en cada paso que lo tenga y el criterio para dejar de intentar y escalar.

RF-15 · Sugerencia de procedimiento de reparación

El sistema debe poder sugerir, a pedido del técnico asignado a una incidencia, una guía de diagnóstico y reparación paso a paso, generada a partir del texto del reporte, la categoría, los datos del equipo y el historial de fallas resueltas de ese mismo equipo. Cada paso indica qué hacer, con qué detalle y qué riesgo tiene. La guía es una sugerencia: el técnico decide qué ejecutar.

CU-018 · Consultar la guía de reparación sugerida

Actor	Responsable Técnico
Precondición	La incidencia existe y el actor puede verla
Disparador	El técnico pide la guía desde el detalle de la incidencia
Flujo principal	1. El sistema arma el contexto con la incidencia, el equipo y hasta cinco fallas resueltas del mismo equipo en los últimos dos años. 2. Consulta al proveedor de IA configurado. 3. Renumerar los pasos y devuelve la guía. 4. Asienta la consulta en la bitácora con el proveedor que respondió.

Flujo alternativo	Si el proveedor no responde o el circuito está abierto, se devuelve la guía marcada como no disponible con el motivo. La incidencia no se ve afectada.
Postcondición	Queda el asiento en bitácora. La guía no se persiste.

Por qué se suma, y por qué recién ahora. El sistema ya sabía *quién* tiene que atender la incidencia y *de qué* se trata, pero no aportaba nada sobre *cómo* resolverla. Para el Segmento A —PyMEs donde el soporte lo hace una persona sola, muchas veces sin formación específica— ése es justamente el momento en que el producto puede aportar más. Y encaja con el motor predictivo: las mismas fallas resueltas que alimentan las reglas son las que hacen útil la guía.

Tres decisiones de diseño que hay que poder defender:

1. **Los pasos van de menos a más invasivos, y eso se prueba.** Un procedimiento que arranca vaciando la cola de impresión o apagando el equipo es peor que no tener procedimiento. La regla está en el prompt del proveedor real y en las recetas del simulado, y hay una prueba que recorre las seis categorías verificando que ningún paso sin riesgo quede después de uno con riesgo.
2. **El sistema no toca el equipo del cliente.** La guía la ejecuta una persona. Está dicho en el prompt de forma explícita para que el modelo no proponga acciones remotas, y es lo que mantiene el alcance del producto donde estaba: PredictIT gestiona y predice, no administra máquinas. Sumar ejecución remota sería otro producto, con otra responsabilidad legal.
3. **No se persiste.** La guía depende del historial del equipo en el momento en que se pide; guardarla invitaría a leer mañana una armada con otro historial. Lo que queda es el asiento en bitácora, que además alimenta el costo por consulta del modelo financiero.

Lo que esto cambia en el resto del trabajo:

Dónde	Qué cambia
Alcance de la IA (1.7, 2.2.1.1, 6.4.1)	De dos funciones a tres
Requisitos (10.2.2)	RF-15 nuevo
Casos de uso (10.2.3, 10.4.2)	CU-018 nuevo, con su diagrama de secuencia
Pantallas (10.5)	El detalle de incidencia suma el panel de la guía
Casos de prueba (10.6)	CP-15: la guía con el proveedor caído devuelve el motivo y no rompe la pantalla
Costo del servicio de IA	Es una consulta más por incidencia sólo cuando se pide , no en cada alta. El presupuesto lo toma de la bitácora, que ya distingue los tres tipos de evento
Propuesta de valor (capítulo 6)	Es el argumento más fuerte para el Segmento A: el sistema no sólo avisa que algo va a fallar, ayuda a arreglarlo

Lo que está hecho: el contrato IProveedorIA con la tercera operación, las

dos implementaciones —la real contra la API de Anthropic y la simulada con recetas por categoría—, el disyuntor cubriendo la operación nueva, el método de negocio con su control de acceso, el endpoint, los dos tipos de evento en el seed y diez pruebas.

Lo que falta: el panel en la pantalla de detalle de incidencia, el CP-15

automatizado, y bajar todo esto al .docx.

D-37 · Las dos metas que no coincidían con el modelo, resueltas

Estado: aplicado en el modelo y en el plan de la Entrega 1 · la meta de equipos **ya estaba corregida en el .docx**

Secciones: 5.3.3 (metas cuantitativas al mes 12) · 1.2 del plan de negocio (participación de mercado)

Eran las dos decisiones de negocio que el plan venía declarando sin resolver. Se resolvieron el 09/09.

La meta de equipos: el Excel estaba mal, no el documento

El plan de la Entrega 1 contrastaba los 480 equipos que proyecta el modelo al mes 12 contra una meta de **1.000**, y reportaba un desvío del -52 %.

Al ir a corregirlo apareció que **el documento ya no dice 1.000**. Dice:

*«Alcanzar 25 clientes pagos activos y **450 equipos** informáticos administrados al mes 12 [...] el Segmento A se define como PyMEs de 5 a 50 empleados, cuyo parque informático es del orden de 18 a 24 equipos, de modo que 25 clientes administran alrededor de 450. Los 1.000 equipos quedan como objetivo del segundo año de operación, cuando la mezcla incorpore clientes del Segmento B.»*

O sea que la corrección estaba hecha en el documento y el modelo seguía midiendo contra la meta vieja. Es exactamente el caso que la cátedra advierte: si el Excel y el PDF dicen distinto, uno de los dos está mal, y acá el que estaba mal era el Excel. Con la meta en 450, el modelo da **480: un 6,7 % por encima**, y el control pasa de ser el peor desvío del libro a cerrar.

Vale la pena dejar escrito por qué 1.000 era insostenible, porque es lo que se pregunta: 1.000 equipos entre 25 clientes son **40 equipos por cliente**, y con 40 equipos cada cliente pagaría 25 adicionales a \$ 4.200 —\$ 105.000— sobre un abono de \$ 89.000. El plan quedaba vendiendo adicionales y llamándolo suscripción.

La participación de mercado: punto intermedio

El plan declaraba **1,1 %** al año 4 y el modelo proyectaba **1,61 %**, un 46 % por encima. Pasa a **1,35 %**, el punto medio.

El motivo de tomar el medio y no una de las dos puntas es que las dos estaban movidas. El 1,1 % se escribió antes de tener el embudo armado y quedó corto; el 1,61 % supone que el embudo sostiene su tasa de conversión durante cuatro años seguidos, y eso es optimista. Mover una sola para que coincidiera con la otra habría sido acomodar el número.

Queda un desvío del 19 % entre objetivo y proyección, declarado. **El contraste lo calcula la hoja Proyección de ventas**** y no un párrafo escrito a mano, así que si mañana cambia una hipótesis, el desvío se mueve solo.

D-38 · El embudo se vuelve autogestionado y el universo se deriva

Estado: aplicado — la tabla del embudo está en 5.3.5, la derivación del universo —de las 549.100 PyMEs empleadoras a las 65.965 del AMBA y a las 21.109 del mercado meta— en 4.2 con sus dos supuestos propios declarados, y en 5.3.2 las metas expresadas sobre ese denominador: 0,12 % al mes 12, 0,64 % en el equilibrio y 1,40 % al año 4

Secciones: 4.2 y 5.3.2 (dimensionamiento del mercado) · 5.3.5 (embudo comercial) · 5.3.3 (metas al mes 12)

Son las dos decisiones de negocio que quedaban abiertas del punto 1 del plan, y se tomaron el 09/09.

El embudo: sale la demostración en vivo

Dice hoy (5.3.5): cuatro etapas —visitas a leads 3-5 % a demostraciones

calificadas 35-45 % a pruebas gratuitas 55-65 % a cliente pago 25-30 %—.

Tiene que decir: dos etapas. **Visitas a prueba gratuita iniciada, 2,0 %** y

prueba gratuita a cliente pago, 12 %. La prueba se inicia desde el sitio, sin intervención humana.

Por qué. El embudo anterior tenía una conversión total del 0,26 % y el nuevo tiene 0,24 %: la diferencia no está en la tasa, está en quién hace el trabajo. Para sostener las altas del cuarto año, el embudo de cuatro etapas pedía **108 demostraciones por mes**, cinco por día hábil, y el único rol comercial de la plantilla es un socio que además gestiona el canal de partners y el ciclo de venta del Segmento B. Era un embudo que el equipo no podía operar.

El argumento de fondo es de economía unitaria y conviene tenerlo a mano en la defensa: un producto de unos USD 75 mensuales no se vende con demostraciones individuales, porque una hora de una persona por prospecto no entra en un costo de adquisición que tiene que recuperarse en meses. Se vende dejando que el cliente lo pruebe.

Lo que cuesta: sin nadie que califique al que se registra, la prueba

convierte al 12 % y no al 27,5 %. El modelo pasa de 370 a **350 clientes** al año 4 y el resultado del año 4 de +205,0 a **+169,6 millones**. **Lo que se gana:** el volumen deja de tener techo. 135 pruebas por mes no consumen horas de nadie; 108 demostraciones sí. La carga se mueve al onboarding, que ya está modelado como costo variable por cliente.

El universo de PyMEs: se deriva de datos públicos

Dice hoy: 72.000 PyMEs de 5 a 50 empleados en AMBA, sin fuente.

Tiene que decir: 65.965, derivado de los datos de SEPyme / CEP XXI de

2023 —549.100 PyMEs empleadoras, 12 % de 10 a 49 empleados, 85 % de 1 a 9, con 17,2 % en CABA y 32,3 % en la Provincia— más dos supuestos propios declarados: que el **20 %** de las microempresas tiene entre 5 y 9 empleados, y que el **75 %** de la Provincia cae dentro del AMBA.

Por qué importa. Era la única fila del plan sin fuente, y de ella cuelga toda la sección de mercado: si el denominador no se puede sostener, la participación tampoco. Las estadísticas oficiales cortan los tramos en 1–9 y 10–49, y el segmento objetivo arranca en 5, así que el recorte exige un supuesto propio; lo mismo con el AMBA, que la estadística no distingue de la Provincia entera.

Con esos dos supuestos el resultado se mueve entre 63.000 y 73.000. **Se toma el extremo conservador y no el medio:** bajar el universo hace más exigente el objetivo de participación, no menos. Los 72.000 anteriores eran el techo del rango.

Con el universo en 65.965, el mercado meta alcanzable pasa de 23.040 a **21.109**, y la participación objetivo se fija en **1,4 %** —el punto intermedio entre el 1,1 % que declaraba el plan y el 1,66 % que proyecta el modelo—.

D-39 · D-32 decía «aplicado» y solo cuatro diagramas lo estaban

Estado: aplicado — los veintidós salen de Enterprise Architect, y también los dieciocho diagramas de caso de uso de 10.4.3.

El diagnóstico de esta entrada estaba equivocado, y el equivocado era mío. Decía que las secuencias de EA tenían proporción 0,9 contra 2,5 las del documento, así que había que insertarlas a mano redimensionando una por una. Lo que pasaba es que los PNG en disco eran de una corrida vieja: al volver a construir el modelo, las trece salen con $2,55 \cdot 2,41 \cdot 2,22 \cdot 2,46$ y así, coincidiendo una por una con las del documento. No había nada que hacer a mano.

Al mirarlo de cerca apareció algo peor y que ninguna medición mostraba: las secciones de CU-013 a CU-017 mostraban las cinco **la misma imagen**. Las secciones se habían clonado de la de CU-013 y la referencia a la imagen se clonó con ellas, así que apuntaban al mismo archivo dentro del .docx —escribir en él cambiaba las cinco a la vez—. Ahora hay dieciocho diagramas de caso de uso, uno por sección, generados desde el modelo.

Y 10.4.7 pasa a ser tres diagramas en lugar de uno. El modelo exportaba las cinco capas una al lado de la otra, 1.436 px de ancho, que en una hoja de 15 cm deja la letra en menos de 3 pt. El corte es por dependencia y no por capa suelta —Api sobre BLL, Service como transversal, DAO sobre Domain—, así cada uno tiene las dos puntas de una relación y se entiende solo.

Secciones: 10.4.4 (secuencias) · 10.4.7 (clases de toda la solución)

Cómo apareció. Al ir a reemplazar los diagramas se comparó el md5 de cada imagen del .docx contra los PNG que exporta el modelo. De las 59 imágenes del documento, sólo **cuatro** eran exactamente un archivo de EA: clases de bitácora, clases de permisos, componentes y despliegue. D-32 figuraba como aplicado desde que se generó el modelo, pero generar los diagramas y ponerlos en el documento son dos cosas distintas y sólo la primera había pasado.

Qué se hizo el 09/09. Se reemplazaron cuatro más —el de paquetes y los tres entidad-relación—, con lo que ya son ocho. El mapeo de los tres DER se resolvió comparando proporciones y no por el número de archivo: 0,82 contra 0,83 para equipos, 0,70 contra 0,71 para incidencias y 0,92 contra 0,93 para servicio. Esa verificación cumple dos funciones a la vez, porque el tamaño en página lo fija el XML del documento y no el archivo: si las proporciones no coincidieran, la imagen nueva entraría deformada.

Lo que falta y por qué no se hizo por bytes. Las trece secuencias del documento tienen proporción 2,5 y las de Enterprise Architect 0,9. Reemplazarlas sustituyendo el archivo las aplastaría a un tercio de su alto. Hay que insertarlas a mano en Word, redimensionando cada una. Lo mismo con 10.4.7, que tiene tres imágenes y el modelo exporta un solo diagrama de clases de la solución: hay que decidir si se parte en tres o si la sección pasa a tener una.

D-40 · Fuera los emojis y las flechas tipográficas

Estado: aplicado

Secciones: 1.7 y 6.4.1 (embudo comercial) · anexo de gestión del proyecto · código fuente entregado

Por qué. Un emoji y una flecha tipográfica no se escriben con el teclado y dependen de una fuente que la máquina del lector puede no tener: en un PDF sin esa fuente salen como un cuadrado, y en una consola de Windows con página de códigos 1252 directamente rompen el proceso. Este proyecto ya se cayó una vez imprimiendo un signo menos tipográfico, así que no es una hipótesis.

Qué se cambió. En el código, las cuatro flechas que quedaban —la del comentario de FactoryDao, las dos de los mensajes de cambio de estado de equipo e incidencia, y la de la recomendación en la pantalla Predictivo— pasan a `->`. Las marcas de avance del ROADMAP y de los casos de prueba pasan de emoji a las marcas de las listas de tareas de Markdown: `[x]`, `[~]` y `[ ]`, más OK y FALLA para el resultado de cada prueba. En el documento se reescribieron las seis frases del embudo y del cuadro de metas que usaban la flecha como conector.

Lo que se aprovechó. Tres de esas seis frases nombraban las etapas del embudo con palabras distintas a las de la hoja de cálculo —«Visitantes web», «Cliente pago»— y ahora dicen exactamente lo que dice la fila del Excel: «Visitas a prueba gratuita iniciada» y «Prueba gratuita a cliente pago». Que el documento y el modelo llamen distinto a la misma fila es justo lo que el control de alineación busca.

Verificación. El barrido sobre los quince `.docx` y los dieciséis PDF del paquete no encuentra ningún emoji ni ninguna flecha. Sobre los 164 archivos de código del ZIP de la entrega queda **uno solo**, y a propósito: el literal de `CifradoTests.cs`, que es el dato con el que se prueba que el cifrado AES-256-GCM soporte acentos, emojis y separadores. Sacarlo debilitaría la prueba.

Lo que además quedó al día. `PATRONES-Y-PRINCIPIOS.md` traía la verificación del 02/09 y afirmaba que no había integración continua, que Docker cubría sólo la base de datos y que la suite tenía 199 pruebas. Las tres cosas dejaron de ser ciertas: hay cinco trabajos de CI sobre cada push, `docker compose up --build` levanta las cuatro piezas, y la suite está en 281 pruebas en verde —250 del backend y 31 del frontend—. Una tabla de estado desactualizada es peor que no tenerla, porque en la defensa la contradice el propio repositorio.

D-41 · La sección financiera se contradecía a sí misma

Estado: aplicado — las tablas 9, 10 y 11 salen del presupuesto y la prosa de alrededor dice los mismos números (09/09)

Sección del documento: 7.7 (presupuesto y punto de equilibrio)

Decía hoy: las tres tablas traían la estructura de costos anterior a D-33

—ARS 45,9 millones de recursos humanos y ARS 53,6 de egresos en el primer año, y un salto de personal «por más de cinco» entre E2 y E3— mientras los párrafos de la misma sección ya citaban el punto de equilibrio en 135 clientes, el capital de ARS 156,2 millones y el valor actual neto de 184,7. Adentro de una sección, las tablas decían una cosa y los párrafos de al lado otra.

Tiene que decir: los números del modelo. Las tres tablas se rehicieron

promediando los meses de cada etapa del presupuesto —E0 son los meses 1 y 2, E1 del 3 al 5, E2 del 6 al 8 y E3 del 9 al 12—, y suman ARS 32,4 millones de egresos y ARS 10,4 de ingresos en el año 1, con un resultado operativo de -22,0. Es lo mismo que informa la Entrega 1 de negocio.

Por qué: cuando el modelo se recalibró (D-33) se actualizó la prosa de 7.7 y

no las tablas. Nadie lo vio porque las dos partes son coherentes por separado: hay que sumar los doce meses de la tabla y compararlos con el libro para que aparezca. docs/negocio/README.md lo tenía anotado —«la reconciliación ya no cierra... los dos números hay que rehacerlos en el documento»— y esa nota quedó pendiente tres iteraciones.

Lo que además se declaró. El documento fija 25 clientes pagos al mes 12 y el modelo proyecta 22,8, un 8,9 % por debajo. La Entrega 1 lo dice con su motivo —haber sacado la demostración comercial del embudo baja la conversión— y el .docx no lo decía. Ahora la meta se sostiene como objetivo y la diferencia queda escrita, que es distinto de presentar la proyección como si coincidiera.

D-42 · Lo que encontró la revisión de coherencia

Estado: aplicado — los seis arreglos están en el .docx y los dos controles nuevos pasan (09/09)

Sección del documento: 6.6.2 · 7.7 · 10.4.10 · capítulos 4 a 7 (referencias

cruzadas) · documento completo (índice)

Qué se encontró, leyendo el paquete completo con el criterio de alguien de afuera:

Hallazgo	Dónde	Qué se hizo
El apartado 7.7.1 estaba insertado justo después del título de 7.7, así que el presupuesto, los ingresos, el resultado y el punto de equilibrio colgaban de un título que habla del valor terminal	7.7	El apartado pasa al final de la sección, que es además donde corresponde: es la conclusión de la evaluación
El CP-15 no tenía título y su tabla estaba rotulada «CP-14 (continuación)»	10.4.10	Se agrega 10.4.10.15 y el epígrafe dice lo que la tabla contiene
Nueve referencias cruzadas apuntaban a apartados que no existen —5.3.2, 5.3.3, 5.3.5, 5.3.6— desde que el capítulo 5 se reenumeró	4.2, 6.1, 6.5.2, 6.5.3, 7, 7.1, 7.7	Cada una apunta a su destino real; la de 4.2 iba a 6.2 y no al capítulo 5
Seis títulos vacíos, que salen en el índice como renglones en blanco	1.2, 10.3.2, 10.4.4, 10.4.9, 10.5.1	Se sacan
El título de 6.6.2 traía basura de codificación: los apóstrofes de «7 C's» eran diéresis sueltas	6.6.2	«7 C»
Los manuales decían que esperaras 131 pruebas del backend y 24 del frontend, y salen 257 y 34; el documento de patrones declaraba 253; la revisión OWASP, 197	Manuales, patrones, OWASP	Al día, y con un control que lo compara contra la corrida
La insignia de pruebas de la hoja de ruta decía «175 + 24» desde que se escribió	ROADMAP	Se cuenta: el backend sale del documento que genera la corrida y el frontend de los archivos de prueba
El plan de sprints describía ocho sprints y el backlog tiene diez; el décimo figuraba como colchón «sin alcance asignado» y tiene diecisiete historias terminadas	Plan de sprints	Se agrega S10 con lo que efectivamente absorbió, y la nota de que el trabajo se adelantó al calendario
El README declaraba 11 ADR con 14, 19 deltas con 43, 16 prompts críticos con 18 y 53 historias con 88	README	Al día, y con el control 17 que cuenta las cuatro cosas de su fuente
El guion de la defensa contestaba «el VAN da -48 millones» y «los usuarios concurrentes no están medidos», las dos falsas; la presentación enumeraba cuatro pendientes de los cuales tres estaban resueltos	Guion y presentación	Al día
El panel de obra mostraba 71 de 76 historias, 227 pruebas y el diagrama de sprints con S6 a S9 sin empezar	Panel de obra	docs/panel/_actualizar-cifras.py cuenta cada cifra de su fuente y redibuja las barras

Los dos controles que deja. El 15 verifica que toda referencia a un apartado numerado apunte a un título que existe: una mención rota no falla, manda al lector a buscar algo que no está, y en una defensa eso es una pregunta. El 16 compara la cantidad de pruebas que declara cada documento contra el total de la corrida. Los dos se probaron en rojo antes de darlos por buenos.

D-43 · El multiidioma no cumplía lo que el requisito promete

Estado: aplicado — el tablero manda claves, el diccionario tiene las 739, y la medición sobre el sistema corriendo da cero (09/09)

Sección del documento: 10.2.5 (RNF de internacionalización) · 10.3.2.1

(Multiidioma)

Dice hoy: «la incorporación de un nuevo idioma no requiere modificaciones en la lógica de la aplicación, sino únicamente la traducción del archivo correspondiente».

Tiene que decir: lo mismo. Lo que cambió es el sistema, que no lo cumplía.

Por qué: con la interfaz en castellano todo se leía bien, y ese es el punto.

Puesta en inglés quedaban en castellano los cuatro indicadores del tablero —que PrediccionBusiness armaba como frases, de modo que sumar un idioma pedía editar una clase de la capa de negocio—, unos noventa rótulos del frontend y treinta y cuatro claves que el código pedía y nadie había cargado en el diccionario. El control que vigilaba esto informaba cero: su heurística exigía un acento, una eñe o un artículo, y «Estado operativo», «Fallos seguidos» o «Configurar reglas» no tienen ninguno.

Cómo quedó. Los indicadores viajan como clave con los números aparte —en inglés el número va delante del sustantivo y en castellano detrás, así que concatenar no alcanza—. La detección mira además los rótulos sin acento, las props propias que llevan texto visible (título, contexto, etiqueta, detalle) y ya no descarta un párrafo por tener un punto y coma. Y hay tres redes nuevas: una prueba de backend que exige que el tablero mande claves, el control 15 de alineación que compara cada clave pedida contra el seed, y `npm run idioma`, que abre el sistema en los dos idiomas y compara pantalla contra pantalla descartando lo que mandó la API. Ese último es el que encuentra lo que la heurística no ve, porque mide en lugar de deducir. Da cero.

D-44 · Lo que quedaba abierto de OWASP, y la capa de negocio

Estado: aplicado fuera del .docx — vive en «Verificación del sistema» y en el código; el documento principal no describe los parámetros de derivación ni la configuración del proxy, y no tiene por qué (11/09)

Sección del documento: ninguna. Se anota igual porque cambia lo que el paquete afirma sobre sí mismo, y eso es lo que este registro sigue.

Qué decía la revisión OWASP: que A02 quedaba con dos reparos —«falta TLS

propio» e «iteraciones por debajo de OWASP»— y que A08 estaba «sin cubrir en la cadena de build».

Qué dice ahora:

Iteraciones. PBKDF2 pasa de 120.000 a **600.000**, la recomendación de la *Password Storage Cheat Sheet*. Subirlo solo no alcanzaba: el número viaja adentro del hash y cada uno se verifica con el suyo, así que los usuarios ya creados se quedaban con el hash débil para siempre. El sistema **vuelve a derivar el hash cuando la persona entra**, que es el único momento en que la contraseña en claro existe, y los datos iniciales ya se generan con 600.000.

TLS. Deja de ser una ausencia y pasa a ser una configuración que no está activa. frontend/nginx-tls.conf trae el servidor de 443 con TLS 1.2 y 1.3, la lista de cifrados, la redirección desde el 80 y HSTS a un año. No se activa en la imagen porque en localhost el único certificado posible es autofirmado, y eso agrega una advertencia del navegador en medio de la defensa sin proteger nada.

Dependencias. El árbol completo queda congelado con su hash y la integración continua restaura en modo bloqueado, igual que la imagen de la API. Actualizar una dependencia pasa a ser un cambio visible en el repositorio.

Y la capa de negocio. Era la de menor cobertura de las cinco y sube de 23,7 % a 29,1 %, con el total del proyecto de 49,4 % a 51,4 %. Son veintidós pruebas sobre reglas con un motivo que no se ve en el código —la garantía anterior a la compra, la baja por la puerta del cambio de estado— y no sobre lo más fácil de cubrir. Apareció además una prueba intermitente que ya estaba: dos clases movían el contador de intentos fallidos del mismo usuario en paralelo.

Lo que sigue abierto y declarado: A07, el segundo factor, que es consecuencia del ADR 0014; y la firma de artefactos de A08.

D-45 · El mantenimiento deja de ser sólo un registro: entra el plan y la agenda

Estado: aplicado — el documento tiene el RF-16 en la tabla de requerimientos, los CU-019 y CU-020 en 10.4.3 con sus flujos, el CP-16 en 10.4.10.16, la pantalla de la agenda en 10.5.1.5 y la de planes en 10.5.1.10.3, y el DER de equipos ya trae PlanMantenimiento y MantenimientoProgramado. Falta sólo el diagrama de secuencia de los dos casos de uso, que se dibuja en Enterprise Architect y está anotado en PENDIENTE.md

Secciones: 10.2.2 (RF-16 nuevo) · 10.2.3 (CU-019 y CU-020 nuevos) · 10.4.7 (DER de negocio: dos entidades) · 10.5 (dos pantallas nuevas) · 10.6 (casos de prueba)

Dice hoy: el mantenimiento es un registro de lo que ya se hizo. RF-07

pide registrar mantenimientos preventivos y correctivos con su descripción, repuestos y costo, y la regla predictiva «Mantenimiento vencido» mide contra un umbral escrito en la propia regla —«hace más de 180 días del último preventivo»—.

Tiene que decir: el sistema además **planifica**. Una organización define

cada cuánto le toca el preventivo a un equipo o a todo un tipo de equipo, de ahí salen fechas concretas, y «vencido» pasa a tener una definición exacta: el trabajo agendado cuya fecha ya pasó.

RF-16 · Planificación del mantenimiento preventivo

El sistema debe permitir definir planes de mantenimiento preventivo, cada uno con su alcance —un equipo puntual o todos los equipos de un tipo—, el tipo de trabajo y cada cuántos días corresponde. A partir de los planes activos, el sistema genera las fechas concretas de mantenimiento para los equipos operativos alcanzados. Cada fecha agendada puede reprogramarse o anularse indicando el motivo, y se cierra sola cuando se registra el mantenimiento que la ejecuta.

CU-019 · Definir un plan de mantenimiento

Actor	Administrador
Precondición	Existe al menos un tipo de mantenimiento preventivo

Disparador	El administrador carga o edita un plan en Configuración › Planes de mantenimiento
Flujo principal	1. Elige el alcance: un equipo o un tipo de equipo. 2. Elige el tipo de trabajo, que tiene que ser preventivo. 3. Indica cada cuántos días (entre 7 y 3650). 4. El sistema valida y guarda, y muestra a cuántos equipos operativos alcanza el plan.
Flujo alternativo	Si el alcance viene vacío o con las dos opciones, se rechaza explicando por qué: un plan sin alcance no se puede aplicar y uno con los dos no dice cuál manda.
Postcondición	El plan queda activo y cuenta para la generación de la agenda.

CU-020 · Gestionar la agenda de mantenimiento

Actor	Administrador
Precondición	Hay al menos un plan activo, o se programa una fecha suelta
Disparador	El administrador entra a la Agenda de mantenimiento
Flujo principal	1. Ve lo vencido y lo que viene, con los días de atraso calculados en el servidor. 2. Genera las fechas que faltan a partir de los planes activos. 3. Sobre una fecha: registra el trabajo, la reprograma o la anula.
Flujo alternativo	Generar dos veces no duplica: informa cuántas fechas ya estaban. Reprogramar y anular exigen motivo.
Postcondición	Cada operación queda asentada en bitácora con su tipo de evento propio.

Por qué se suma. El motor predictivo ya recomendaba «Programar el mantenimiento preventivo» y el sistema no tenía con qué programarlo: la recomendación terminaba en un papel o en la memoria de alguien. El bucle quedaba abierto, y cerrarlo es lo que convierte al sistema en una herramienta de gestión y no sólo en un registro.

Tres decisiones que hay que poder defender:

1. **Con plan, el plan manda sobre el umbral.** Un equipo cuyo próximo preventivo está agendado para la semana que viene no es un equipo descuidado, aunque hayan pasado 250 días del anterior. Antes la regla decía que sí. El umbral no se va: es lo que sigue funcionando para los equipos sin plan, y además es lo que satura el atraso —un mes de atraso sobre un plan mensual es un ciclo entero perdido; sobre uno anual, poco—.
2. **«Vencido» no es un estado guardado.** Es PROGRAMADO con la fecha pasada. Guardarlo obligaría a un proceso que recorra la tabla todos los días, y el día que ese proceso no corra el sistema mostraría datos viejos como si fueran ciertos.

3. **La agenda no cierra nada por su cuenta.** Lo que cierra un trabajo agendado es haberlo hecho, y eso son los datos del mantenimiento —qué se hizo, repuestos, costo—. El botón «Registrar» lleva el identificador del trabajo hasta el servidor y no sólo el equipo y el tipo: dos preventivos iguales sobre el mismo equipo, uno atrasado y otro del mes que viene, sólo se distinguen así.

Entidades nuevas en el DER de negocio: PlanMantenimiento (alcance

excluyente por CHECK, cada_dias entre 7 y 3650) y MantenimientoProgramado (con su id_plan nulo cuando se agendó a mano, y su id_mantenimiento cuando se ejecutó).

Patente nueva: MANTENIMIENTO_PLANIFICAR, aparte de

MANTENIMIENTO_REGISTRAR. Registrar es contar lo que se hizo; planificar es comprometer a la organización a hacerlo. El técnico hace lo primero; lo segundo lo decide quien responde por el parque.

D-46 · La facturación del servicio: el modelo de ingresos entra al sistema

Estado: aplicado — el documento tiene el RF-17, el CU-021 en 10.4.3.21, el CP-17 en 10.4.10.17, la pantalla en 10.5.1.10.4 y un DER propio de la facturación como cuarta figura de 10.4.8. Falta sólo el diagrama de secuencia del caso de uso, anotado en PENDIENTE.md

Secciones: 10.2.2 (RF-17 nuevo) · 10.2.3 (CU-021 nuevo) · 10.4.7 (DER de negocio: dos entidades) · 10.5 (pantalla nueva) · 10.6 (CP-16) · y el puente con los capítulos 5 a 7, que definen los planes comerciales

Dice hoy: el capítulo 10 no menciona la facturación. Los planes comerciales

—Inicial, Estándar y Partner, con su abono, sus equipos incluidos y su precio por equipo adicional— viven en los capítulos de negocio y en la pantalla de Organización, que muestra cuánto sale el mes.

Tiene que decir: el sistema **factura su propio servicio**. Cada mes cerrado

genera un comprobante por organización con el desglose del plan, y lleva su estado de cobro.

RF-17 · Facturación del servicio

El sistema debe poder generar, por organización y período mensual, un comprobante del servicio prestado, con el abono del plan contratado y los

equipos administrados por encima de los incluidos, medidos al cierre del período. El comprobante admite ajustes mientras está en borrador, se emite con número correlativo y vencimiento, registra su pago, y se anula con motivo.

CU-021 · Facturar el período

Actor	Administrador
Precondición	La organización tiene un plan comercial asignado
Disparador	El administrador factura el último mes cerrado
Flujo principal	1. El sistema cuenta los equipos administrados al cierre del período. 2. Arma el borrador con el abono y, si los hay, los equipos adicionales. 3. Calcula IVA y total. 4. El administrador emite: el comprobante recibe número, fecha y vencimiento. 5. Cuando se cobra, se registra el pago.
Flujo alternativo	Si el período ya está facturado, se devuelve el comprobante que hay sin tocarlo. Un comprobante emitido no se modifica: se anula con motivo y se genera otro. Uno cobrado no se anula: se corrige con un ajuste en el período siguiente.
Postcondición	Cada paso queda asentado en bitácora con su tipo de evento propio.

Por qué se suma. El plan de negocio define un modelo de ingresos por abono más equipos administrados, y el sistema lo calculaba pero no lo registraba: no había forma de saber qué se facturó, cuándo vence ni qué está impago. Es el puente que faltaba entre los capítulos 5 a 7 y el capítulo 10, y en la defensa es la pregunta natural: «el modelo dice que cobrás por equipo administrado, ¿dónde está eso en el sistema?».

Tres decisiones que hay que poder defender:

1. **El corte entre borrador y emitido.** Hasta la emisión es un cálculo que se rehace; después es lo que se le reclama a la organización, con número y fecha. Ese corte es lo que hace auditable la facturación, y por eso emitir no se deshace: se anula con motivo.
2. **Los números del plan quedan copiados en el comprobante.** Si se leyera del plan al consultar, cambiar el precio reescribiría los comprobantes del año pasado. No es un detalle contable: es el registro de lo que la organización efectivamente pagó.
3. **No pretende ser un comprobante fiscal.** No hay CAE, ni punto de venta, ni integración con AFIP, y por eso la entidad se llama Comprobante y no Factura. Es el registro interno del servicio. Integrarlo con facturación electrónica es trabajo futuro y está anotado como tal.

Entidades nuevas en el DER de negocio: Comprobante (único por organización)

y período, con el correlativo garantizado por un índice único filtrado) y LineaComprobante (el desglose, que se borra con su comprobante).

Patentes nuevas: FACTURACION_VER y FACTURACION_GESTIONAR. Quien

administra la organización tiene que poder consultar qué le facturaron y desde cuándo debe, y eso no lo habilita a emitir ni a dar algo por cobrado.

Un hallazgo del camino, que vale para la defensa. La unicidad del correlativo estaba escrita como UNIQUE (id_organizacion, numero). SQL Server trata el NULL como un valor más en una restricción de unicidad, así que la organización admitía **un solo borrador** y el segundo fallaba con un error del motor. Se reprodujo contra el sistema corriendo y se corrigió con un índice único filtrado por numero IS NOT NULL. Queda una prueba de integración que lo cubre, porque la restricción vive en la base y ningún doble la reproduce.

D-47 · La fecha de la Entrega 2 de negocio, corregida en el sentido contrario

Estado: aplicado — el documento, el presupuesto, el plan de sprints, el backlog y el control de alineación dicen ahora **23 de octubre**.

Qué pasó. D-33 había corregido esta misma fecha el 08/09, con la nota de que la cátedra alterna el orden de los grupos entre entregas y que la Entrega 2 la abre el Grupo 2 —el autor— el 16 de octubre. La lámina 5 de Clase 1 - Presupuesto, vista de nuevo el 12/09, muestra el orden invertido: abre el Grupo 1 el 16 de octubre, y el Grupo 2 entrega el **23**.

No se tocó la entrada de D-33. Este registro es cronológico: D-33 documenta lo que se sabía y se decidió el 08/09, con su motivo, y sigue siendo cierto que eso fue lo que pasó ese día. Lo que cambia es el dato de la cátedra, no la decisión de aquel momento.

Dónde se corrigió: docs/negocio/README.md (tabla y prosa),

docs/negocio/_entrega-1.py, docs/PENDIENTE.md, docs/gestion/PLAN-DE-SPRINTS.md, la historia H-53 del backlog, los dos LEEME de ENTREGABLES/ENTREGA NEGOCIO/ y la constante FECHAS de docs/_verificar-alineacion.py, que es la que la cátedra usa como fuente única para el control de fechas.

D-48 · D-47 estaba mal: la lámina se leyó cruzada, no la fecha

Estado: aplicado — vuelve a decir **16 de octubre** para el Grupo 2, que es lo mismo que decía antes de D-47.

Qué pasó. D-47 se apoyó en una foto del proyector de Clase 1, tomada en ángulo, para "corregir" la fecha en sentido contrario. Al cotejarla contra el PDF real de la clase —texto, no foto— la lámina 5 dice lo mismo que siempre: Entrega 2, Grupo 2 el 16 de octubre y Grupo 1 el 23. La foto se leyó con las dos filas de grupos cruzadas.

No se corrigió D-47 en su lugar. Por la misma razón que D-33 quedó escrito tal cual: este registro es cronológico, y D-47 documenta con exactitud qué se creyó y por qué durante esas horas del 12/09. Fue un error real, no una opinión, y decirlo así es preferible a que no quede rastro de él.

Dónde se revirtió: los mismos lugares que tocó D-47 —

docs/negocio/README.md, _entrega-1.py, docs/PENDIENTE.md, docs/gestion/PLAN-DE-SPRINTS.md, la historia H-53 del backlog, los dos LEEME de ENTREGABLES/ENTREGA NEGOCIO/ y FECHAS en docs/_verificar-alineacion.py—, ahora citando el PDF de la clase como fuente, no la foto.

D-49 · Distrito Tecnológico y derechos de exportación, declarados como no aplicables

Estado: aplicado — en 7.7, junto a los demás tributos que no alcanzan al proyecto.

Por qué se agrega. Clase 3 (Impuestos) nombra dos beneficios que el documento no mencionaba ni para aplicarlos ni para descartarlos: la exención de Ingresos Brutos y Sellos del Distrito Tecnológico (CABA/Córdoba) y el 0 % de derechos de exportación para servicios tecnológicos. Un beneficio no mencionado se lee como un olvido, no como una decisión.

Las dos decisiones:

1. **Distrito Tecnológico: no se fija.** El documento define la sede operativa en la Ciudad Autónoma de Buenos Aires o su área metropolitana, sin comprometer un barrio. Fijarla en el Distrito Tecnológico sumaría el beneficio, pero es una decisión de alcance que se dejó afuera a propósito para no atar el proyecto a una dirección que no se necesita definir todavía.
2. **Derechos de exportación: no aplican.** El proyecto vende únicamente en el mercado argentino; no exporta servicios, así que no hay base sobre la que el 0 % pueda aplicarse.

Dónde: el párrafo de 7.7 que ya declaraba qué tributos no alcanzan al

proyecto —y por qué— se extiende con estas dos, en docs/tfi/TFI-PredictIT.docx y su fuente en docs/tfi/_escenarios-y-tributos.py.

D-50 · Los tres diagramas de secuencia que faltaban, en 10.4.4

Estado: aplicado — 10.4.4 tiene ahora las secciones .19, .20 y .21, cada una con su figura.

Qué faltaba. Los CU-019, CU-020 y CU-021 entraron al documento con D-45 y D-46 con su ficha y sus dos flujos, pero sin diagrama de secuencia. El registro anterior decía que alcanzaba con sumarlos a `secuencias_por_caso` en `docs/tecnico/_diagramas-ea.py` y volver a correrlo. No alcanzaba: eso genera los PNG, pero nada los mete en el documento. Faltaban dos cosas más — las secciones, que no existían, y las entradas en el MAPA de `docs/tfi/_diagramas-al-documento.py`, que llegaba hasta 10.4.4.13. —.

La numeración salta de .13 a .19. No es un error de tipeo: en 10.4.4 el número de sección sigue al del caso de uso, igual que en las fichas de 10.4.3. CU-014 a CU-018 nunca tuvieron diagrama de secuencia, y dejar el hueco lo dice; renumerar de corrido habría puesto «10.4.4.14» sobre el CU-019 y roto la correspondencia en todo el resto de la sección.

De dónde salen los mensajes. De leer `PlanesMantenimientoController`, `MantenimientosProgramadosController` y `FacturacionController` con sus `business`, que es el criterio que ya estaba escrito en `_diagramas-ea.py`: un diagrama de secuencia que no coincide con el código se le cree igual, y ahí está el problema. El de CU-021 muestra el borrador y la emisión como dos llamadas separadas, porque así está implementado: entre una y otra se cargan los ajustes.

Lo que queda. Los tres son los que genera el script, no versiones rehechas a mano: EQUIVALENCIA en `_diagramas-del-modelo.py` cubre CU-001 a CU-013 y el modelo que se entrega no tiene los nuevos. Son correctos pero se ven distintos. Queda anotado en `PENDIENTE.md`.

Dónde: `docs/tfi/TFI-PredictIT.docx` (secciones 10.4.4.19 a .21),

`docs/tfi/_diagramas-al-documento.py` (el MAPA) y la función `secuencias_por_caso` de `docs/tecnico/_diagramas-ea.py`.

D-51 · El 18/09 era la entrega, no la devolución

Estado: aplicado — corregido en `docs/PENDIENTE.md` y en el LEEME de la Entrega 2 de negocio.

Qué pasó. `PENDIENTE.md` llamaba al 18 de septiembre «Devolución de la Entrega 1 de negocio», y el LEEME de la Entrega 2 decía que se arma «con la devolución de la primera entrega, que es

del 18 de septiembre». Las dos frases daban a entender que el 18 se **recibe** una corrección. Es al revés: el 18 de septiembre es cuando el **Grupo 2 entrega** la primera entrega de negocio.

Por qué no lo agarró ningún control. Las fechas sí estaban bien donde importa. La constante FECHAS de docs/_verificar-alineacion.py —que es la fuente única— dice ('1', '18 de septiembre', '11 de septiembre'), o sea Grupo 2 el 18 y Grupo 1 el 11, y las tablas de docs/negocio/README.md y del LEEME de la primera entrega la reproducen. El control 4 compara **las fechas**, no cómo se las nombra alrededor, así que el número correcto convivía con una etiqueta que decía otra cosa.

Es la misma clase de error que D-47, pero al revés: allá el dato estaba mal y el rótulo bien; acá el dato estaba bien y el rótulo mal. Y cuesta más de encontrar, porque no hay nada que contrastar: sólo se ve leyendo la frase entera.

Lo que confirma la corrección. La cátedra alterna el orden de los grupos entre entregas, que es la regla que D-33 ya había dejado escrita. Si en la Entrega 2 el Grupo 2 abre el 16 de octubre y el Grupo 1 cierra el 23, en la Entrega 1 el orden es el inverso: Grupo 1 el 11 de septiembre y Grupo 2 el 18. La fila de la tabla decía exactamente eso desde el principio.

Consecuencia práctica. La sección 1 de PENDIENTE.md estaba escrita sobre la lectura equivocada —«lo que espera la devolución del 18/09»— y describía como trabajo futuro algo que en realidad hay que entregar esta semana. Reescrita: la Entrega 1 está armada y verificada, y lo que espera la corrección es la Entrega 2 del 16/10.

D-52 · La reasignación manual no reasignaba, y el CP-08 no lo veía

Estado: aplicado — el endpoint quedó en su método, con tres pruebas de integración que lo cubren.

Qué pasó. En IncidenciasController, los atributos del CU-013 habían quedado huérfanos entre dos métodos:

```
/// <summary>Reasignación manual (CU-013).</summary>
[HttpPut("{id:guid}/tecnico")]
[RequierePatente(IncidenciaReasignarTodas, IncidenciaReasignarPropias)]
/// <summary>Qué transiciones admite la incidencia ahora.</summary> ← otro método
[HttpGet("{id:guid}/transiciones")]
[RequierePatente(IncidenciaVerTodas, IncidenciaVerPropias)]
public ActionResult<IReadOnlyList<TransicionDto>> Transiciones(Guid id) => ...
```

En C# los atributos se aplican al **siguiente miembro**, y el comentario del medio no los corta. Resultado: Transiciones terminó con dos rutas y dos exigencias de patente, y Reasignar — que estaba más abajo— quedó **sin ninguna ruta**, o sea fuera de la API.

Por qué es peor de lo que parece. No devolvía 404. El frontend llamaba a `PUT /api/incidencias/{id}/tecnico`, la petición caía en Transiciones, y volvía **200** con la lista de transiciones. La pantalla daba la reasignación por hecha y el técnico no cambiaba. Un 404 se ve; esto no.

Por qué las pruebas no lo agarraron. El CP-08 —«reasignación manual con control de permisos»— prueba la regla en la capa de negocio: `IncidenciaBusiness.Reasignar` y su verificación de patentes. Eso estaba y sigue estando bien. Lo que no había era una prueba que pasara por la API, y el defecto vivía exactamente ahí: entre el controlador y el negocio.

Cómo apareció. Al activar `GenerateDocumentationFile` para que Swagger mostrara las descripciones, el compilador avisó CS1587: el comentario XML no está situado en un elemento válido. Un aviso por un comentario mal ubicado terminó destapando un caso de uso caído. Es el argumento más concreto a favor de prender la documentación: no sirve sólo para documentar.

Lo que se agregó, además del arreglo: tres pruebas de integración en

`ReasignacionPorLaApiTests`. La primera comprueba que el PUT devuelva 204 y que el técnico efectivamente cambie —se verificó que falla contra el código roto antes de corregirlo—; la segunda, que `/transiciones` siga respondiendo en su propia ruta; la tercera, que el solicitante reciba 403, que es la exigencia que había quedado puesta sobre el endpoint equivocado.

D-53 · El RNF-05 pasa a ser verificable

Estado: aplicado fuera del .docx — vive en el ADR 0015 y en el código; el documento no cambia de texto, cambia lo que se puede afirmar de él.

Qué pasaba. El capítulo 10.2.2 compromete diez requisitos no funcionales. Tres de los cuatro medibles tienen su medición —el percentil 95 con `medir-carga.py`, la estabilidad bajo carga con el perfilado del ADR 0008, la degradación con CP-14 y CP-15—. El RNF-05, «disponibilidad del 95 % del horario laboral», no tenía ninguna: era el único requisito que el trabajo afirmaba y no podía comprobar.

Qué se hizo. El sistema expone sus métricas en formato Prometheus, en `GET /api/metricas`, detrás de un token. No se incorporan Prometheus ni Grafana: la aplicación expone y la infraestructura recolecta, que además es lo que corresponde al alcance, porque el despliegue en servidor está declarado afuera.

El tiempo en línea es lo que cierra el RNF-05. De paso quedan medibles de forma continua —y no puntual— la latencia por endpoint del RNF-04 y la memoria del RNF-09, y quedan

observables los dos mecanismos del ADR 0009 que se pueden consultar sin instrumentar el negocio: los disyuntores abiertos y lo que rechaza el limitador.

Lo que no resuelve, dicho para que no se lea como una promesa de más. Sin recolector no hay histórico ni alertas: el sistema puede decir «el disyuntor está abierto ahora», no «se abrió ocho veces el martes». Y las métricas vuelven a cero al reiniciar el contenedor. Es consecuencia de la decisión de no incorporar el stack, y está escrito en el ADR.

Dónde tocar el documento, si se decide. El 10.2.2 podría agregar, en la columna de verificación del RNF-05, que se mide con el tiempo en línea que expone `/api/metricas`. No se hizo todavía porque esa columna no existe: los RNF del documento no declaran cómo se verifica cada uno, y agregarla es un cambio de la tabla entera, no de una fila.

D-54 · El frontend se alinea al prototipo, y el prototipo se corrige

Estado: aplicado — los siete incrementos: capa de tokens, almacén de la aplicación, contadores de la navegación, tablas y formularios, el desglose del motor predictivo, los segmentos del inventario y el tablero de incidencias.

Qué pasaba. `docs/design/prototipo-completo.html` es la maqueta de la que salió el diseño, y la interfaz real se fue separando de ella: distinta tipografía, distinta escala tipográfica, distintos grises y distintos radios. No era una diferencia de criterio sino deriva: nadie decidió apartarse, se apartó solo.

Qué se hizo. La capa de tokens —`frontend/src/estilos/tokens.css`— se llevó a los valores del prototipo: 66 variables, entre la escala tipográfica, los anchos de las barras, los radios y las tres paletas. La tipografía pasa de Manrope a Plus Jakarta Sans, que es la del prototipo.

Lo que el prototipo tenía mal. Al pasar sus colores por `docs/design/_contraste.py` aparecieron tres pares por debajo del piso de la WCAG AA:

Par	Prototipo	Corregido
--línea-fuerte sobre --sup-2, tema oscuro	#526597 (2,45:1)	#56699e (3,01:1)
--línea-fuerte sobre --lienzo, tema claro	#8492b6 (2,82:1)	#7a89b0 (3,17:1)
--acento-hover sobre blanco, tema oscuro	#5480ff (3,59:1)	#3569ff (4,54:1)

Se movió sólo la luminosidad y se dejó el tono: la intención del prototipo es el color, lo que había que corregir era cuánto se separa del fondo. El RNF-07 —accesibilidad AA— manda sobre la maqueta, no al revés.

Lo que no se trae del prototipo. La maqueta tiene una barra superior para cambiar de rol, de tema y de densidad, que existe para poder mostrar las variantes en un HTML suelto. En el sistema el rol lo da la sesión y el tema y la densidad viven en Apariencia, así que esa barra no se implementa.

Incremento 2: el armazón. La barra lateral y el encabezado pasan a ser los del prototipo, y con eso se acomodan tres cosas que estaban repartidas:

- **La organización sube a la barra.** Era el primer renglón del encabezado de cada pantalla. No es contexto de la pantalla sino de toda la sesión, y para el perfil Partner —que salta entre clientes— tenerlo donde siempre está el menú es lo que evita el error caro. Debajo del nombre va el plan contratado.
- **El encabezado dice por dónde se llegó.** Donde estaba la organización ahora va el rastro, PREDICTIT / CONFIGURACIÓN, con la sección como enlace. Eso hizo desaparecer un enlace que la ficha del equipo y el detalle de la incidencia venían dibujando cada una por su cuenta, en el renglón de contexto, para poder volver al listado.
- **La barra sigue el tema.** Era una losa oscura fija en los dos temas, porque los tokens `--nav-*` tenían su propia paleta; ahora es la misma superficie que las tarjetas. El borde derecho, que se había sacado cuando la barra era oscura siempre, vuelve: en el tema claro sin borde no había límite entre la barra y el contenido.

Y dos cambios de pieza: el submenú de Configuración se hunde un escalón —fondo secundario, con el ítem activo en barra de color a la izquierda, que es lo que distingue un segundo nivel de navegación de una segunda barra del mismo rango—, y las tarjetas de indicador toman el rótulo en versalitas, la cifra en la tipografía monoespaciada y una barra de color a la izquierda en lugar del ícono teñido. La barra se lee de reojo, que es para lo que sirve un tablero.

Incremento 3: los contadores. Las insignias del prototipo son datos del servidor, así que traen endpoint propio: `GET /api/organizacion/resumen` devuelve equipos, incidencias abiertas y mantenimientos vencidos.

El endpoint no lleva patente, y esa es la decisión. Cada contador viaja en nulo si el usuario no puede verlo, y quién decide eso es la capa de negocio. Exigir una patente en el controlador hubiera dejado afuera al solicitante, que tiene su propio contador de pedidos. Y viajan en **nulo y no en cero**: cero es un dato —«no hay incidencias abiertas»— y la falta de permiso no lo es; con cero, el solicitante vería un «0» al lado de Activos afirmando que el parque está vacío.

`incidenciasAbiertas` es el mismo campo con dos significados según el perfil: las de toda la organización para quien las atiende, las propias para quien las reporta. Es la misma distinción que ya hacía el menú, y la prueba que la sostiene compara el contador contra el listado que **esa**

misma persona ve, no el de un usuario contra el de otro: así depende del recorte y no de cuántas incidencias tenga el seed en ese momento.

Dos cosas se apartan del prototipo, a propósito. La maqueta pone los vencidos al lado de «Mantenimientos»; acá van al lado de «Agenda», que es la pantalla donde efectivamente están. Y la tarjeta de la organización muestra los equipos administrados contra los incluidos — «52/15 eq.»—, en ámbar cuando se pasa: es el número que explica el excedente que se factura, y tenerlo ahí evita la sorpresa de fin de mes.

Se vuelve a pedir en cada cambio de pantalla y no una sola vez al entrar: una insignia dice cuánto hay ahora, y un número congelado en el del inicio de sesión es peor que no tener número, porque se lo cree. Si la consulta falla no se muestra nada ni se avisa: son insignias, no contenido.

Incremento 4: las tablas y los formularios.

- **La tipografía monoespaciada era la equivocada.** El `index.html` descargaba IBM Plex Mono —la del prototipo— y el token `--fuente-mono` apuntaba a Cascadia Mono. La fuente se bajaba en cada carga y el navegador dibujaba con otra: todos los códigos, todas las cifras y todos los identificadores del sistema salían en una tipografía que nadie había elegido.
- **El buscador y su tabla pasan a ser un solo objeto.** Eran dos tarjetas apiladas con un hueco en el medio. Un filtro que flota aparte se lee como un panel más, y lo que hace es gobernar lo de abajo.
- **El contenido de la tabla va al contraste pleno.** Estaba en el color secundario, que es el de lo que acompaña: una tabla entera en gris hace que todo parezca una nota al pie. Y el código de cada fila —el identificador— va en negrita y en el color del acento, así la vista salta de identificador en identificador sin leer el resto.
- **Los campos se hunden un tono.** Con el mismo fondo que la tarjeta, un campo vacío es un rectángulo con borde y hay que mirarlo dos veces para saber si se puede escribir ahí.

Y una corrección de accesibilidad que no viene del prototipo. El repaso con la base de reglas de UX —ver E-11 en `docs/trazabilidad/AI-USAGE-LOG.md`— encontró que **el campo rechazado por el servidor no estaba enlazado con el mensaje que lo explica**. En pantalla se veía el borde rojo y el texto debajo; lo que faltaba —`aria-invalid` y `aria-describedby`— no se ve. El RNF-07 compromete accesibilidad AA, y un motivo de error que sólo existe visualmente no la cumple.

Se resolvió con dos piezas en `Estados.tsx`: `ErrorDeFormulario`, que pone el aviso arriba del formulario y le da el foco al aparecer —cuando el motivo está al final, quien envía un formulario largo aprieta «Guardar», no pasa nada visible, y la explicación quedó tres pantallas más abajo—

, y `marcaDeError`, que enlaza el campo con su mensaje. Dos pruebas lo sostienen, comprobadas en rojo antes de darlas por buenas.

Incremento 5: el desglose del motor, y una tarjeta que se aplastaba.

El prototipo pone el desglose del riesgo como una banda de ancho completo debajo del encabezado de la ficha: una tarjeta por regla, con lo que aportó sobre lo que podía aportar y el motivo escrito. En el sistema vivía en la columna angosta de la derecha, como una tabla de dos columnas apretada y debajo de todo lo demás. **El motor predictivo es el centro del trabajo y estaba dibujado como un dato al margen.**

De paso cambia lo que cada regla dice de sí misma. Antes mostraba el porcentaje que aportaba sobre el total, y ese número se movía solo: si una regla dejaba de aplicar al equipo, las otras «subían» sin que hubiera cambiado nada. Ahora dice 20 / 30 pts, que es lo que aportó sobre su propio techo y no depende de las demás.

Y apareció un defecto de layout que no se veía como defecto. .cuerpo es un flex en columna: cuando el contenido no entra en la pantalla, reparte el faltante encogiéndolo a sus hijos. Una tarjeta lleva overflow: hidden para recortar sus esquinas redondeadas, y eso le da **mínimo automático cero**: se aplastaba a la altura de sus bordes. El desglose quedó con 2 px de alto y 400 px adentro, y las tablas de Activos e Incidencias venían recortadas al llegar al pie de la pantalla desde antes.

Lo que lo hace difícil de ver es que **la captura no se ve rota**: se ve una tarjeta vacía con su título, y entra al documento igual. Así que la guarda va donde único se puede: el script de capturas ahora mide, después de cada pantalla, si alguna tarjeta es más baja que su contenido, y termina en rojo si la hay. Se comprobó provocando el colapso a propósito: lo levantó en tres pantallas.

.filtros ya traía su propio flex: none por esta misma razón. Se resolvió para todos los hijos del cuerpo en vez de seguir tapando el agujero de a una pieza.

Incremento 6: los segmentos del inventario. El prototipo pone cuatro píldoras arriba de la tabla de activos. No son un filtro más por columna —de esos ya hay cuatro— sino las tres preguntas que alguien se hace parado frente al parque: **qué está por romperse, a qué le debo el preventivo, y qué ya no tiene garantía**, con su número al lado.

Es lo que trae el motor predictivo al inventario. Hasta ahora, para saber qué equipo estaba en riesgo había que ir a otra pantalla.

GET /api/equipos/segmentos devuelve los cuatro recuentos y el listado acepta segmento. Dos decisiones que no son obvias:

- **Los recuentos se recalculan con los demás filtros puestos**, y no son totales fijos del parque. El número al lado de la píldora es una promesa: si dice tres, al pulsarla tienen que aparecer tres. Con un total fijo, al filtrar por ubicación la píldora seguiría diciendo un número que no existe. La prueba que lo sostiene compara el recuento contra el total del listado del mismo segmento, que es donde las dos consultas se separan.
- **Un segmento desconocido se ignora en vez de rechazarse.** Llega por la dirección, así que puede venir de un enlace viejo; un 400 dejaría la pantalla sin inventario por un nombre mal escrito.

El estado activo va en `aria-pressed` y no sólo en el color, que es la misma regla del RNF-07 que ya aplica al resto de la interfaz.

Nota sobre el SQL. La primera versión contaba con `SUM(CASE WHEN EXISTS (...))`, que se lee bien y SQL Server no admite: no se puede agregar sobre una expresión que contiene una subconsulta. Las marcas se calculan por equipo en un `CROSS APPLY` y recién después se suman. Lo levantaron las pruebas del endpoint, no la compilación.

D-55 · Las figuras pierden el epígrafe y ganan texto alternativo

Estado: aplicado — vive en `docs/_md_a_docx.py`, así que alcanza a todos los documentos generados.

Qué pasaba. Cada figura llevaba debajo un epígrafe numerado —«Figura 12. Diagrama de despliegue»— y cada figura va, por construcción, debajo del título que la explica. El epígrafe repetía ese título con un número adelante.

Qué se hizo. Se sacó el epígrafe de las figuras. Los de las tablas quedan: una tabla no siempre está debajo de su título y su rótulo sí identifica qué contiene.

Por qué no rompe nada. Ningún texto del documento referencia una figura por su número —se verificó antes de tocarlo—, así que no quedan menciones a una «Figura 12» que ya no se numera.

Lo que se ganó de paso. El epígrafe no desaparece del documento: pasa al **texto alternativo** de la imagen, que es lo que lee un lector de pantalla. Antes la descripción estaba en la hoja y la imagen no tenía ninguna; ahora la imagen la tiene. Sacarlo de la vista no tenía por qué sacarlo del archivo.

D-56 · Los vínculos de los DER dejan de cruzar por encima de las tablas

Estado: aplicado — vive en docs/tecnico/_diagramas-ea.py.

Qué pasaba. Enterprise Architect dibuja los vínculos, por omisión, como una línea recta entre los centros de las dos cajas. En un DER de veinte tablas eso son diagonales que atraviesan todo lo que quede en el medio: en el diagrama de equipos, las relaciones de las tablas de catálogo del pie cruzaban por encima del texto de Equipo, PlanMantenimiento y AlertaPredictiva.

Qué se hizo. Los vínculos de los cuatro DER pasan a **ruteo ortogonal** (`DiagramLink.LineStyle`): los tramos son horizontales y verticales, salen por el borde de la caja y corren por los pasillos entre columnas. Y la separación entre tablas sube de 45 a 60 px, para que esos pasillos existan.

Lo que se probó primero y no era. `LayoutDiagramEx`, el reacomodado automático de EA. No funcionaba por dos motivos, y el segundo importa: se lo llamaba **antes** de `SaveDiagram`, que escribe los objetos que están en memoria —con las posiciones viejas— encima de lo recién calculado. Corregido el orden, tampoco convenía: reescribe dónde va cada caja, y esas posiciones están calculadas para que la figura entre en la caja de texto de la hoja a un tamaño en que el texto se lee, que es lo único que decide si el diagrama sirve impreso.

Lo que no resuelve, dicho para que no se lea como una promesa de más. EA no rutea esquivando obstáculos: ningún ajuste de estilo hace que un vínculo rodee una tabla. Quedan tramos verticales que pasan por encima de alguna caja. Para sacarlos habría que partir los DER en áreas más chicas, que es la misma decisión que ya se tomó una vez —el DER de negocio está partido en tres— y que tiene su costo: cuantas más áreas, más relaciones quedan cortadas por el borde.

Un defecto del generador, de paso. Si algo fallaba después de abrir EA por COM, la instancia quedaba huérfana —sin ventana, reteniendo el `.ldb` del modelo— y la corrida siguiente no podía borrar la carpeta de salida y moría en `os.makedirs`. Se llegaron a acumular tres. Ahora la carpeta se prepara antes de abrir EA y su creación tolera que ya exista.

D-57 · El documento del TFI también pierde los epígrafes de figura

Estado: aplicado — sobre docs/tfi/TFI-PredictIT.docx, que se mantiene a mano.

Por qué hace falta un delta aparte del D-55. El D-55 sacó los epígrafes de los documentos **generados**, tocando `_md_a_docx.py`. El documento del Trabajo Final no sale de ahí: se mantiene a mano desde el 7 de septiembre, así que sus 73 epígrafes de figura seguían puestos.

Y no se habían detectado antes porque el control que se hizo buscaba el texto «Figura 12» y no lo encontraba: Word numera los epígrafes con un **campo `SEQ`**, así que el número no está como texto en el XML. La verificación dio cero y la conclusión fue la equivocada. La comprobación buena es contar los campos, no el texto.

Qué se hizo. Fuera los 73 epígrafes de figura, y fuera el **índice de figuras**, que sin epígrafes no tiene de dónde sacar las entradas: un índice vacío en el documento entregado es peor que no tenerlo.

Qué se conserva. Los 98 epígrafes de tabla y su índice. Una tabla no siempre está debajo del título que la explica, y su rótulo sí identifica qué contiene.

Por qué no rompe nada. El documento no tiene una sola referencia cruzada a una figura —ni campos REF, ni «ver la figura N» en la prosa—, así que no quedan menciones a una numeración que ya no existe.

D-58 · Los diagramas del documento estaban cuatro versiones atrás

Estado: aplicado — sobre el mapa de docs/tfi/_diagramas-al-documento.py.

Qué pasaba. El script que mete los diagramas de Enterprise Architect en el documento trabaja con un mapa de sección a archivos. La sección 10.4.8 declaraba **tres** DER y la sección tiene **cuatro**: el de facturación se sumó con el módulo (D-46) y nunca se agregó al mapa.

Ante esa discrepancia el script no adivina: saltea la sección entera. Así que los cuatro DER del documento eran los de antes del rediseño —con las diagonales que cruzaban por encima de las tablas— mientras los archivos en disco ya estaban corregidos.

Qué se hizo. El DER de facturación entra al mapa, en la posición en la que efectivamente está en la sección: equipos, incidencias, facturación, servicio. Con eso los 47 diagramas del documento quedan en la versión actual.

Lo que esto vuelve a mostrar. Es la segunda vez que un mapa escrito a mano se queda atrás de lo que el sistema produce —la primera fueron las imágenes de los manuales—. El script avisa cuando el recuento no coincide, y ese aviso estaba saliendo en cada corrida: «la seccion tiene 4 imagenes y el mapa da 3».

Incremento 7: el tablero de incidencias. El prototipo ofrece dos vistas de la bandeja —lista y tablero— y hasta ahora sólo estaba la lista. El tablero es lo que contesta «dónde está parado el trabajo» sin leer una tabla de nueve columnas.

Las columnas salen del catálogo, no de una lista escrita en la pantalla. Los estados los define la base, vienen en el orden del ciclo, y `esFinal` es un dato del dominio —está así desde el principio, con el comentario «se guarda como dato y no se deduce del nombre: la organización puede renombrar sus estados»—. El catálogo no lo exponía, así que se agregó `EstadoIncidenciaDto`: si alguien escribiera las columnas a mano, una organización que agregue un estado tendría incidencias que no aparecen en ninguna, y eso no se ve porque el tablero se sigue dibujando bien.

Las tres terminales —resuelta, cerrada, anulada— se juntan en una columna: un tablero muestra dónde está el trabajo, y lo terminado no tiene tres lugares distintos donde estar.

Es de sólo lectura y no se arrastra, a propósito. Dos razones, y la primera es de diseño: el ciclo de vida lo valida la capa de negocio (ADR 0012) y no toda transición está permitida desde todo estado; arrastrar sugiere que cualquier movimiento vale y lo desmiente recién al soltar. La segunda es de accesibilidad: la WCAG exige una alternativa por teclado para toda acción de arrastre, y acá cada tarjeta ya es un enlace, que es accesible sin agregar nada.

Dos decisiones sobre cuánto mostrar. El tablero pide 200 y no 20: un tablero paginado esconde justo lo que se vino a mirar. Y si aun así se recorta, lo dice, porque un recorte silencioso hace parecer que hay menos trabajo del que hay. La columna de terminadas, en cambio, se corta en seis y avisa cuántas quedan: crece sin techo, nadie actúa sobre ella desde el tablero, y con veinte tarjetas aplastaba a las columnas que importan. El recuento del encabezado sigue siendo el real —veinte, no seis—, porque si dijera seis la columna estaría mintiendo sobre cuánto se cerró.

D-59 · La barra de color al costado de la tarjeta se va

Estado: aplicado

De dónde sale. Se pasó `impeccable` —Apache-2.0, detector de antipatronos visuales— sobre el frontend. Encontró **cinco hallazgos, y los cinco son el mismo**: `border-left: 3px solid` sobre una tarjeta, que su regla describe como *el tell más reconocible de una interfaz generada por IA*.

Estaban en las tarjetas de indicador del tablero, en el desglose del motor predictivo y en la primera celda del historial. Los tres venían del prototipo.

Qué se hizo.

- **Indicadores:** el tono pasa a un punto de color al lado del rótulo. La barra no aportaba nada que la cifra teñida no dijera ya, y la cifra es lo que se mira.

- **Desglose del motor:** la regla que llegó a su techo se distingue por el borde completo de la tarjeta —no por una franja a un lado— y por el color de su cifra.
- **Historial:** la franja de la primera celda se elimina sin reemplazo. No codificaba nada: era del mismo color para todas las filas.

Lo que se conserva y por qué. El `border-left` del aviso de error se queda. Usa `currentColor` y no un acento decorativo, y una regla vertical al costado de un mensaje de alerta es una convención tipográfica anterior a las interfaces, no un adorno. El detector tampoco lo marca.

El resultado. El detector pasa sin hallazgos. Las tres correcciones mantienen el contraste AA: el script lo verifica y sigue en verde.

D-60 · Dos defectos del tablero, encontrados revisando lo recién escrito

Estado: aplicado

Una revisión del código del día encontró dos, encadenados, en la vista de tablero:

Limpiar los filtros achicaba el tablero. `limpiar()` reescribía el filtro entero con el tamaño de página de la lista, sin mirar qué vista estaba activa. Estando en el tablero, la vista seguía marcada como tablero y la consulta pasaba a pedir veinte en vez de doscientas: las incidencias que no entraban desaparecían de las columnas sin que nada lo explicara.

Y el aviso de recorte declaraba un número que no era. El texto interpolaba la constante `POR_TABLERO` en vez de cuántas trajo la respuesta. Encadenado con lo anterior, el pie decía «el tablero muestra las 200 más recientes de 27»: un número mayor que el total y distinto de lo que había en pantalla.

Los dos están cubiertos por pruebas comprobadas en rojo antes de darlas por buenas. La prueba que ya existía sobre el aviso estaba escrita contra el comportamiento equivocado —esperaba el «200»— y se corrigió: ahora afirma que el número es el de las tarjetas efectivamente dibujadas.

D-61 · Todo el paquete toma el formato del trabajo de 3.º año

Estado: aplicado — en `docs/_md_a_docx.py` para los generados, y sobre el `.docx` del TFI, que se mantiene a mano.

De dónde sale el formato. De leer los estilos de Documentacion Trabajo de diploma - Pettini Patricio.docx, el trabajo que la cátedra ya evaluó, y no de mirarlo a ojo. Lo que se tomó:

	Valor
Página	Carta, 21,59 × 27,94 cm
Márgenes	2,54 cm los cuatro
Cuerpo	Calibri 12
Títulos	Calibri Light, azul #2F5496 — 20 pt el nivel 1, 16 pt el 2
Subtítulo	14 pt, #595959

Una corrección de lectura, porque casi entra mal. El documento declara un estilo Body Text en Times New Roman 12 y era lo primero que aparecía al listar los estilos. Pero lo usan **seis** párrafos: el cuerpo real son 12.610 caracteres en Calibri 12. Contar el uso y no leer la tabla de estilos es lo que lo evitó.

Qué cambió de lo que había. El acento de los títulos era un verde petróleo elegido para este trabajo; pasa al azul de 3.º. Los márgenes bajan de 3 cm a 2,54, así que la caja de texto crece 1,5 cm de ancho. Y el documento del TFI pasa de A4 a Carta.

Se comprobó antes de cambiar la hoja. La caja de Carta con esos márgenes da 22,86 × 16,51 cm; la imagen más alta del documento mide 18,80 y la más ancha 15,01. Ninguna de las 74 se recorta. El documento crece algunas páginas porque la hoja es 1,84 cm más corta, y nada más.

Y el azul no alcanzaba con declararlo. Los títulos del TFI y los del trabajo de 3.º heredaban el color del tema de Word, y les tocaba un tema distinto: 0F4761 contra 2F5496. Se fija explícito en el estilo, para que no dependa de qué tema traiga el archivo.

Alcance. Los 33 documentos de ENTREGABLES y docs, verificado uno por uno contra los estilos de la referencia. ENTREGA FINAL no es una carpeta aparte sino una copia, así que se rehace con las otras dos.

D-62 · Fuera también los epígrafes de tabla

Estado: aplicado

El D-57 sacó los 73 epígrafes de figura del documento del TFI y dejó los de tabla, con el argumento de que una tabla no siempre está debajo del título que la explica. El criterio cambió: se van los **98** y su índice, igual que con las figuras.

Los 33 documentos entregables quedan sin un solo epígrafe, verificado contando los campos SEQ y no el texto — que es el error que hizo fallar la primera comprobación del D-57.

D-63 · Las referencias pasan de APA 6 a APA 7

Estado: aplicado

El paquete decía «normas APA» y estaba escrito en la 6.^a edición, que es lo que circula en los apuntes. La 7.^a es la vigente desde 2020 y se separa de la anterior en tres marcas literales, que es justamente lo que hace que un jurado lo note leyendo una línea:

- **«Recuperado de» / «Disponible en» delante de la URL.** La 7.^a los sacó: la URL va sola. «Recuperado el {fecha} de» sobrevive sólo para fuentes que cambian sin dejar versión archivada, y ninguna de las 23 lo es. Había 14.
- **El punto que cierra el elemento autor.** Anthropic (2025) pasa a Anthropic. (2025). Afectaba a los diez autores institucionales.
- **Mayúscula de oración en los títulos,** no mayúscula en cada palabra. Los nombres de publicaciones periódicas la conservan, por eso *IEEE Software* no cambia.

En el capítulo «Fuentes» del documento principal, además, la lista estaba numerada y en orden de aparición. Una lista de referencias no se numera ni se viñetea: va con sangría francesa y ordenada alfabéticamente por autor, que es como ya salían las de los documentos satélites.

Alcance. Es la decisión de aplicar APA **sólo a las citas y las fuentes**, no al cuerpo: el formato del texto sigue siendo el del trabajo de 3.^º año del D-61.

El control «Referencias en APA 7» de `_verificar-alineacion.py` mide las tres marcas y el orden alfabético. Se probó en rojo antes de darlo por bueno.

D-64 · La línea del encabezado se cortaba justo arriba del escudo

Estado: aplicado

En el PDF, la línea que separa «UNIVERSIDAD ABIERTA INTERAMERICANA» de la fila de abajo aparecía cortada sobre la columna del escudo, y sólo ahí. En el Word se veía entera.

No era el PDF ni el visor. Los bordes de esa tabla estaban en `w:sz="4"`, que son cuatro octavos de punto: **media línea de 0,5 pt.** Word exporta cada tramo de borde como un rectángulo relleno independiente, uno por celda, así que el tramo de la columna del escudo tiene sus propios límites verticales; cuando el visor redondea a píxeles enteros, ese tramo puede quedarse en cero mientras los de al lado sobreviven.

Medido sobre el PDF: al 150 % desaparecían 86 pt de línea, entre los 42,0 y

los 128,0 pt de la página, y la columna del escudo va de 36,5 a 142,3. Al 100 % y al 200 % se veía entera, que es lo que hacía parecer que el problema era del visor.

El resto de las tablas del documento usa `w:sz="8" —1 pt—` y por eso ninguna hace esto. El encabezado pasa al mismo grosor: 0,96 pt en el PDF y cero píxeles faltantes al 100, 125, 150, 175 y 200 %.

D-65 · Las 24 capturas del 10.5 eran anteriores al rediseño del frontend

Estado: aplicado

Comparadas byte por byte contra `docs/capturas/`, **ninguna de las 24 coincidía**. El capítulo de pantallas mostraba el frontend previo al D-54: sin la marca en la barra lateral, sin la tarjeta de organización con el plan y el consumo, sin las insignias de la navegación y sin el panel de riesgo del tablero. Los 50 diagramas de Enterprise Architect, en cambio, estaban todos al día.

Lo natural habría sido correr `_formato-y-capturas.py`, que es de donde sale el mapeo de apartado a captura. No se puede: ese guion además cambia los diagramas por los que genera el guion viejo y vuelve a numerar figuras y tablas, y las dos cosas desharían el trabajo del D-56 y devolverían los epígrafes del D-57 y el D-62. Por eso está fuera de la lista del armado.

`docs/tfi/_capturas-al-dia.py` reemplaza **sólo los bytes** de cada imagen y deja en su lugar el anclaje, el tamaño dibujado, el texto alternativo y el párrafo. Es idempotente y entró a la lista del armado.

Por qué nadie se enteró. El archivo estaba ahí, con su nombre de siempre, y el documentoabría igual: no había ninguna señal. El control «Figuras de los documentos al día» la crea, comparando por contenido y no por nombre ni por fecha: si un `.png` de `capturas/` o de `diagramas-ea/` no está dentro de ningún documento, es que el documento que lo mostraba se quedó con la versión anterior.

D-66 · El diagrama general de casos de uso, rehecho

Estado: aplicado

La figura de 10.4.3 tenía dos problemas, y sólo el segundo se veía.

Estaba atrasada. Mostraba una versión que no coincidía con ninguna de las dos que el proyecto genera: ni con el modelo hecho a mano —trece casos y un actor «Sistema»— ni con los dieciocho diagramas individuales de 10.4.3.x, que usan los cuatro perfiles reales. Era una

imagen huérfana, pegada en su momento y nunca vuelta a tocar, y por eso ninguna comprobación la alcanzaba: no era igual a ningún archivo del repositorio.

Y era ilegible. Los dieciocho casos se ubicaban por número, sin relación con quién los ejecuta, así que las cuarenta y tantas asociaciones cruzaban la figura entera y pasaban por encima de las elipses.

La versión nueva pone **los actores en el centro y los casos a los dos lados**, agrupados por perfil: los exclusivos a la altura de su actor y los compartidos entre los dos que los comparten. Los dos pasillos que quedan —entre cada columna de casos y la columna de actores— son por donde corren todos los vínculos, y por eso ninguno pasa por encima de un texto. Se cruzan entre sí, que con seis casos compartidos por dos perfiles es inevitable.

De paso apareció una divergencia real. Los actores de cada caso estaban escritos dos veces — en el diagrama general y en los individuales— y ya habían empezado a diferir: el general daba CU-004 a dos perfiles y el individual a tres. Ahora los dos salen de la misma tabla. Y el perfil pasa a llamarse **Usuario Solicitante** en los diecinueve diagramas, que es como lo nombra la prosa del documento.

El diagrama se rehace dentro del modelo que se entrega, no en el script: el .eap es el artefacto, y un diagrama que sólo existe en la imagen exportada no se puede abrir en la defensa.

D-67 · El paquete deja de tener figuras chicas y tablas mal cortadas

Estado: aplicado

Dos defectos de presentación, los dos medidos antes de tocar nada.

Un tercio de cada diagrama era papel en blanco. Enterprise Architect no exporta el dibujo: exporta la **hoja**, y la hoja tiene un tamaño mínimo propio. Un caso de uso con dos actores ocupa 614 px de ancho y salía en un lienzo de 871, o de 1053. Treinta y uno de los cincuenta diagramas tenían entre el 30 % y el 46 % de margen derecho vacío.

Eso no es estético: la figura entra al documento con un ancho fijo, así que ese margen se llevaba la mitad del ancho útil y el dibujo se imprimía a la mitad del tamaño que podía. El recorte respeta el marco de EA en lugar de sacarlo —se reubica la tira de píxeles del borde, así que el rectángulo no queda abierto de dos lados— y el tamaño con que cada figura está dibujada en el documento se recalcula conservando el ancho, que es lo que hace que el recorte se traduzca en dibujo más grande y no en figura más chica.

Y ninguna tabla deja ya una sola fila de un lado del salto de página. Partida por la mitad está bien; una fila huérfana, no. Word no trae esa regla: sabe repetir el encabezado y sabe no partir

una fila, pero deja sin problema el encabezado solo al pie de una hoja o la última fila sola arriba de la siguiente. Se resuelve con tres marcas —`cantSplit` en cada fila, `tblHeader` en la de encabezado, y «conservar con la siguiente» en la primera y en la anteúltima—, aplicadas a las 98 tablas del documento principal y, desde el generador, a las del resto del paquete.

Lo que esto destapó. Tres imágenes del documento no coincidían con su archivo aunque fueran la misma figura: Word las había vuelto a comprimir al guardar. Por eso la correspondencia entre imagen y diagrama ya no se deduce del contenido sino que está escrita en `docs/tfi/diagramas-del-documento.json`, y el guion avisa si una entrada deja de existir.

D-68 · Ningún vínculo pasa ya por encima de una entidad

Estado: aplicado

El modelo de dominio tenía una asociación que atravesaba Incidencia de punta a punta: Equipo estaba arriba, AlertaPredictiva abajo, y las tres filas apiladas en tres columnas dejaban a Incidencia justo en el medio.

El defecto se corrigió, pero lo que importa es cómo se buscaron los demás. **No se miraron las imágenes: se midió sobre el modelo.** Cada objeto de cada diagrama tiene su rectángulo y cada vínculo sus dos extremos, así que alcanza con preguntar si el segmento que une los dos centros corta el rectángulo de un tercero —recorte de Cohen-Sutherland—, con dos refinamientos: de un caso de uso se mide la **elipse inscripta** y no su caja, que es bastante más grande, y de un actor sólo el tercio central, que es lo que ocupa el monigote.

Sobre 53 diagramas medidos aparecieron 81 vínculos por encima de una caja ajena.

Lo que se corrigió moviendo cajas:

- *Modelo de dominio:* los cuatro hijos de Equipo pasan a una sola fila debajo de él, con Equipo centrado encima. Cada vínculo es una diagonal corta.
- *Paquetes y dependencias:* PredictIT.Service sube a la altura de la BLL, a su derecha. La dependencia de la Api hacia Service pasaba justo por encima de la BLL, que es la capa que el diagrama tiene que mostrar en el medio.
- *Ciclo de vida de la incidencia:* Anulada sube a la altura de Asignada. Las transiciones que venían de Pendiente y de Asignada cruzaban Espera de repuesto.
- *Casos de uso por perfil:* las dos columnas de casos pasan a zigzag y CU-001 —el único que usan los cuatro perfiles— se corre al lado de la columna de actores.

Lo que se corrigió con ruteo ortogonal: los tres DER hechos a mano y los tres

diagramas de clases del modelo. El D-56 había puesto ortogonal en los generados y en los hechos a mano habían quedado vínculos en línea directa. Es la misma decisión, aplicada donde faltaba.

Y lo que no se pudo corregir, dicho como es. En el diagrama general de casos de uso quedan cuatro vínculos que rozan una elipse. No es descuido: con dieciocho casos, cuatro actores y seis casos compartidos por dos perfiles, un actor que conecta con el primero y con el último de una columna pasa por encima de lo que quede en el medio, y **no existe ubicación que lo evite** con líneas rectas. Se probó ruteo ortogonal y quedó peor: EA hace correr los tramos verticales justo por encima de la columna de elipses. Los cuatro que quedan rozan el borde, no el texto.

El guion docs/tecnico/_verificar-diagramas.py deja la medición corriendo, y declara aparte lo que **no** mide —los diagramas de secuencia, donde los mensajes no van de centro a centro, y los vínculos en ruteo ortogonal, donde el recorrido lo calcula EA— en lugar de contarlos como aprobados.

D-69 · Qué archivo es el último, después de un `git clone`

Estado: aplicado

El repositorio tiene los documentos dos veces: la fuente en docs/ y la copia que se entrega en ENTREGABLES/. Después de clonar no había forma de saber cuál mirar, ni si el que se abrió era el de la última corrida o quedó de una anterior. Las fechas de archivo no ayudan: `git clone` le pone a todo la fecha en que se clonó.

ENTREGABLES/VERSIONES.md se genera con el armado y lista los 58 archivos con su **sha-256** y el commit del que salieron. El sha es lo único que sobrevive a un clon y a una copia: dos archivos con el mismo sha son el mismo archivo, y si difiere del que figura ahí, el archivo no es el de esa versión. El README lo dice arriba de todo, con el enlace.

D-70 · El sistema pasa a estar desplegado y accesible

Estado: aplicado

El documento describe el entorno de ejecución como una instalación propia. Ahora el sistema además **corre en <https://predictit-web.fly.dev>**, para que se pueda probar sin instalarlo.

Por qué Fly y no Vercel ni Netlify. Los dos sirven archivos estáticos y funciones sin estado: no corren un proceso ASP.NET Core de larga vida —el motor predictivo es un servicio alojado que se despierta cada 24 horas, ADR 0011— ni alojan SQL Server. Servirían para el frontend y nada

más. Fly corre los contenedores que el proyecto ya tenía, así que el sistema subió **sin cambiar una línea de código de la aplicación**.

Lo único que se tocó fue el destino del proxy de nginx, que estaba escrito a mano como `api:8080` —el nombre del servicio en Docker Compose—. Pasa a salir de una variable de entorno, con ese mismo valor por omisión: la misma imagen sirve para las dos cosas.

Una sola superficie expuesta. De las tres aplicaciones, sólo el frontend tiene dirección pública; la API y la base viven en la red privada de Fly, sobre WireGuard, y no tienen IP. Swagger sale por el mismo proxy, así que la documentación de la API queda disponible para quien evalúa sin abrir el backend.

SQL Server Express y no Developer. El contenedor de desarrollo usa Developer, que es gratis pero cuya licencia permite sólo desarrollo y prueba. Un sistema al que entra cualquiera desde internet no es ninguna de las dos, así que en producción va Express, que sí está licenciada para uso productivo. El límite —10 GB por base— sobra, y lo único que Express no trae y podría importar, el Agente de SQL Server, no se usa: el motor se dispara desde la aplicación.

Los secretos son otros que los de desarrollo, a propósito: si el `.env` local se filtra, no abre lo que está publicado. Viven en `fly secrets`, cifrados, y ninguno está en el repositorio.

El detalle del despliegue, con lo que hay que saber para rehacerlo, está en `fly/LEEME.md`.

D-71 · El entorno de demostración no es la infraestructura prevista

Estado: aplicado

El D-70 dejó el sistema corriendo en Fly.io, y el capítulo 10.2.5 dice «Contenedores Docker; **despliegue previsto** sobre Microsoft Azure». Las dos cosas conviven, pero hay que decirlo o la pregunta aparece sola apenas alguien abre la dirección.

No son lo mismo y el documento no se contradice. «Previsto» es un plan de infraestructura para el servicio productivo, no una afirmación de que el sistema ya esté alojado ahí. Lo que hay hoy es un **entorno de demostración**, que existe para que se pueda probar el sistema sin instalarlo.

Lo que **no** cambia, y por eso el plan de negocio queda intacto: la infraestructura prevista para producción sigue siendo Microsoft Azure. De ahí salen el costo de infraestructura por cliente del Plan Inicial, el criterio de localización —«priorizando la latencia y la disponibilidad para usuarios ubicados en Argentina»— y la responsabilidad del CTO sobre esa infraestructura. Mover la demostración a otro proveedor no toca ninguna de las tres.

Por qué se puede cambiar de proveedor sin cambiar el sistema. Es la consecuencia de una decisión que ya estaba tomada: el sistema se empaqueta en contenedores y toda su configuración —cadenas de conexión, claves, orígenes permitidos— entra por variables de entorno. Subirlo a Fly no necesitó una sola línea de código de la aplicación; lo único que se agregó fue que el destino del proxy de nginx saliera de una variable, porque estaba escrito a mano.

Qué haría falta para que la demostración corra en Azure. Una máquina virtual chica con docker compose up, que es exactamente lo que el entorno técnico describe. **No** sirve Azure SQL Database: el servicio de respaldos hace BACKUP y RESTORE DATABASE —CU-017 y Req. Arq. 003— y ese servicio gestionado no lo permite, porque los respaldos son automáticos y no los controla la aplicación. Montarlo ahí costaría un caso de uso ya entregado y probado.

Sobre la suscripción de Azure. Se activó Azure for Students con la cuenta institucional: USD 100 de crédito, vigentes hasta el 16/09/2027. El primer intento por línea de comandos parecía indicar que el convenio de la universidad no la habilitaba —el portal de la oferta devolvía a la página de presentación—, pero era el flujo del portal y no una restricción: entrando por «Education → Azure for Students» desde el portal, la suscripción se creó.

Queda entonces la posibilidad de mover el entorno de demostración a la infraestructura que el documento prevé. Lo que no cambia es el motivo por el que se puede: contenedores y configuración por variables de entorno.

D-72 · El entorno de demostración pasa a Microsoft Azure

Estado: aplicado

El D-70 lo había puesto en Fly.io y el D-71 registró que eso era el entorno de demostración y no la infraestructura prevista. Activada la suscripción de estudiante, se movió a **Microsoft Azure**, que es lo que el capítulo 10.2.5 declara: «Contenedores Docker; despliegue previsto sobre Microsoft Azure». Las aplicaciones de Fly se eliminaron.

<https://predictit-uai.mexicocentral.cloudapp.azure.com>, sobre una máquina Standard_B21s_v2 con docker compose. El sistema subió **sin cambiar una línea de código**: es el mismo stack que se levanta en cualquier máquina con Docker.

La región es lo único que no sale como el documento lo pide, y conviene decirlo antes de que lo pregunten. El documento fija que la localización «debe seleccionarse priorizando la latencia y la disponibilidad para usuarios ubicados en Argentina», y para eso la región correcta es Brazil South. La suscripción de estudiante tiene una directiva —«Allowed resource deployment

regions»— que sólo habilita cinco regiones, ninguna en Sudamérica. Se eligió la más cercana de las disponibles, Mexico Central.

Medido desde Buenos Aires: **231 ms** para establecer la conexión, contra 403 ms de North Central US y 413 ms de Canada Central. El criterio se aplicó sobre las opciones que había. Para referencia, el despliegue anterior en São Paulo daba 96 ms: la limitación es de la suscripción, no del diseño, y en producción con una suscripción sin esa restricción se usaría Brazil South.

Lo que se estrenó. `frontend/nginx-tls.conf` estaba escrito desde hacía semanas y nunca se había usado. Decía de sí mismo que existía «para que poner el sistema detrás de TLS sea montar dos archivos y cambiar una línea, y no un trabajo de investigación el día del despliegue». Fue exactamente eso: el certificado de Let's Encrypt entró montando ese archivo y los dos del certificado.

Lo que hubo que corregirle al sistema. Una sola cosa, y la encontró el despliegue: el primer certificado se pide **sin** TLS, así que quien responde el desafío de validación es `nginx.conf.template`, que no tenía la ubicación `/.well-known/acme-challenge/`. La SPA se comía la petición y la autoridad certificadora recibía el HTML de la aplicación en lugar del testigo. Se le agregó, y ahora las dos configuraciones —con y sin TLS— pueden renovar.

El detalle operativo está en `despliegue/LEEME.md`.

D-73 · El plan de negocio vendía cuatro capacidades y el sistema tiene seis

Estado: aplicado — secciones 1.7 (capacidades centrales) y 1.8 (propuesta de valor)

Medido contando menciones por capítulo, el desvío era objetivo: «guía de reparación», «plan de mantenimiento» y «mantenimiento preventivo» aparecían **dos veces cada uno en el capítulo 10 y cero veces en los capítulos 1 a 9**. La única mención de la guía de reparación en toda la documentación de negocio estaba en la sección de costos, como consumo de inteligencia artificial: el plan la pagaba y no la vendía.

Son las dos piezas que convierten la detección en acción. El motor detecta el riesgo y recomienda intervenir; sin plan de mantenimiento no hay con qué actuar, y sin guía de reparación el técnico que recibe la incidencia —en el Segmento A suele ser una persona sola, muchas veces sin formación específica— sigue igual de solo que antes. Es lo que justifica el precio, y no estaba escrito.

Se agregaron como quinta y sexta capacidad central en 1.7, y 1.8 pasó de «la combinación de cuatro elementos» a seis. Lo escribe `docs/tfi/_capacidades-de-negocio.py`, que es idempotente y tiene `--verificar`.

D-74 · El documento saltaba del capítulo 7 al 10: se escriben el 8 y el 9

Estado: aplicado — capítulos 8 (PLAN FINANCIERO-ECONÓMICO) y 9 (FACTIBILIDADES)

El índice de la cátedra pide un capítulo 8 con doce apartados y un capítulo 9 de factibilidades. El documento no tenía ninguno de los dos: **saltaba del 7 al 10**, y todo el contenido financiero vivía dentro del capítulo 7 como 7.7, 7.7.1, 7.8, 7.8.1 y 7.8.2. Un evaluador que abriera el índice buscando «8.3 Modelo de Inversión» no lo encontraba, aunque el modelo estuviera calculado en el libro y explicado en el documento.

Capítulo 8. Se abre donde empezaba 7.7 y el capítulo 7 termina en 7.6, como en el índice. Ocho apartados se arman moviendo contenido que ya existía —los subtítulos en negrita «Ingresos mensuales estimados por etapa» y «Presupuesto mensual estimado por etapa» pasan a ser 8.1 y 8.2— y **cuatro se escribieron desde cero**: 8.4 amortizaciones, 8.6 ingresos brutos, 8.7 remuneraciones y 8.12 plan de salida. Los números no se inventaron: salen del libro de cálculo ya verificado.

El 8.12 no estaba pensado y se propuso: continuidad con distribución de resultados desde el cuarto ejercicio, venta a un integrador **valuada con el mismo múltiplo de ingresos recurrentes que el valor terminal de 8.9** —para que el precio de salida sea consistente con la evaluación y no un número aparte—, y cierre ordenado con preaviso, exportación de los datos en formato abierto y cancelación sin penalidad. Esa tercera obligación no es una concesión: el sistema administra información que la organización necesita para operar.

Capítulo 9. Es el único capítulo que no aporta contenido nuevo: cruza lo ya escrito y responde una sola pregunta por apartado —qué tendría que ser cierto para que el proyecto no se pudiera hacer—. Seis apartados: técnica, económica y financiera, operativa, legal, ambiental y la conclusión, con un cuadro que a cada factibilidad le pone su condición al lado. La conclusión que sostiene el capítulo es que **la restricción del proyecto no es técnica sino financiera y de adopción**, situación inversa a la habitual en un trabajo de ingeniería: el sistema está construido, desplegado y medido, y lo que falta demostrar es que hay PyMEs dispuestas a pagarlo.

Dos cosas que 9.4 declara y que antes no estaban escritas en ninguna parte: que el modelo financiero **supone otorgada** la adhesión al régimen de la Ley 27.506, que no es automática y cuyo beneficio está cuantificado en 8.5; y el encuadre en la Ley 25.326, donde la empresa actúa como encargada del tratamiento por cuenta del cliente.

Lo escriben `docs/tfi/_capitulo-8-financiero.py` y `docs/tfi/_capitulo-9-factibilidades.py`, los dos idempotentes y con `--verificar`. Quedan en la lista de `_armar-entregables.py`, antes del paginado de tablas, porque agregan tablas y es el paginado el que les pone la regla de corte.

Dos defectos que dejó la primera corrida y se corrigieron. El texto de 8.12 traía marcas `**` de Markdown, que Word imprime tal cual: ahora los tramos se escriben como negrita real y `--verificar` falla si aparece un `**` suelto. Y la referencia cruzada del capítulo 6 a «la sección 7.7» apuntaba a una sección que ya no existe; ahora apunta a 8.2, que es donde está la estructura de costos por etapa. Lo marcó en rojo el control de referencias cruzadas.

D-75 · La estrategia de producto se contradecía consigo misma

Estado: aplicado — secciones 6.4.1 (qué es y qué no es) y 6.4.3 (decisión de compra)

Dos frases del mismo apartado decían cosas distintas. El primer párrafo de 6.4.1 afirmaba que la inteligencia artificial interviene en «dos puntos del ciclo de la incidencia» y los nombraba: clasificación y asignación. Cuatro párrafos más abajo, la delimitación de lo que el producto no es decía que «esa intervención abarca tres funciones acotadas» e incluía la guía de reparación. Ganan las tres, porque es lo que el sistema hace: RF-15 está implementado y probado. El que sobraba era el «dos».

Tres omisiones más, todas en el mismo apartado:

- **El Partner.** 6.4.1 enumeraba los perfiles y nombraba tres. El Partner es el cuarto y es el perfil del Segmento C, que tiene su propio plan comercial y su propia columna en el modelo de ingresos. Omitirlo en la descripción del producto dejaba un segmento entero sin sustento en el producto que lo atiende.
- **El mantenimiento preventivo (RF-16),** que es la quinta capacidad central desde el D-73 y no aparecía en el capítulo de marketing.
- **La facturación del servicio (RF-17).** El sistema emite el comprobante mensual de cada organización a partir del plan contratado y de los equipos administrados al cierre del período. Es la contrapartida operativa del modelo de ingresos del capítulo 8: el esquema de abono más equipos adicionales no es una intención del plan comercial, es lo que el sistema liquida. Se agregó además a la lista de lo que el producto **no** es, para que no se confunda con un sistema de facturación del cliente.

En 6.4.3, la lista de características que influyen sobre la decisión de compra no mencionaba ninguna de las dos capacidades nuevas —justo las que separan la propuesta de un inventario con tickets—. Se agregó una viñeta: asistencia durante la resolución y no sólo diagnóstico.

Lo escribe `docs/tfi/_estrategia-de-producto.py`, idempotente y con `--verificar`, que falla si vuelve a aparecer el «dos puntos».

D-76 · La meta de equipos quedó en dos números distintos dentro del documento

Estado: aplicado — secciones 5.2.3 (metas cuantitativas) y 2.2.1.1 (cuadro comparativo)

El D-56 bajó la meta del mes 12 de **1.000 equipos a 450** y dejó escrito por qué: 1.000 equipos entre 25 clientes son 40 por cliente, y con 40 cada cliente pagaría 25 adicionales a \$ 4.200 sobre un abono de \$ 89.000 — el plan quedaba vendiendo adicionales y llamándolo suscripción.

Esa corrección se aplicó a la prosa y no a la tabla. Las secciones 6.1, 6.2 y 8.8 dicen 450; la tabla de 5.2.3 seguía diciendo 1.000, y 5.2.3 es justamente la sección que el resto del documento cita como fuente de las metas —«conforme a las metas cuantitativas de la Etapa E3»—. El libro de cálculo, mientras tanto, mide contra 450 y proyecta 461.

La meta de la etapa E2 arrastraba el mismo error sin que nadie lo hubiera mirado: 350 equipos entre 10 clientes son 35 por cliente. Con el parque de 18 a 24 equipos que el propio documento le asigna al Segmento A, 10 clientes administran unos 200. El modelo proyecta 209. Pasa a 200.

Y la fila de la etapa E1 —cien equipos con cero clientes pagos— **estaba bien y parecía mal**, que a los efectos de quien corrige es lo mismo: son los equipos de los cinco pilotos gratuitos. Se agregó la aclaración debajo de la tabla, junto con la regla de derivación, para que la meta de equipos no parezca fijada aparte de la de clientes.

El cuadro comparativo de competidores declaraba el precio «a definir». Los precios están definidos desde 5.2.6 y el capítulo 8 entero se apoya en ellos. Dejar «a definir» en la única fila que describe la propuesta, en una tabla donde los ocho competidores sí tienen su modelo de costo declarado, es perder la comparación por no completarla. Pasa a «Suscripción mensual en ARS, desde \$ 89.000».

En el mismo apartado, el párrafo que resume el diferencial enumeraba activos, incidencias, historial y predicción, sin nombrar la guía de reparación asistida —que es la única capacidad que ninguna de las ocho alternativas relevadas ofrece—. Es el mismo hueco que el D-73 cerró en 1.7 y el D-75 en 6.4.

Lo escribe `docs/tfi/_metas-y-competencia.py`, idempotente y con `--verificar`.

D-77 · El capítulo 9 prometía algo que la pantalla declara fuera de alcance

Estado: aplicado — secciones 9.1, 9.3 y 6.4.3

Escrito el capítulo 9, se contrastaron sus afirmaciones una por una contra las fuentes que cita. Tres no daban.

La importación desde planilla, que es la más grave. El apartado 9.3 decía que el plan responde al problema de la carga inicial «por tres vías, y las tres están en el producto y no en el discurso comercial», y una de las tres no está: el botón «Importar desde planilla» existe en la pantalla de Activos y está **deshabilitado**, con el cartel «Fuera del alcance de esta versión: la carga masiva necesita una plantilla y una previsualización que no están definidas».

El sistema es más honesto que el documento, que es la peor forma de que no coincidan: el profesor que haga clic en ese botón encuentra la contradicción antes de terminar de leer el capítulo. Y la sección 8.11 ya la trataba bien, como la primera función que se adelanta si la demanda supera la capacidad.

9.3 pasa a decir lo que hay: dos vías que existen —el acompañamiento del responsable de soporte y la guía de reparación— y una tercera priorizada que todavía no. Y 6.4.3, que la ofrecía como característica que influye sobre la decisión de compra, deja de ofrecerla hasta que exista: es la diferencia entre un argumento de venta y una promesa incumplible.

El rango de casos de prueba. 9.1 decía «CP-01 a CP-15». Hay diecisiete, y los dos que faltaban son justamente los de las capacidades que el D-73 y el D-75 acaban de incorporar al plan: CP-16, el plan de mantenimiento define el vencimiento, y CP-17, facturación del período y emisión del comprobante. El capítulo entregaba de menos.

El resultado de la revisión OWASP. 9.1 nombraba «una revisión sobre las diez categorías» sin decir cómo dio, lo que se lee como si las diez estuvieran limpias. Dos no lo están: A07 no exige segundo factor de autenticación y A08 no firma los artefactos de compilación. Ninguna impide poner el servicio en funcionamiento, pero un capítulo de factibilidad que enumera lo que dio bien y omite lo que dio parcial deja de ser una verificación.

De paso, la propia revisión OWASP declaraba el TLS «configurado y no activo en la demostración», que era cierto hasta el D-72: desde el despliegue en Azure está activo con certificado de Let's Encrypt. Corregido en docs/tecnico/REVISION-OWASP.md, junto con el rango de casos de prueba del texto de la entrega.

Lo escribe docs/tfi/_capitulo-9-al-dia.py, idempotente y con --verificar.

D-78 · El capítulo 9 tenía seis apartados y el índice de la cátedra pide cuatro

Estado: aplicado — capítulo 9 completo

El capítulo 9 se escribió con la división habitual de la bibliografía —técnica, económica y financiera, operativa, legal, ambiental y una conclusión— porque el índice de la cátedra mostraba «9 FACTIBILIDADES» sin desglosar. Con el desglose a la vista, son otros cuatro: **Técnica, Comercial, Administrativa y Legal**.

No es una diferencia de nombres, y eso es lo que hacía que la primera versión no sirviera aunque estuviera bien escrita:

- La **económica y financiera** no está porque ya es el capítulo 8 entero. Tenerla como apartado del 9 era decir dos veces lo mismo.
- La **comercial** faltaba, y es la que más importa: si hay quien lo compre, a ese precio, y si se puede llegar hasta él. Ese apartado no existía en ninguna parte del capítulo.
- La **administrativa** no es la operativa. Pregunta si la organización —la sociedad, sus roles y su gente— puede sostener la operación; si el cliente puede usar el producto es una cuestión comercial, porque es donde una falla de adopción se manifiesta. La primera versión las tenía mezcladas en un solo apartado.
- La **ambiental** no se pide. Lo único de ese análisis que era una condición y no un comentario —que la actividad no requiere habilitación ambiental ni estudio de impacto— quedó dentro de la legal, que es donde corresponde una exigencia normativa.

El cuadro final se conserva **sin numerar**, como cierre del capítulo: es lo que permite leer las cuatro condiciones juntas y no agrega un apartado al índice.

La conclusión también cambió al reordenarse. Antes decía que la restricción del proyecto «no es técnica sino financiera y de adopción», que era una forma de nombrar dos cosas porque no había dónde poner ninguna de las dos. Ahora dice lo que el índice permite decir: **la restricción es comercial**, y el capital lo sostienen los socios, que es una condición administrativa y está en 9.3.

El texto vive en `docs/tfi/_capitulo_9_contenido.py` y lo escribe `_capitulo-9-factibilidades.py`, que detecta la estructura de seis apartados y rehace el capítulo entero en lugar de reacomodarlo: mover apartados con contenido es lo que dejó el capítulo 8 fuera de orden en su primera corrida.

Fuentes

ISO/IEC. (2011). *Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models* (ISO/IEC 25010:2011).

Schwaber, K. y Sutherland, J. (2020). *La guía de Scrum*. <https://scrumguides.org>